

酷遊 學習指南文件

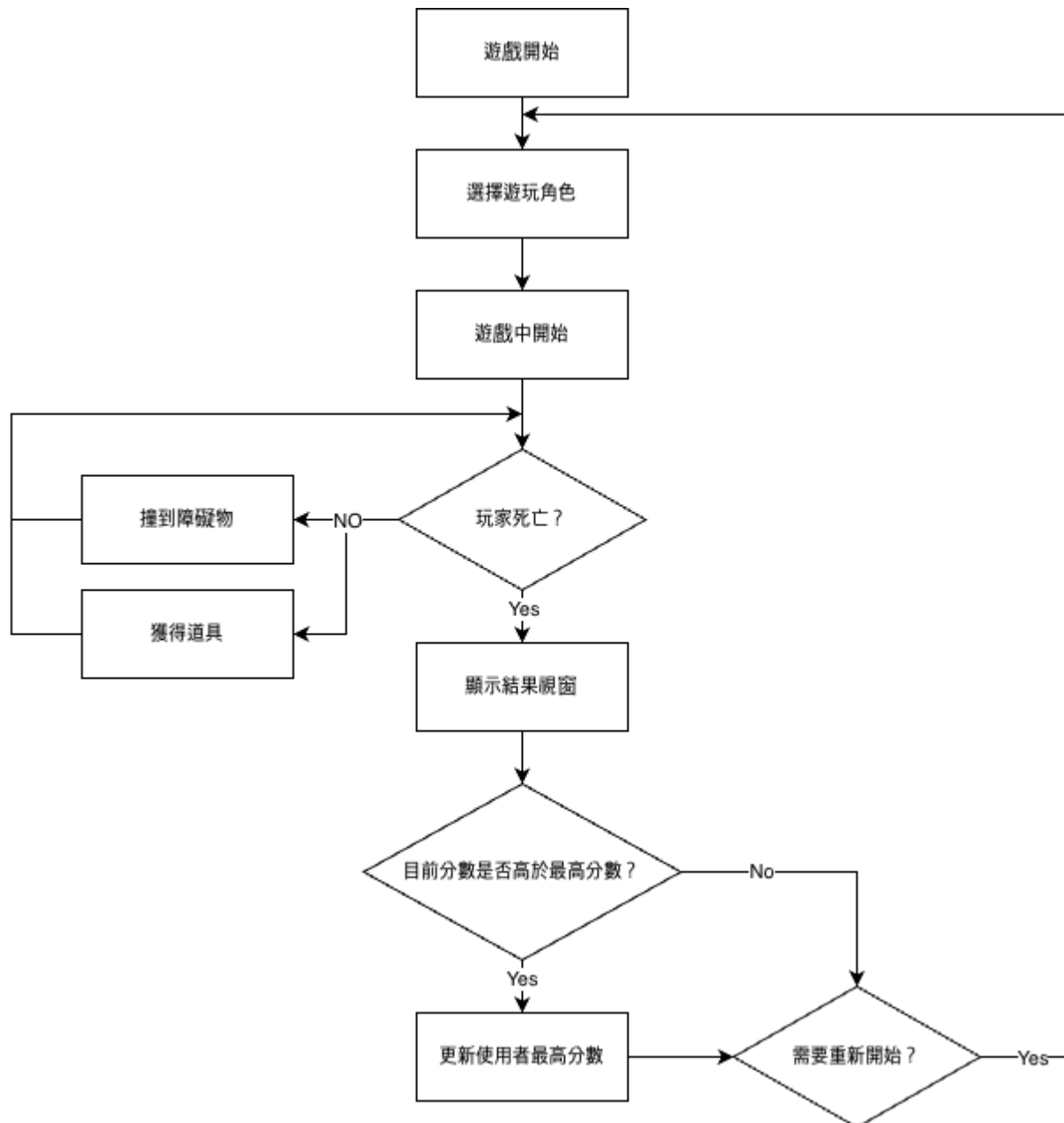
學習目標

- 透過指南文件與範例世界，學習 酷遊 核心功能的製作方法。
 - 學習在楓之谷世界中，依據想製作的遊 方式來設定地圖的方法。
 - 學習理解Logic， 利用 設計遊 架構的方法。
 - 能將製作完成的世界調整成想要的類型 製作。
-

學習核心功能

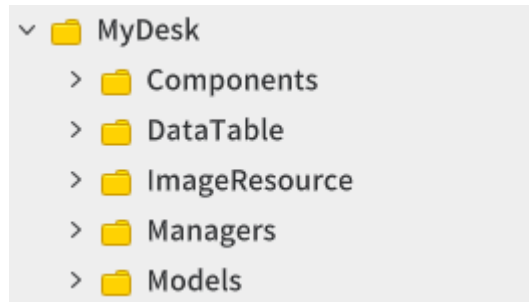
- **Physics Component**：除了基本提供的Body元件(Rigid、Kinematic、Sideview)外，也學習套用物理引擎的PhysicsBody與PhysicsCollider元件使用方法。
- **程序化地圖建立**：學習不只是單純隨機建立角色可奔 的平台，而是讓地形自然相連的建立方式。
- **** 物件池****： 了管理大量平台圖塊、道具，學習物件池的原理 可自行製作。

核心遊 循環



1. **遊 開始**：可透過標題 面確認開始遊 或遊 操作方式。
2. **玩家角色選擇**：在標題 面按下開始遊 按 時，會顯示選擇角色的UI。
3. **遊 中開始**：倒數後正式開始遊 。依角色套用不同捲動 面速度，圖塊、背景元素會朝角色反方向移動， 與道具、障碍物元素互動。
4. **角色體力狀態檢**：確認作 遊 結束條件，會隨時間持續下降的體力是否已變成0。
5. **結果確認**：在遊 結果視 顯示目前分數與原有最高分數，若目前分數高於原有最高分數，則更新玩家最高分數。
6. **確認重新開始請求**：若在遊 結果視 要求回到標題 面，則回到初始標題狀態。

資料夾結構



名稱	說明
Components	用來存放為了製作範例世界而附加在實體上的元件的資料夾。
DataTable	用來存放組成範例世界所需DataSet的資料夾。
ImageResource	這是存放製作遊戲所需ImageResource的資料夾。目前只包含用來地板圖塊與障礙物的Image Resource。
Managers	用來存放以楓之谷世界功能Logic製作的腳本的資料夾。
Models	用來存放model化後實體的資料夾。

製作順序

- 💡 **基礎學習**：包含楓之谷世界基本功能與該類型基礎功能的學習項目。
- 📖 **進階學習**：學習 製作範例世界中的功能項目。透過此流程的學習，可提升類型功能擴充與學習理解度。

[💡 基礎學習]1.建構世界環境

- 學習楓之谷世界提供的地圖種類與各Body元件的關係。
- 學習製作 酷遊 時需要什 樣的遊 製作環境。
- 學習PhysicsRigidbody、PhysicsCollider元件。
- 實際套用到世界中，掌握需要 些設定。

[💡 基礎學習]2.理解Logic

- 學習Logic扮演什 角色。
- 理解Manager的 念 學習使用方法。
- 理解GameLogic、DataSetLogic、UIManagerLogic的建構理由與目的。

[💡 基礎學習]3.建構捲動系統

- 學習製作 酷遊 時角色與環境要素的移動方式。
- 學習控制世界中可見背景元素的方法。
- 學習遊 進行後，隨時間改變環境要素外觀的方法。

[💡 基礎學習]4.透過程序化建立方式管理地圖

- 學習 酷遊 中什 是程序化建立方式，以及其必要性。
- 製作遊 中使用的地板圖塊， 學習如何組成Model。
- 以資料表 基礎製作DataSet，使地板圖塊可透過程序化建立方式 生。
- 利用程序化建立方式 生地板圖塊，製作無限捲動環境。

[💡 基礎學習]5.建立物件池

- 學習物件池 念 理解其目的。
- 製作物件池Logic。
- 透過物件池的生成與回收，直接確認運作方式。
- 學習地板圖塊被刪除時重新建立圖塊的方法。

[💡 基礎學習]6.製作玩家角色

- 學習讓 酷遊 角色執行跳躍、滑行動作的方法。
- 學習 製作建立多種角色所需的DataSet。
- 學習遊 重新開始時還原角色狀態與資訊的方法。
- 學習玩家按下技能鍵時發動各角色技能的方法。
- 製作提供玩家選擇多種角色的選擇視 與角色選擇功能。

[💡 基礎學習]7.製作Life系統

- 學習正式開始遊 後持續減少體力功能的製作方法。
- 學習當體力持續下降時，製作可讓玩家確認的UI元素方法。

[💡 基礎學習]8.建構UI

- 學習什 是UIGroup，以及其使用與編輯方法。
- 製作標題 面使用的UI元素 學習UI編輯方法。
- 學習透過Manager Logic以程式控制出現在標題 面的UI元素。
- 製作遊 中應顯示的UI元素 與 時資料連動。
- 製作遊 結束時顯示各分數的結果視 UI。

[📖 進階學習]9.製作障 物物件

- 學習如何在持續移動的地板圖塊上放置障 物。
- 學習如何設定角色與障 物的 撞偵測。
- 套用角色與障 物 撞時體力減少功能。

[📖 進階學習]10.製作道具

- 學習如何在持續移動的地板圖塊上配置道具元素。
- 學習 建立多種效果的道具而建立DataSet的方法。
- 學習以已儲存的道具資料表 基礎設定效果與固有數 的方法。
- 製作角色與道具 撞時發動多種道具效果與實際動作。

[📖 進階學習]11.製作資料儲存系統

- 學習玩家角色死亡時，若目前分數高於原最高分數，則更新 儲存的方法。

- 製作玩家執行世界時，若先前有最高分數，則讀取 顯示在標題 面。

[💡 基礎學習] 製作世界環境

影片時間軸

- 可在課程影片中以下時間點學習。
- 建立世界環境：00:00:01

學習目標

- 理解楓之谷世界提供的各種圖塊地圖模式特性， 掌握各地圖適合的Body元件關聯性。
- 分析 酷遊 類型特性， 能自行建構最佳化的遊 製作環境。
- 學習 製作角色移動與 撞偵測所需的PhysicsRigidbody與PhysicsCollider元件核心功能與屬性。
- 將學習到的物理系統實際套用到實體， 透過測試遊玩掌握依遊 企劃調整物理設定 的方法。

觀看該影片後可嘗試執行的內容

- 建立適合 酷遊 的地圖環境
- 建立角色與地板圖塊互動所需元件
- 測試是否能正常 在自製地板圖塊上 移動

##楓之谷世界地圖模式

Key Point

遊戲視角會隨地圖類型而變化！

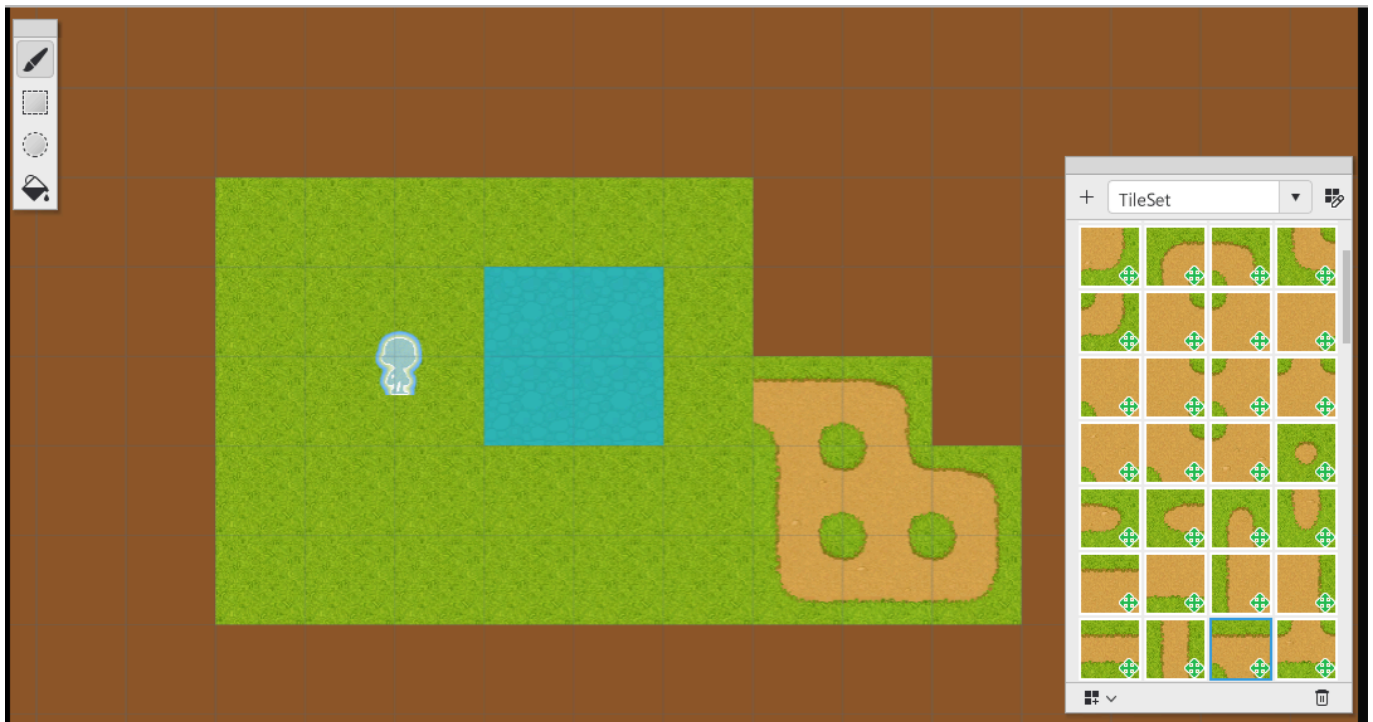
地圖類型	使用的 Component	套用的遊戲視角
TileMap	Rigidbody Component	楓之谷風格的橫向捲軸
RectTileMap	Kinematicbody Component	俯視視角
SideViewRectTileMap	Sideviewbody Component	使用棋盤格形式Tile的橫向捲軸

TileMap



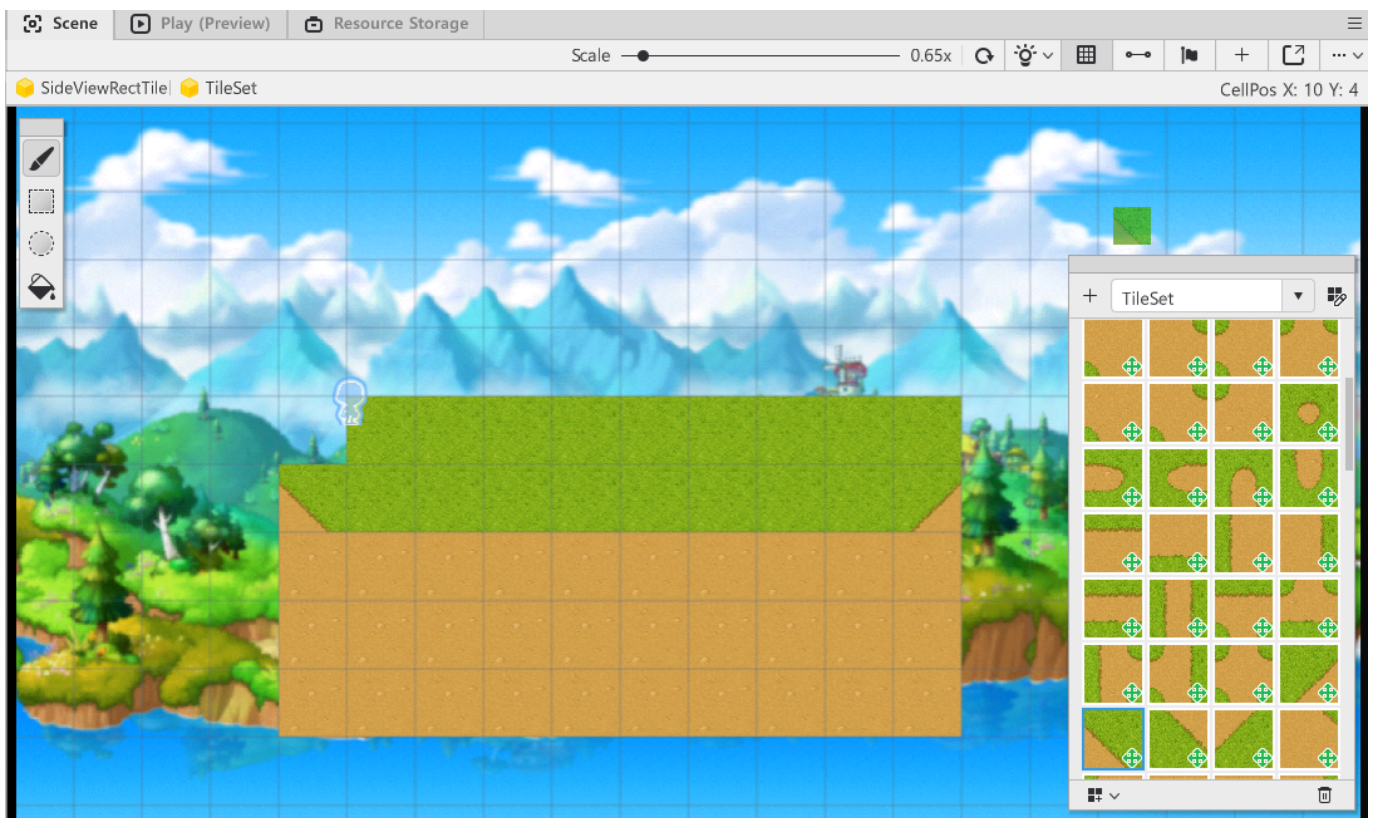
- 在楓之谷世界中初次建立世界時預設指定的地圖類型。
- 是最能展現楓之谷風格的世界方式。
- 利用楓之谷世界提供的Tile生成地面，配置實體來建立地圖。
- 可依圖塊的配置製作斜坡。
- 在製作橫向捲軸類型的遊戲時，是能展現楓之谷風格的有用地圖方式。

RectTileMap



- 在製作俯視視角類型的遊 時，是有效的地圖類型。
- 可自行製作圖塊集，或使用楓之谷世界提供的圖塊集資源來建立地圖。
- 可透過棋盤格類型配置圖塊來建立地圖。
- 可 每個圖塊賦予可移動與不可移動的性質。
- 與TileMap、SideViewRectTileMap方式不同，可製作4方向、8方向移動類型的遊 。

SideviewRectTileMap



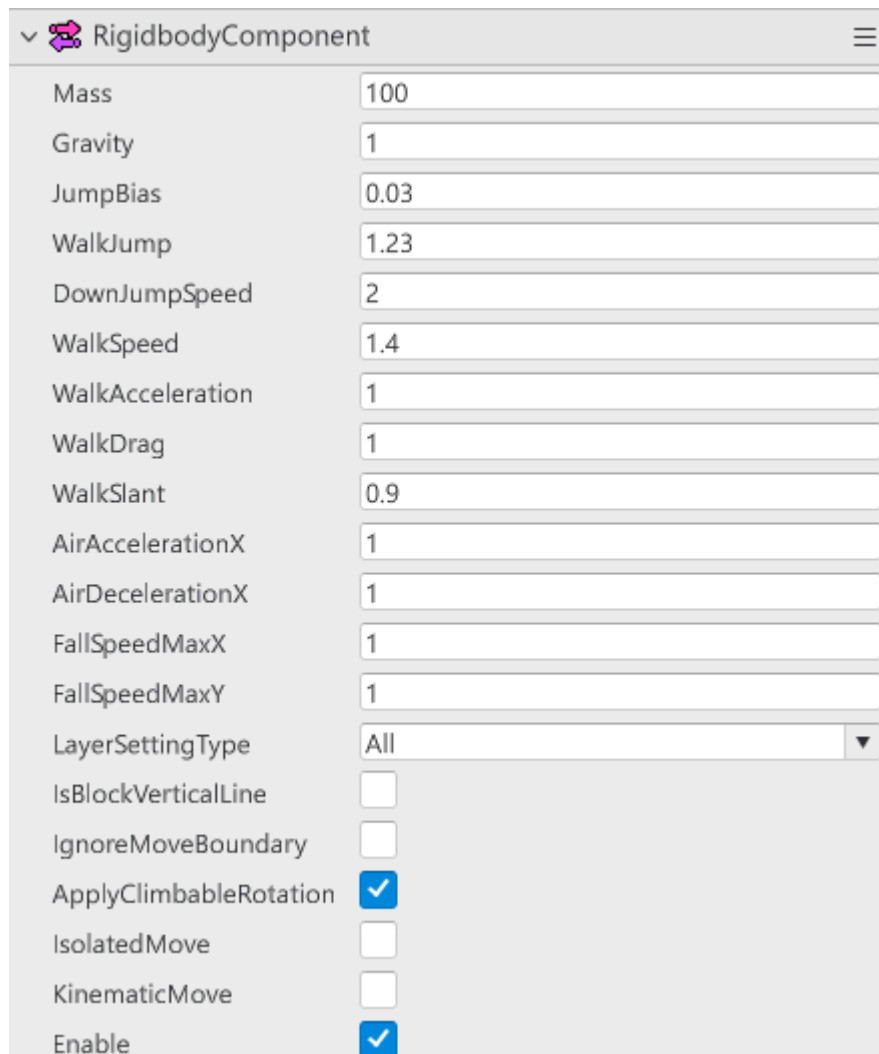
- 在製作橫向捲軸類型遊 時，是有效的地圖類型。
- 可自行製作圖塊集，或使用楓之谷世界提供的圖塊集資源來建立地圖。

- 可透過棋盤格類型配置圖塊來建立地圖。
- 與RectTileMap不同，若不是不可移動的圖塊集，角色會穿過圖塊。
- 因此使用SideViewRectTileMap建立地圖時，必須將不可移動的圖塊集製作 地面。

BodyComponent

- 在楓之谷世界中， 了與作 地面的圖塊互動，需要BodyComponent。
- BodyComponent會依地圖類型要求不同的BodyComponent。
- BodyComponent大致分 3種類型。

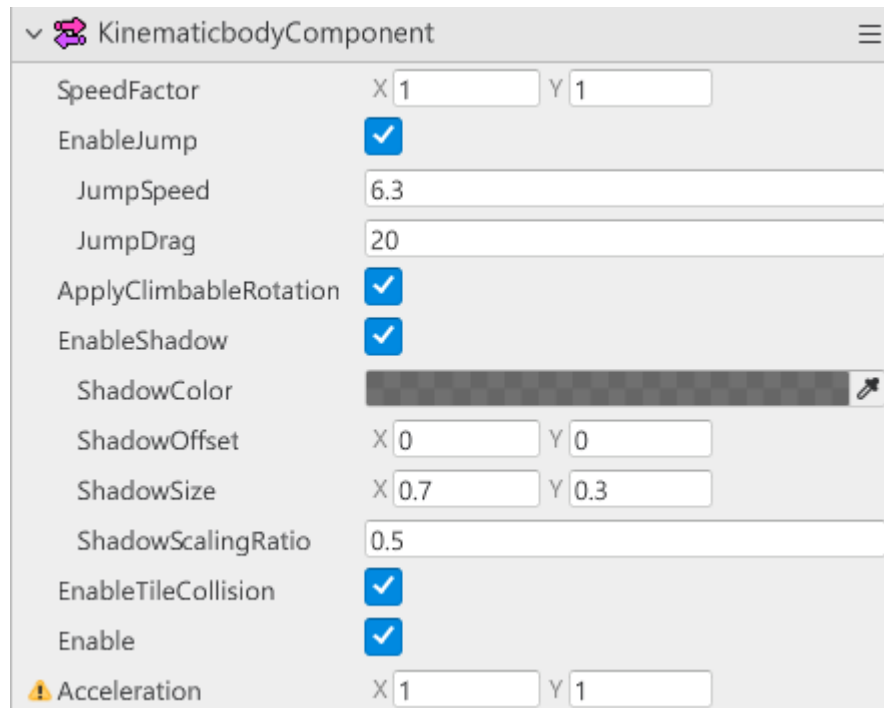
RigidbodyComponent



RigidbodyComponent	
Mass	100
Gravity	1
JumpBias	0.03
WalkJump	1.23
DownJumpSpeed	2
WalkSpeed	1.4
WalkAcceleration	1
WalkDrag	1
WalkSlant	0.9
AirAccelerationX	1
AirDecelerationX	1
FallSpeedMaxX	1
FallSpeedMaxY	1
LayerSettingType	All
IsBlockVerticalLine	☐
IgnoreMoveBoundary	☐
ApplyClimbableRotation	☒
IsolatedMove	☐
KinematicMove	☐
Enable	☒

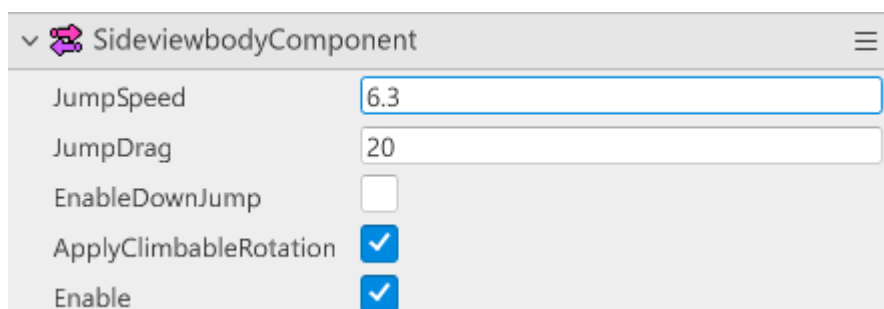
- 在楓之谷世界的TileMap方式中控制角色的物理。
- 若地圖 TileMap但沒有RigidBodyComponent，角色無法與圖塊互動。
- 在RigidBodyComponent中提供角色跳躍控制、重力控制等多種物理屬性 。
- 可透過變更該數 來調整物理效果。

KinematicComponent



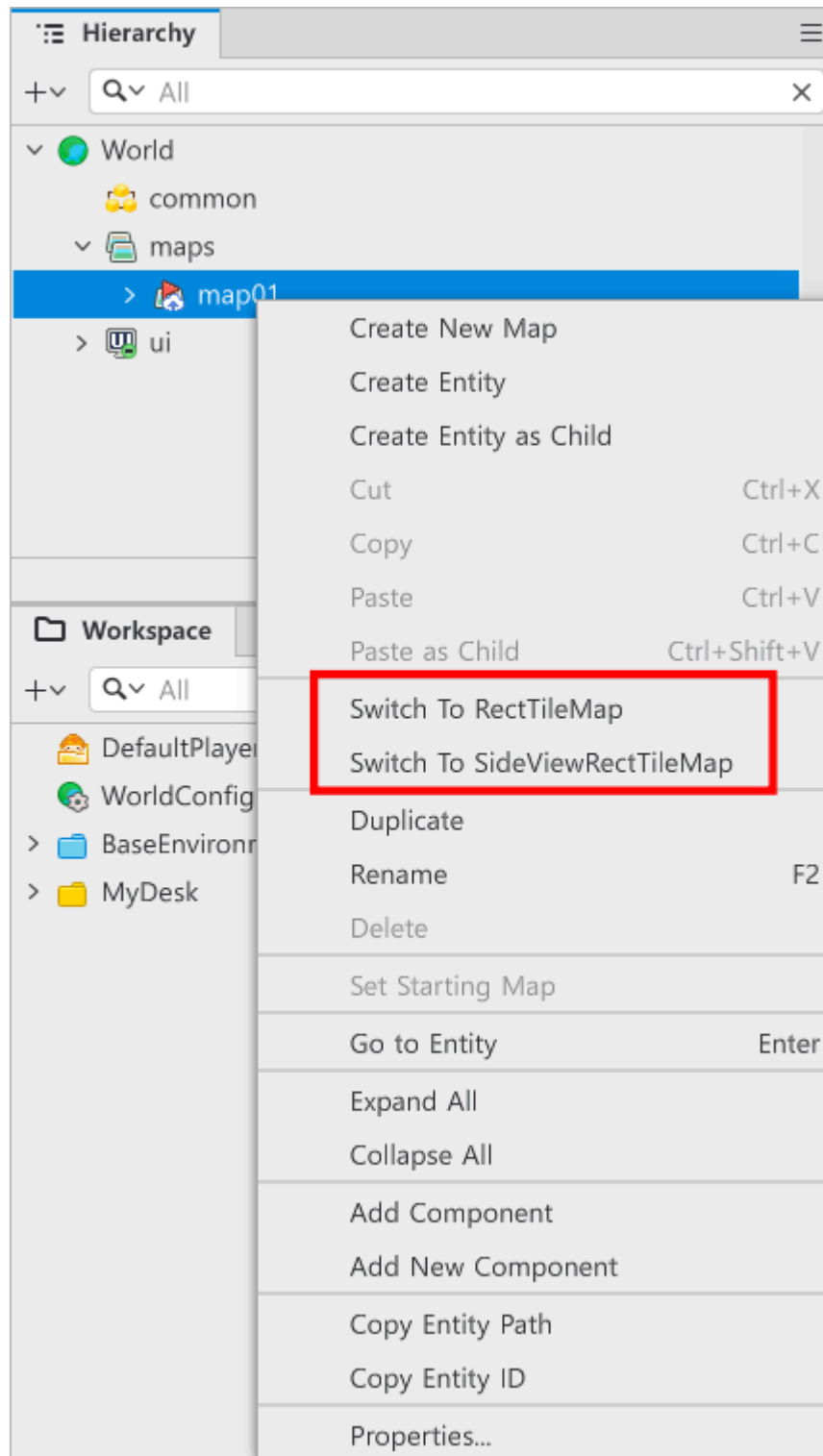
- 在楓之谷世界的RectTileMap方式時控制角色的物理。
- 當地圖 RectTileMap時，若沒有KinematicBodyComponent，將不會與RectTileMap的圖塊 生互動。
- 在KinematicBodyComponent中可調整角色的移動速度、是否可跳躍、是否顯示陰影等。

SideviewbodyComponent



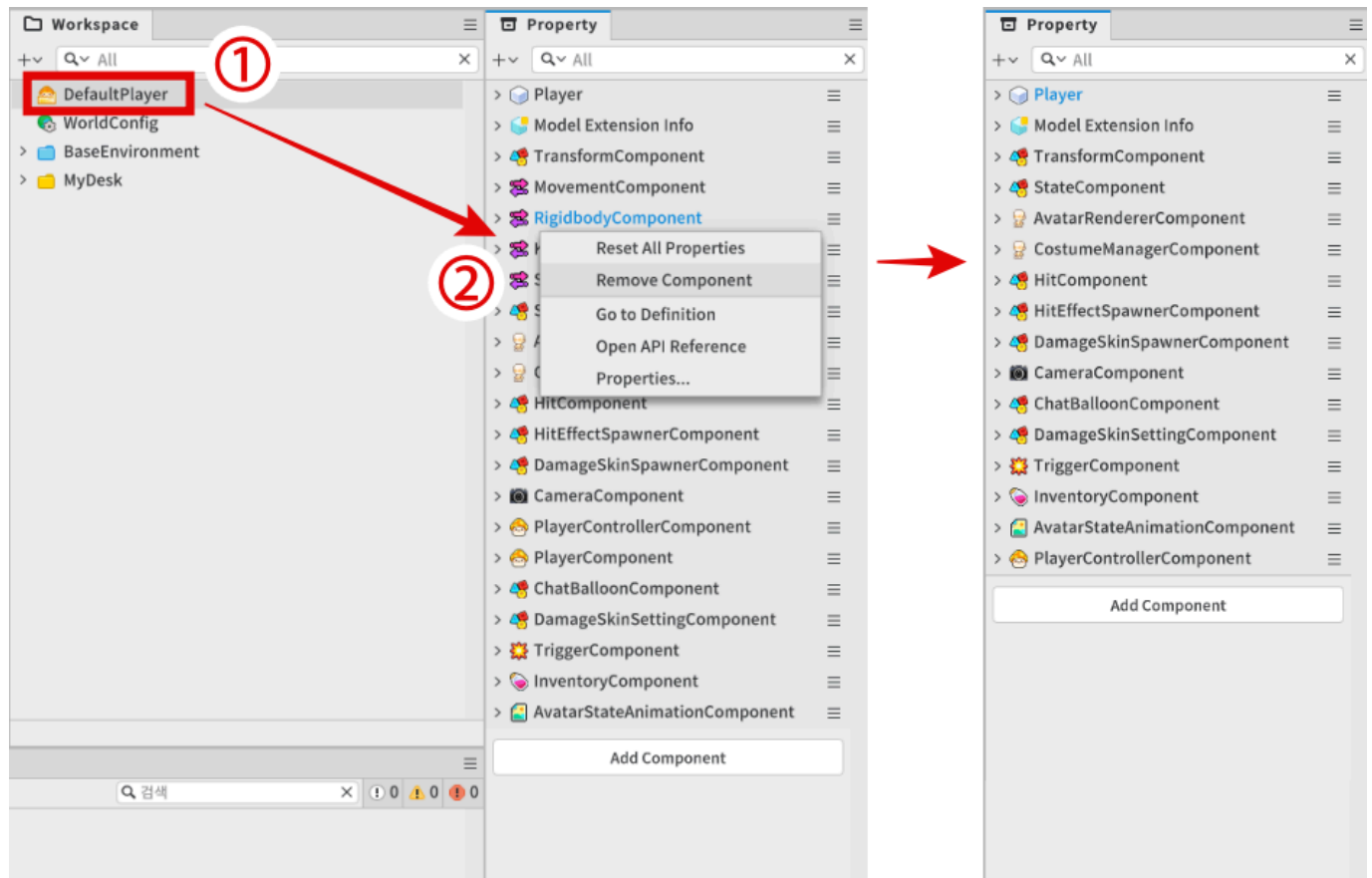
- 在楓之谷世界中使用SideViewRectTileMap方式時控制角色的物理。
- 若沒有該元件，角色無法與地圖圖塊互動。
- 可調整角色的跳躍速度與重力加速度等。

變更地圖類型



- 在Hierarchy視 中對要變更的地圖按右鍵。
- 點擊Switch To RectTileMap或Switch To SideViewRectTileMap其中之一。
 - 若目前地圖不是圖塊地圖，則會顯示Switch To TileMap，而切換至目前地圖狀態的選項會被隱藏。
- 製作 酷遊 ，點擊Switch To SideViewRectTileMap，將地圖切換 SideViewRectTileMap。

移除DefaultPlayer原有Component

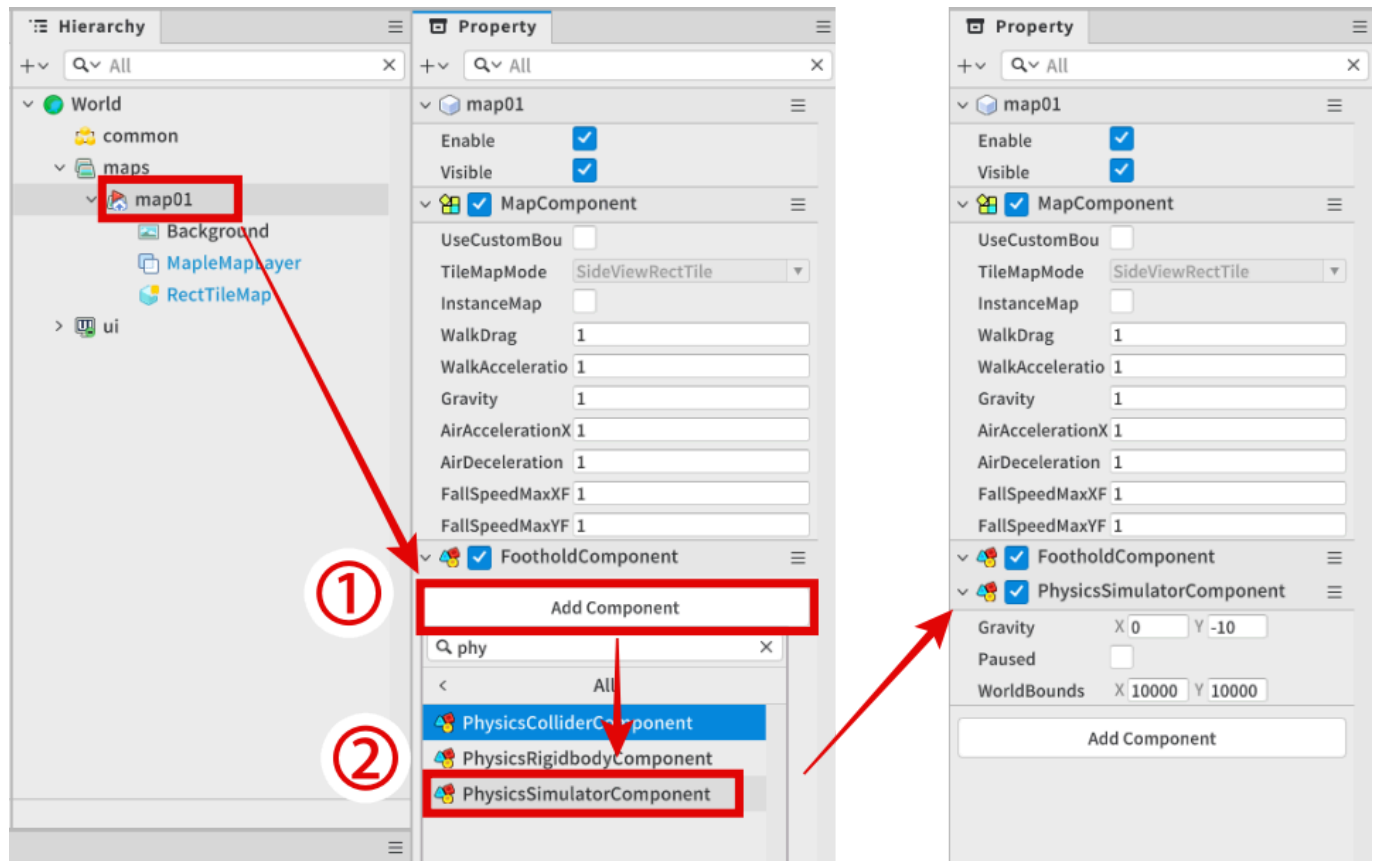


1. 在WorkSpace中選擇作 玩家操作角色範本的DefaultPlayer(Model)。
2. 在Property分頁中， 了防止新物理與互動功能造成干擾，使用Remove Component移除 有目標元件。
(Rigidbody、Kinematicbody、Sideviewbody、Movement、Player)

使用Physics元件套用物理系統

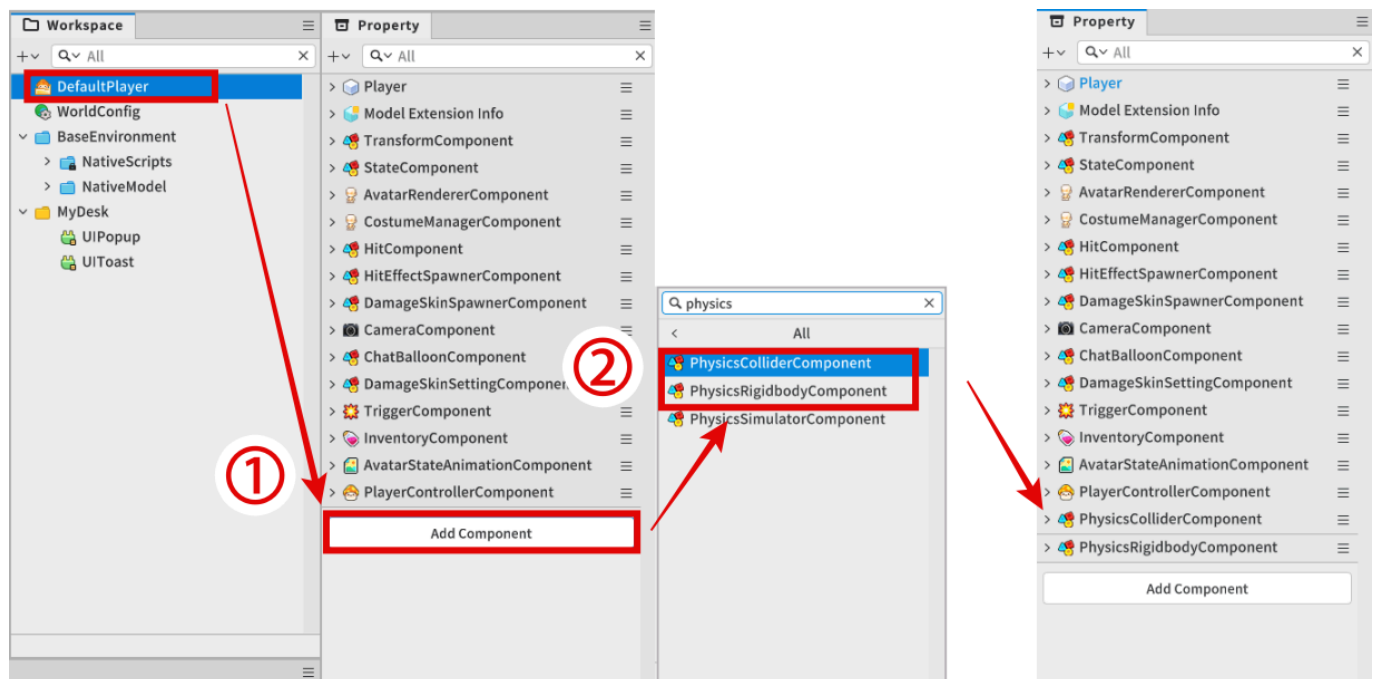
- 由於 酷遊 的特性，必須持續生成與銷 地板圖塊。因此， 了讓管理更順暢 賦予獨立功能，將地板圖塊本身製作一個完整的實體。
- 因 不是以TileSet製作的地形，所以需要額外的Physics設定。

在地圖實體 建立PhysicsSimulator



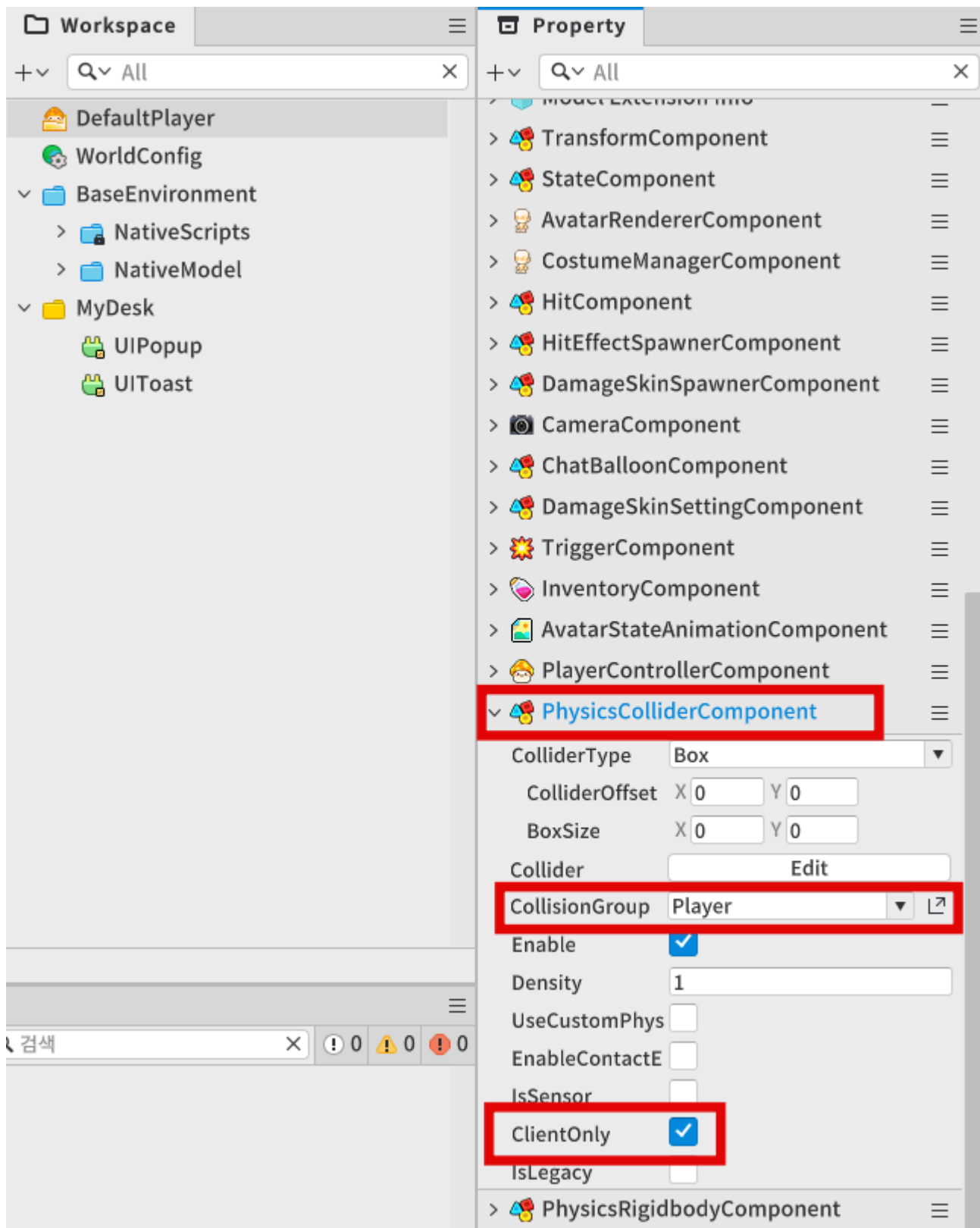
1. 在Hierarchy分頁中選擇Map實體後，點擊AddComponent按 。
2. 建立PhysicsSimulatorComponent。

建立角色PhysicsComponent



1. 選擇原始DefaultPlayer，在Property分頁中點擊AddComponent按 。
2. 建立PhysicsCollider、PhysicsRigidbodyComponent。

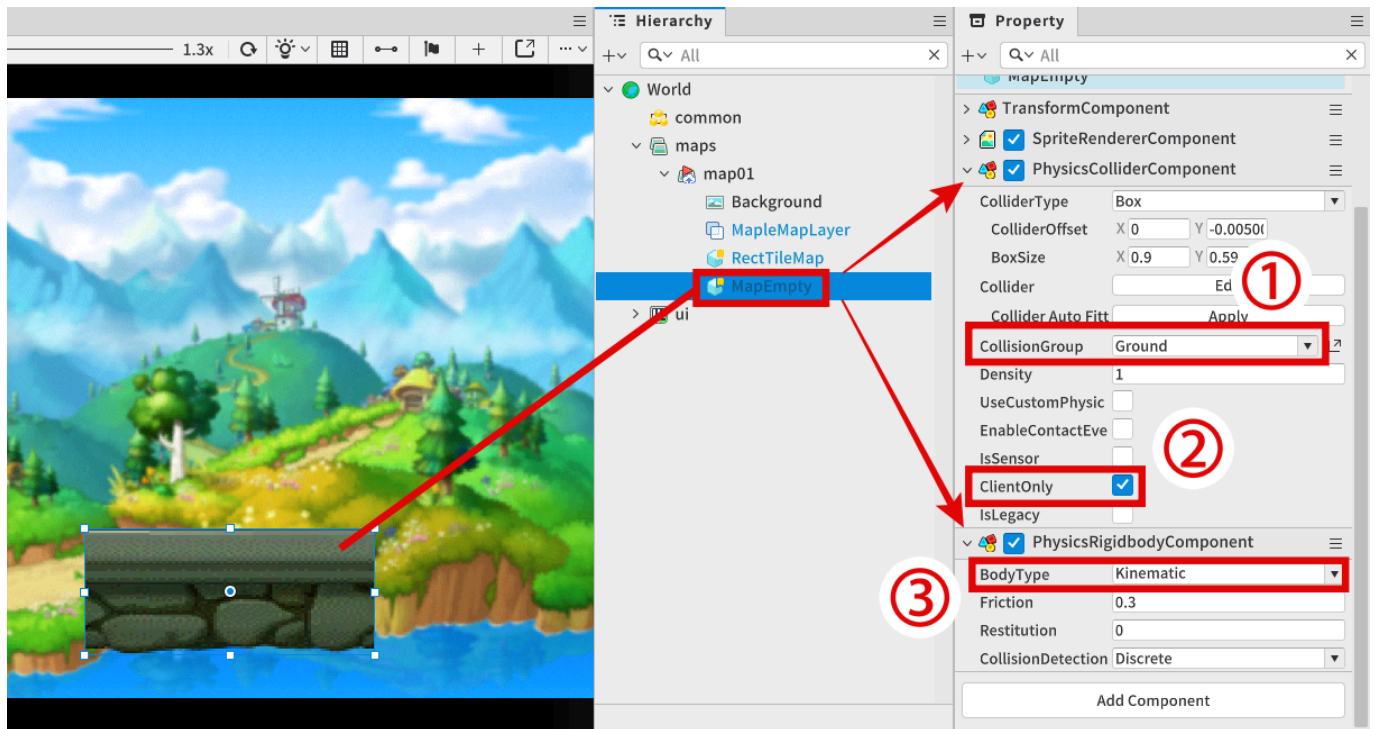
設定PhysicsColliderComponent



- 了設定 撞對象，新增 設定CollisionGroup 「Player」群組。
- 因 是與其他玩家不互動的遊 ，所以在PhysicsColliderComponent的內部屬性中勾選ClientOnly項目。

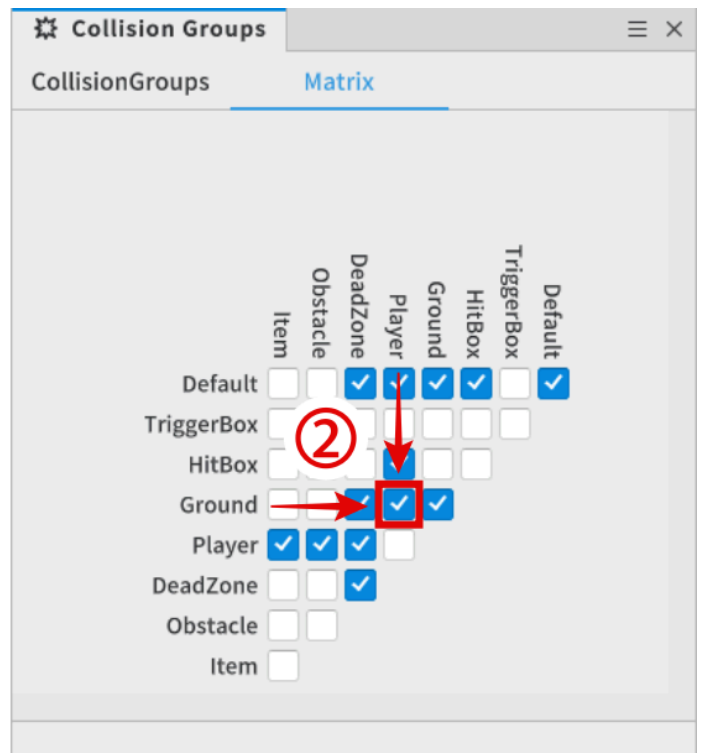
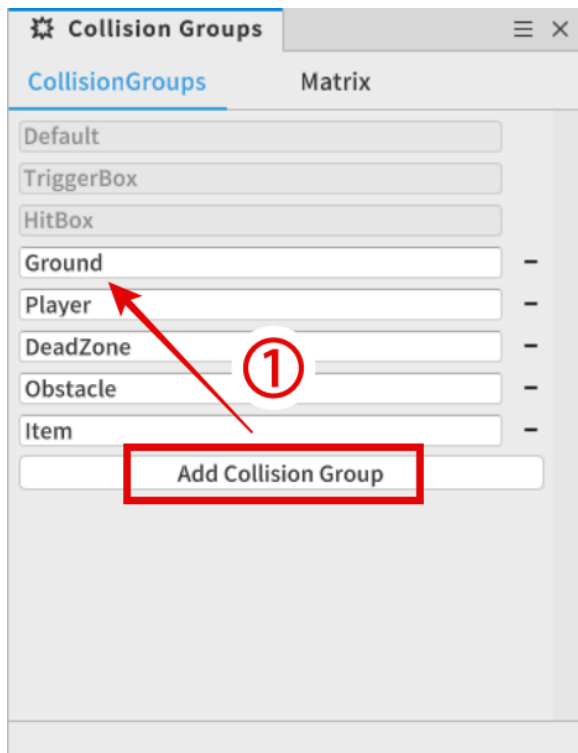
地板圖塊實體建構與互動測試

設定測試地板圖塊



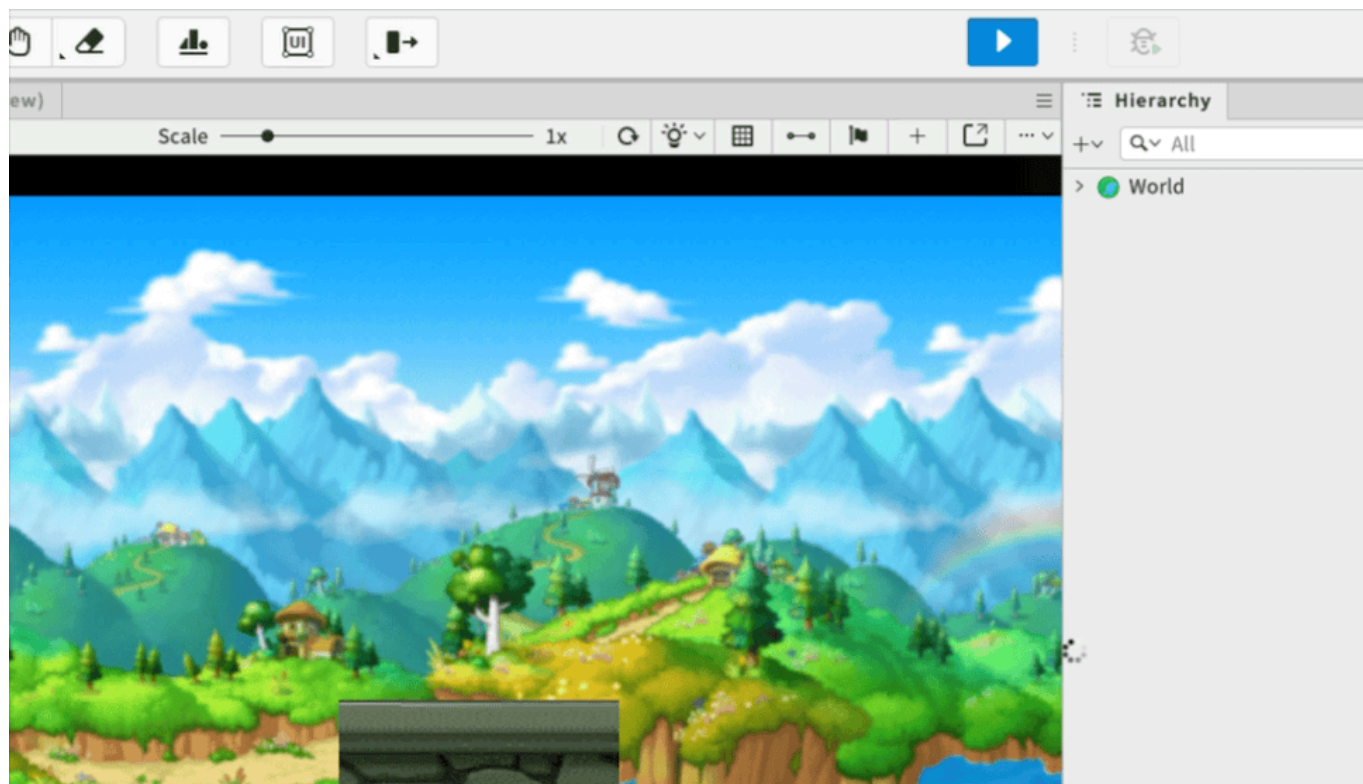
- 建立用來測試的實體，和角色一樣 建立PhysicsRigidbody、PhysicsColliderComponent。
1. 將CollisionGroup設定 可與角色互動的Ground。
 2. 和角色一樣，因 只在本地環境中運作，所以在PhysicsColliderComponent內部屬性中勾選ClientOnly項目。
 3. 作 地板使用的實體必須維持靜態狀態，只進行移動處理，因此在PhysicsRigidbodyComponent內部屬性中將BodyType設定 Kinematic。

新增 設定CollisionGroup



1. 可透過CollisionGroup的Add Collision Group建立新的CollisionGroup項目。
2. 了偵測Player與Ground的 撞，請確認在Matrix分頁中Player與Ground交叉線上是否已勾選 用。

互動測試



- 透過點擊編輯器上方的播放按 ，在執行的世界環境中確認角色是否正常受重力影響， 站在圖塊實體上方。

最低製作標準

- 建立角色可正常奔 的 酷遊 地圖。
 - 角色不可以漂浮在空中，也不可以穿過地板圖塊。

應用與進階應用

- 可調整要遊玩的角色TransformComponent的Scale 來調整角色大小， 可透過PhysicsCollider的collider Edit按 修改 撞器尺寸。透過這些設定，可以直觀地設定遊 難度。

[💡 基礎學習]理解Logic

影片時間軸

- 可在課程影片中以下時間點學習。
- 理解Logic : 00:13:54

學習目標

- 理解楓之谷世界中全域(Global)運作的Logic核心 念， 明確區分與依附一般實體的元件的差異。
- 掌握在遊 內集中控制各種系統的「管理者」必要性， 學習藉此降低專案程式碼複雜度與提高維護性的方法。
- 可設計成 酷遊 骨架的GameLogic、DataSetLogic、UIManagerLogic， 製作各邏輯有機傳遞資料的結構。

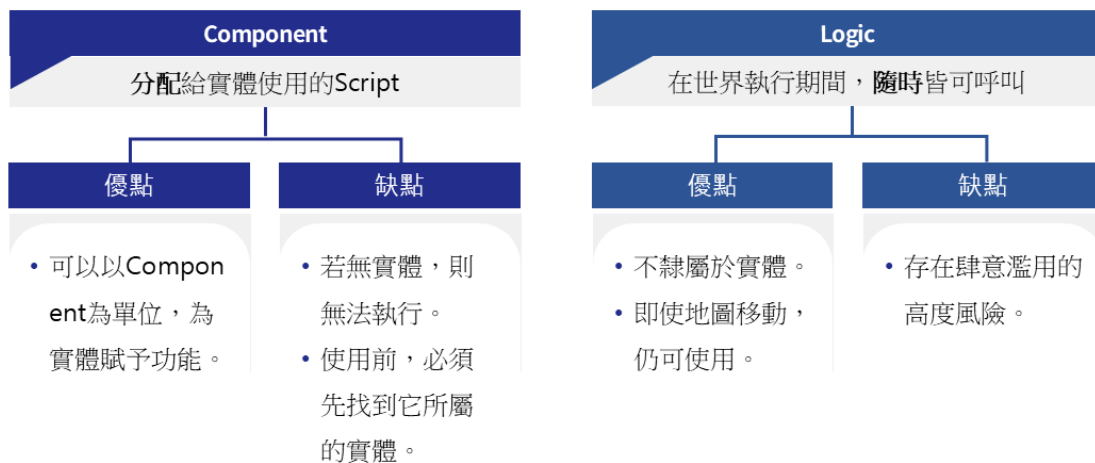
觀看該影片後可嘗試執行內容

- 分析有 些功能可以集中管理， 將其製作成Logic 本。
- 測試已製作的Logic 本是否能從外部(Logic外的其他元件)正常呼叫。

Logic是什 ？

Key Point

依據實體從屬關係而產生的Script差異



- 是楓之谷世界提供的 本類型之一。
- 建立Logic時，會在Workspace中以綠色 頭圖示顯示。
- 與Component不同， 使不附加在實體上也可以存取 本。

Manager 念

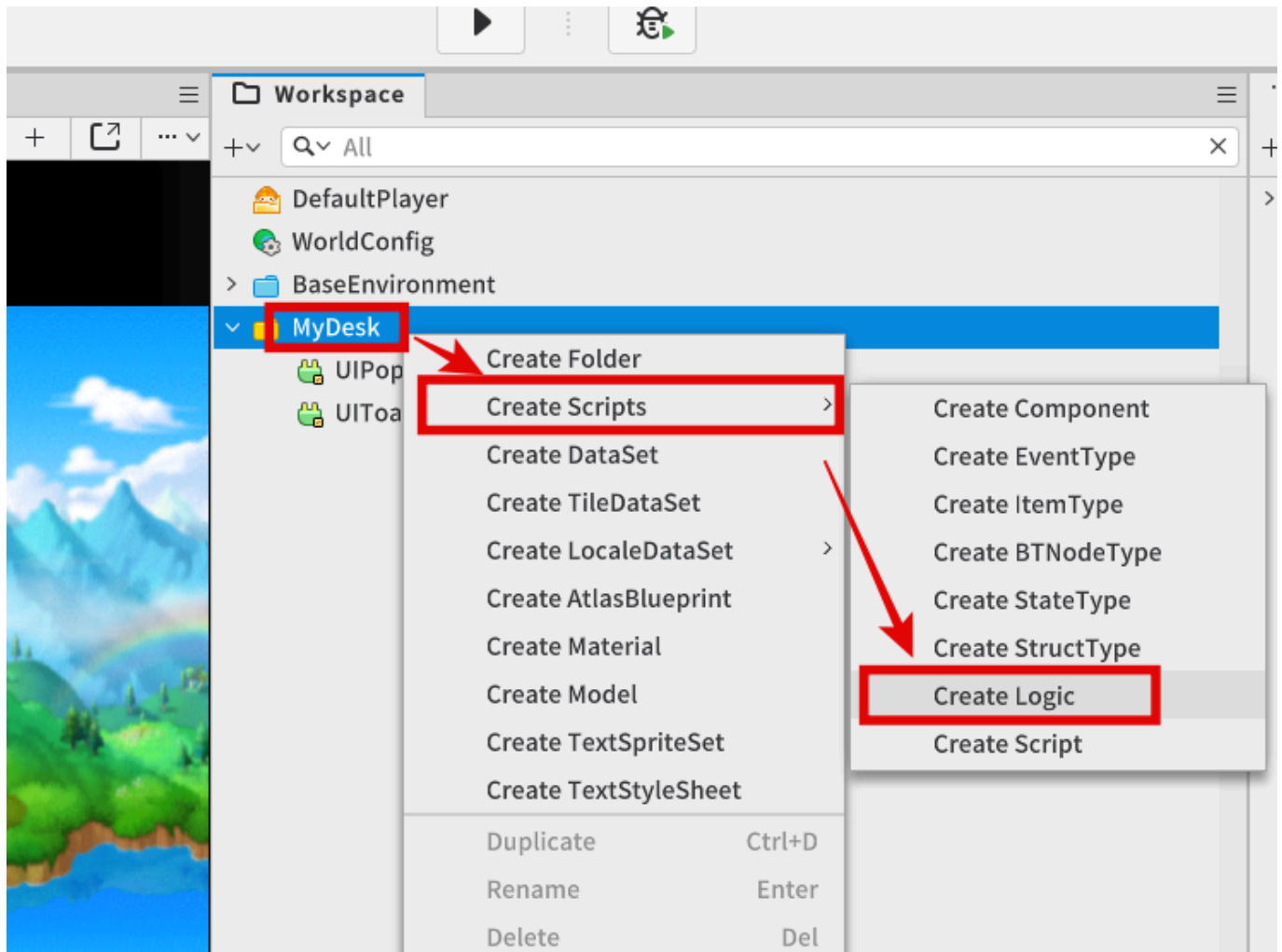
Key Point

透過Manager配置負責特定功能



- Logic可以指定特定功能或角色，以負責該功能的類型製作。
- 負責單一功能或角色的單位稱 Manager。
- 若特定功能發生問題，只需修改Manager內容可快速解決。
 - Case 1：角色計算傷害 傳遞給怪物的情況
 - 每次變更傷害公式或新增計算要素時，都需要修改公式。
 - 必須逐一 出 修改所有角色的程式碼。
 - Case 2：將傷害計算交由Logic製作的Manager處理
 - 在Logic中修改傷害計算公式。
 - 因 所有角色與怪物的傷害計算都由Logic負責，不需要再修改其他部分。
 - 使之後傷害計算出現問題，也只需修改Manager 可解決。

Logic製作方法



- 在Workspace中對MyDesk按右鍵。
- 在選單 面中點擊Create Scripts。
- 點擊Create Logic以建立新的Logic。

範例世界中Manager Logic的使用用途

名稱	使用用途
GameLogic	這是為了管理依遊戲進行而需要處理的功能所製作的Manager Logic。負責體力與分數控制、遊戲開始、遊戲結束、玩家資料儲存等遊戲進行所需的處理。
DataSetLogic	這是用來管理與使用DataSet的Logic。每當需要資料時逐一載入所需DataSet會很麻煩。當遊戲開始時，DataTableLogic會快取所有DataSet，並在需要時回傳已快取的資料。
UIManagerLogic	這是用來管理UI的Manager Logic。當元件或GameLogic需要進行與UI相關的處理時，會將請求傳遞給該Manager Logic。UIManagerLogic管理所有UI的元件，因此專責處理所有與UI相關的請求。

- 像上述這樣，範例世界會建立 使用Logic。
- 當需要製作或擴充其他功能時，也可以宣告 製作屬於 們自己的Logic來使用。

範例世界內Logic使用範例

GameLogic的情況

- 當遊 開始時，會處理持續減少體力 增加分數的功能。
- 當管理中的體力變成0時，會請求結果 面， 比較目前分數與 有最高分數後更新玩家最高分數。
- 當遊 正式進行時，執行每隔一定時間切換關 的計時器功能。

DataSetLogic的情況

- 用楓之谷世界的內建功能DataService，讀取道具、圖塊、角色資訊等原始資料。
- 了讓讀取的資料在程式碼中只要輸入名稱就能立刻 到，會加上標籤(Key)， 整理成容易理解的Lua資料表變數類型。

UIManagerLogic的情況

- 集中管理遊 內存在的UI元素之參照路徑，例如標題 面、角色選擇視 、體力與分數顯示、結果彈 等。
- 將尋 各UI元素的複雜路徑整合 一處，方便其他 本在需要特定UI操作時能輕 存取。
- 在設定 實體參照的狀態下，會依遊 狀態(開始、遊玩、結束)執行只批次開 或關閉所需UI群組的彈 控制功能。

撰寫GameLogic程式碼

```
@Logic
script GameLogic extends Logic

property number discript = 0 -- 這是用來控制遊戲整體流程的邏輯。

property ScrollingController scrollingController = nil

property number currentScore = 0 -- 這是用來存放目前遊戲分數值的變數。

@Sync
property number bestScore = 0 -- 這是用來存放玩家最高分數的變數。

property number scoreIncreaseTimer = 0 -- 這是用來定期更新分數的計時器變數。

property number currentStage = 1 -- 這是用來儲存目前關卡編號的變數。

property number stageTimer = 0 -- 這是用來測量關卡切換時機的計時器變數。

property number hp = 0 -- 這是用來儲存玩家體力值的變數。

property boolean isEnd = true -- 這是用來記錄遊戲已結束的狀態變數。

property boolean isMobile = true -- 這是用來儲存執行環境是否為手機環境的狀態變數。

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- 這是在世界開始時呼叫的方法。

-- 進行遊戲邏輯的初始化。
self:Init()

-- 以狀態變數儲存世界是否在手機環境中進行。
self.isMobile = Environment:IsMobilePlatform()
```

```
-- 在世界開始時播放背景音樂。
_SoundService:PlayBGM("f5bb92d91e134c699e7bb2cf1aaca8e6",0.4)
end

@ExecSpace("ClientOnly")
method void Init()
-- 進行管理整體遊戲的主要邏輯初始化。

-- 連接內部成員參照。
self:ConnectRefs()

-- 進行其他核心邏輯的初始化。
self:OrderInit()
-- 初始化分數值。
self:SetScore(0)
-- 初始化目前體力值。
self:SetHP(100)
end

@ExecSpace("ClientOnly")
method void ConnectRefs()
-- 這是用來連接內部成員參照的方法。

-- 連接捲動畫面控制器實體。
self.scrollingController =
_EntityService:GetEntityByPath("/maps/MainGame/Scrolling").ScrollingControl
ler
end

@ExecSpace("ClientOnly")
method void OrderInit()
-- 由遊戲邏輯對各核心邏輯、元件發出初始化命令。

-- 進行捲動畫面控制器初始化。
self.scrollingController:Init()
-- 進行資料集邏輯初始化。
_DataSetLogic:Init()
-- 執行UI管理器邏輯初始化。
_UIManagerLogic:Init()
-- 進行物件池邏輯初始化。
_ObjectPoolLogic:Init()
end

@ExecSpace("ClientOnly")
method void OnUpdate(number delta)
-- 這是每一幀都會被呼叫的方法。

-- 如果遊戲並未進行中
if self.isEnd == true then
-- 就結束該方法。
return
end
```

```

-- 依delta值自動減少體力。
self:AutoDecreaseHp(delta)
-- 依delta值自動增加分數。
self:AutoIncreaseScore(delta)

-- /分支1/ 如果關卡計時器還有剩餘時間
if self.stageTimer > 0 then
    -- 就在關卡計時器的值上減少delta值。
    self.stageTimer -= delta
-- /分支2/ 如果關卡計時器沒有剩餘時間值
else
    -- 就呼叫關卡切換。
    self:StageChange()
    -- 將關卡計時器值初始化為Config中設定的數值。
    self.stageTimer = _Config.stageChangeTime
end

end

@ExecSpace("ClientOnly")
method void AutoIncreaseScore(number delta)
-- 這是每當increaseScoreIntervalSecond(週期)時增加分數的方法。

-- /分支1/ 如果測量計時器的值小於作為基準的Interval值
if self.scoreIncreaseTimer < _Config.increaseScoreInterval then
    -- 就在計時器上加上delta值後結束方法。
    self.scoreIncreaseTimer += delta
    return
end

-- /分支2/ 如果測量計時器的值大於或等於作為基準的Interval值(即到達特定週期的時點。)

-- 增加分數。
self:AddScore(_Config.increaseScore)
-- 初始化測量計時器。
self.scoreIncreaseTimer = 0
end

@ExecSpace("ClientOnly")
method void AutoDecreaseHp(number delta)
-- 這是自動減少體力的方法。

-- 將應該減少的數值傳遞到目前體力值。
self:AddHP(delta * _Config.decreaseHpValue)
end

@ExecSpace("ClientOnly")
method void SetScore(number value)
-- 這是將作為參數傳入的值設定為Score值的方法。

-- 參數1 number value: 傳入要設定為分數值的數值。

-- 將目前分數值設定為特定數值。

```

```
self.currentScore = value
-- 將特定數值設定到登錄於UIManager的Text中。
_UIManagerLogic:SetScoreText(value)
end

@ExecSpace("ClientOnly")
method void AddScore(number addValue)
-- 這是把作為參數傳入的值加到Score值上的方法。

-- 參數1 number addValue : 傳入要加到原有分數值上的數值。

-- 將傳入的數值加到目前分數中。
self.currentScore += addValue
-- 將目前分數值傳遞到登錄於UIManager的Text中。
_UIManagerLogic:SetScoreText(self.currentScore)
end

@ExecSpace("ClientOnly")
method void SetHP(number value)
-- 這是將作為參數傳入的值設定為Hp值的方法。

-- 參數1 number value : 傳入要設定為Hp值的數值。

-- 將體力值設定為特定數值。(clamp用來限制低於與超過的情況。)
self.hp = math.clamp(value,0,_Config.maxHp)

-- 將體力值更新到登錄在UIManager的滑桿上。
_UIManagerLogic:SetHpSliderValue(self.hp)
end

@ExecSpace("ClientOnly")
method void AddHP(number addValue)
-- 這是把作為參數傳入的值加到體力值上的方法。

-- 參數1 number addValue : 傳入要增加的體力值。

-- 將參數值加到目前體力上。
self.hp += addValue

-- 使用clamp限制低於與超過的情況。
self.hp = math.clamp(self.hp,0,_Config.maxHp)

-- 將體力值更新到登錄在UIManager的滑桿上。
_UIManagerLogic:SetHpSliderValue(self.hp)

-- 如果體力值小於或等於0
if self.hp <= 0 then
    -- 就執行遊戲結束邏輯。
    self:EndMainGame()
end
end

@ExecSpace("Client")
method void EndMainGame()
```

```
-- 這是用來結束遊戲的方法。(僅客戶端方法)

-- 切換為遊戲已結束狀態。
self.isEnd = true

-- 將角色設為不可見。
_UserService.LocalPlayer.Visible = false

-- 遊戲結束時若執行環境為手機，則停用搖桿。
_UIManagerLogic:ShowMobileJoyStick(false)

-- 指定並輸出要設定在結果彈跳視窗中的數值。(傳遞最高分與目前分數。)
_UIManagerLogic:ShowResultPopup(tostring(math.floor(self.bestScore)),tostring(math.floor(self.currentScore)))

-- 要求捲動畫面控制器回收(停用)所有圖塊集物件。
self.scrollingController:ReturnAllTiles()

-- 降低背景捲動畫面速度。
self.scrollingController.background.ScrollRate = 0.5

-- 如果目前分數高於客戶端內所管理的最高分數
if self.currentScore > self.bestScore then
    -- 就將最高分數值設為目前分數。
    self.bestScore = self.currentScore
    -- 因為是最高分數變更的時點，所以更新伺服器端的最高分數值。
    self:UpdateScore(self.bestScore)
end

-- 更新主標題的最高分數值。
_UIManagerLogic:UpdateTitleBestScore(self.bestScore)
end

@ExecSpace("ClientOnly")
method void GameStart(number selectCharacterId)
-- 開始主遊戲時呼叫的方法。

-- 從CharacterInfoTable取得在角色選擇視窗中選擇角色的速度值。
local scrollSpeed =
_DataSetLogic.characterInfoTable[selectCharacterId].Speed
-- 登錄基本速度值。
self:SetDefaultScrollSpeed(scrollSpeed)

-- 將目前(開始)階段初始化為1。
self.currentStage = 1

-- 請求變更階段。
self:StageChange()

self:SetScore(0)

-- 初始化分數計時器值。
self.scoreIncreaseTimer = 0
```

```
-- 初始化體力值。
self:SetHP(100)

-- 將階段計時器設為config中設定的切換時間。
self.stageTimer = _Config.stageChangeTime

-- 若為行動裝置環境則顯示行動搖桿。
_UIManagerLogic:ShowMobileJoyStick(true)

-- 進行捲動控制器的初始化。
self.scrollingController:Reset()

-- 進行玩家角色的初始化。
_UserService.LocalPlayer.CustomPlayerController:ResetCharacter()

-- 設定顯示應提供給使用者的UI。
_UIManagerLogic:ShowPlayerInfo(true)

-- 進行技能邏輯的初始化。
_SkillLogic:Reset()

-- 為防止技能特效資源緩衝延遲，只播放資源特效。
_SkillLogic:PlaySkillEffect()

-- 套用各角色所擁有的專屬技能圖示圖片。
_UIManagerLogic:RegistSkillIcon()

-- 執行道具邏輯的初始化。
_ItemLogic:Reset()

-- 正式開始遊戲前執行3、2、1倒數。(包含Wait方法。)
_UIManagerLogic:CountTimerStart()

-- 切換為遊戲進行中的狀態。
self.isEnd = false

end

@ExecSpace("ClientOnly")
method void SetDefaultScrollSpeed(number speed)
-- 設定角色的基本捲動速度值。
-- 區分Defalut與Currrent的原因：使用加速器道具時速度值會變更，因此需另外儲存角色的基本速度值。

-- 參數1 number speed：傳入遊戲進行時套用的捲動速度值。

-- 將捲動元件的基本速度值設為參數值。
self.scrollingController.defaultSpeed = speed
-- 將捲動元件的目前速度值設為參數值。
self.scrollingController.currentScrollSpeed = speed
end

@ExecSpace("ClientOnly")
method void StageChange()
-- 變更階段的方法。
```

```
-- 階段編號增加1，但利用%運算子使資源要素可循環。
self.currentStage = (self.currentStage %
(_DataSetLogic.stageResourceCount+1)) + 1

-- 切換為目前設定階段編號對應的背景畫面。
self.scrollingController:ChangeBackground()
end

@ExecSpace("Server")
method void LoadData()
-- 取得已記錄資訊的方法。(僅伺服器方法)
-- 伺服器端執行：為取得伺服器端管理中的資料。

-- 利用資料儲存服務取得名為「MyData」的資料儲存空間。
local data = _DataStorageService:GetGlobalDataStorage("MyData")

-- 取得該資料儲存空間內 'BestScore' 鍵所對應的Value值。
local errorCode,value = data:GetAndWait("BestScore")

-- 登錄臨時分數值變數。
local tempScore

-- 若沒有已儲存資料
if value == nil then
    -- 將臨時分數值設為0。
    tempScore = 0

-- 若有已儲存資料
else
    -- 將臨時分數值設為原有分數值。
    tempScore = tonumber(value)
end

-- 更新bestScore值。
-- 因為是以Sync宣告的屬性，當值發生變更時，
-- 客戶端也會同步該值並自動呼叫OnSyncProperty方法。
self.bestScore = tonumber(tempScore)
end

@ExecSpace("Server")
method void SaveScore(number score)
-- 儲存使用者資訊的方法。

-- 將分數儲存至資料儲存空間的方法。(僅伺服器方法)

-- 利用資料儲存服務取得名為「MyData」的資料儲存空間。
local data = _DataStorageService:GetGlobalDataStorage("MyData")

-- 在該儲存空間內 'BestScore' 鍵對應的Value值登錄 'score' 值。
local errorCode = data:SetAndWait("BestScore",tostring(score))
end

@ExecSpace("Server")
```

```

method void UpdateScore(number score)
-- 更新伺服器端管理分數值的方法。(僅伺服器方法)

-- 參數1 number score : 傳入客戶端管理的分數值。

-- 將伺服器端bestScore值設為參數值。
self.bestScore = score
end

@ExecSpace("ClientOnly")
method void OnSyncProperty(string name, any value)
-- 僅限以Sync宣告的屬性，當伺服器端值變更時自動呼叫的方法。
-- 但若伺服器端變更的值與原本客戶端值相同，則不會呼叫此方法。

-- 若變更的屬性變數名稱為bestScore
if name == "bestScore" then
    -- 請求UIManagerLogic更新主標題畫面的最高分文字。
    _UIManagerLogic:UpdateTitleBestScore(tonumber(value))
end
end

@EventSender("Service", "UserService")
handler HandleUserEnterEvent(UserEnterEvent event)
-- 使用者進入、登入時呼叫的方法。

-- 載入伺服器端資料。
self:LoadData()

end

@EventSender("Service", "UserService")
handler HandleUserLeaveEvent(UserLeaveEvent event)
-- 使用者離開時呼叫的方法。

-- 執行向伺服器端登錄最高分的方法。
self:SaveScore(self.bestScore)
end

end

```

撰寫DataSetLogic程式碼

```

@Logic
script DataSetLogic extends Logic

property string discript = "" -- 用來管理該世界所管理資料的邏輯。

property table tileSetInfoTable = {} -- 用來儲存圖塊集資料的表格。

```

```

property number totalTileSetCount = -1 -- 儲存圖塊種類數量的變數。

property table itemInfoTable = {} -- 用來儲存道具資料的表格。

property table tilesFloorInfoTable = {[1]={} , [2]={}} -- 用來儲存圖塊集樓層資訊
的表格。/ ex table[1] = {2,4} 2、4號圖塊集從1樓開始。

property table characterInfoTable = {} -- 儲存角色資訊的表格。

property table skillInfoTable = {} -- 儲存技能資訊的表格。

property table stageRenderInfoTable = {} -- 儲存各階段需渲染資源資訊的表格。

property number stageResourceCount = 0 -- 階段資源種類數。

@ExecSpace("ClientOnly")
method void Init()
-- 進行DataSetLogic元件初始化的方法。

-- 初始化圖塊集表格。
self:InitTileSetInfoTable()
-- 初始化道具表格。
self:InitItemInfoTable()
-- 初始化角色表格。
self:InitCharacterInfoTable()
-- 初始化技能表格。
self:InitSkillInfoTable()
-- 初始化階段渲染表格。
self:InitStageRenderInfoTable()
end

@ExecSpace("ClientOnly")
method void InitTileSetInfoTable()
-- 用來儲存圖塊物件相關資料集的方法。

-- 用來在本地端存取儲存在工作空間資料集名稱的變數。
local dataSetName = "TileSetInfo"

-- 目標資料集的Row數。
local rowCount = self:GetDataSetRowCount(dataSetName)

-- 儲存圖塊集種類數。
self.totalTileSetCount = rowCount

-- 用來儲存各列資料的變數。
local rowData

-- 從1開始依資料集Row數進行巡覽。
for i = 1 , rowCount , 1 do
    rowData =
    {
        -- 圖塊集物件第一個圖塊的樓層。
        FirstFloor =
tonumber(_DataService:GetCell(dataSetName,i,"FirstFloor")),

```

```

        -- 圖塊集物件最後一個圖塊的樓層。
        LastFloor =
tonumber(_DataService:GetCell(dataSetName,i,"LastFloor")),
        -- 用來建立圖塊集物件的原始實體模型(Prefab)Id。
        EntryID = _DataService:GetCell(dataSetName,i,"EntryID")
    }

    -- 將以rowData儲存的資料存入表格。
    self.tileSetInfoTable[i] = rowData

    -- 將剛登錄的圖塊集樓層存入變數。
    local firstTileFloor = self.tileSetInfoTable[i].FirstFloor

    -- /分支1/ 若第一個圖塊樓層為1樓
    if firstTileFloor == 1 then
        -- 存入1樓表格清單。
        table.insert(self.tilesFloorInfoTable[1],i)
    -- /分支2/ 若第一個圖塊樓層不是1樓
    else
        -- 存入2樓表格清單。
        table.insert(self.tilesFloorInfoTable[2],i)
    end
end

end

@ExecSpace("ClientOnly")
method number GetDataSetRowCount(string Name)
    -- 回傳目標資料集的Row數。

    -- 參數1 string Name: 傳入目標資料集名稱。

    -- 回傳確認過的資料集Row數。
    return _DataService:GetRowCount(Name)
end

@ExecSpace("ClientOnly")
method void InitItemInfoTable()
    -- 用來儲存道具資料集的方法。

    -- 用來在本機端存取儲存在工作空間資料集名稱的變數。
    local dataSetName = "ItemInfo"

    -- 目標資料集的Row數。
    local rowCount = self:GetDataSetRowCount(dataSetName)

    -- 用來儲存各列資料的變數。
    local rowData

    -- 從1開始依資料集Row數進行巡覽。
    for i = 1 , rowCount , 1 do
        rowData =
        {

```

```

        -- 儲存道具名稱。
        Name = tostring(_DataService:GetCell(dataSetName,i,"Name")),
        -- 儲存道具的RUID。
        RUID = tostring(_DataService:GetCell(dataSetName,i,"RUID")),
        -- 儲存道具擁有的效果固有數值。
        Value = tonumber(_DataService:GetCell(dataSetName,i,"Value")),
        -- 儲存道具生成機率。
        SpawnRate =
tonumber(_DataService:GetCell(dataSetName,i,"SpawnRate")),
        -- 儲存道具模型(Prefab)的EntryID值。
        EntryID = tostring(_DataService:GetCell(dataSetName,i,"EntryID"))
    }

    -- 將以rowData儲存的資料存入表格。
    self.itemInfoTable[i] = rowData
end
end

@ExecSpace("ClientOnly")
method void InitCharacterInfoTable()
-- 用來儲存角色資料集的方法。

-- 用來在本地端存取儲存在工作空間資料集名稱的變數。
local dataSetName = "CharacterInfo"

-- 目標資料集的Row數。
local rowCount = self:GetDataSetRowCount(dataSetName)

-- 用來儲存各列資料的變數。
local rowData

-- 從1開始依資料集Row數進行巡覽。
for i = 1 , rowCount , 1 do
    rowData =
    {
        -- 儲存角色名稱。
        Name = tostring(_DataService:GetCell(dataSetName,i,"Name")),
        -- 儲存角色專屬捲動速度值。
        Speed = tonumber(_DataService:GetCell(dataSetName,i,"Speed")),
        -- 儲存角色專屬技能Id。
        SkillId = tonumber(_DataService:GetCell(dataSetName,i,"SkillId")),
        -- 儲存套用至角色附加Costume渲染器的RUID值。
        RUID = tostring(_DataService:GetCell(dataSetName,i,"RUID"))
    }

    -- 將以rowData儲存的資料存入表格。
    self.characterInfoTable[i] = rowData
end
end

@ExecSpace("ClientOnly")
method void InitSkillInfoTable()
-- 用來儲存技能資料集的方法。

```

```

-- 用來在本地端存取儲存在工作空間資料集名稱的變數。
local dataSetName = "SkillInfo"

-- 確認目標資料集row數。
local rowCount = self:GetDataSetRowCount(dataSetName)

-- 儲存內部成員資源種類數。
self.stageResourceCount = rowCount

-- 用來儲存各列資料的變數。
local rowData

-- 從1開始依資料集Row數進行巡覽。
for i = 1 , rowCount , 1 do
    rowData =
    {
        -- 儲存技能名稱。
        Name = tostring(_DataService:GetCell(dataSetName,i,"Name")),
        -- 儲存技能使用效果固有數值。
        Value = tonumber(_DataService:GetCell(dataSetName,i,"Value")),
        -- 儲存技能特效RUID值。
        EffectRUID =
tostring(_DataService:GetCell(dataSetName,i,"EffectRUID")),
        -- 儲存遊戲中UI RUID值。
        UI_RUID = tostring(_DataService:GetCell(dataSetName,i,"UI_RUID"))
    }

    -- 將以rowData儲存的資料存入表格。
    self.skillInfoTable[i] = rowData
end
end

@ExecSpace("ClientOnly")
method void InitStageRenderInfoTable()
-- 用來儲存各階段資源資訊資料集的方法。

-- 用來在本地端存取儲存在工作空間資料集名稱的變數。
local dataSetName = "StageRenderInfo"

-- 確認目標資料集row數。
local rowCount = self:GetDataSetRowCount(dataSetName)

-- 用來儲存各列資料的變數。
local rowData

-- 從1開始依資料集Row數進行巡覽。
for i = 1 , rowCount , 1 do
    rowData =
    {
        -- 儲存各階段套用的地面圖塊RUID值。
        TileRUID =
tostring(_DataService:GetCell(dataSetName,i,"TileRUID")),
        -- 儲存各階段套用的障礙物件RUID值。
        ObstacleRUID =

```

```

tostring(_DataService:GetCell(dataSetName,i,"ObstacleRUID")),
    -- 儲存各階段套用的背景畫面RUID值。
    BackgroundRUID =
tostring(_DataService:GetCell(dataSetName,i,"BackgroundRUID"))
}

-- 將以rowData儲存的資料存入表格。
self.stageRenderInfoTable[i] = rowData
end
end

end

```

撰寫UIManagerLogic程式碼

```

@Logic
script UIManagerLogic extends Logic

property number discript = 0 -- 用來管理整體UI、物件、內部數值的邏輯。

property SliderComponent hpSlider = nil

property Entity toastUIGroup = nil

property TextComponent inGameCurrentScoreText = nil

property Entity resultPopup = nil

property Entity resultBestScoreText = nil

property Entity currentScoreText = nil

property Entity resultToMenuButton = nil

property Entity mainTitlePanel = nil

property Entity characterSelectPopup = nil

property SpriteGUIRendererComponent skillRenderer = nil

property TextComponent titleBestScore = nil

property Entity inGameScoreText = nil

property TextComponent counterText = nil

property Entity titleGameStartButton = nil

property Entity defaultUIGroup = nil

```

```

property Entity mobileSlideButton = nil

property Entity mobileJumpButton = nil

@ExecSpace("ClientOnly")
method void Init()
-- 執行UIManagerLogic初始化的方法。

-- 連接內部成員參照。
self.ConnectRefs()

-- 初始化進行時啟用存放UI物件的主畫布。
self.toastUIGroup.Enable = true
-- 連接結果視窗內主選單按鈕點擊時回到遊戲初始狀態的方法。
self.resultToMenuButton:ConnectEvent(ButtonClickEvent,self.ReturnToMenu)
-- 連接標題畫面按下開始遊戲按鈕時開啟角色選擇視窗的方法。
self.titleGameStartButton:ConnectEvent(ButtonClickEvent,self.ShowCharacterSelectPopup)
end

@ExecSpace("ClientOnly")
method void ConnectRefs()
-- 連接內部所需成員參照的方法。

-- 連接toastUIGroup實體。
self.toastUIGroup = _EntityService:GetEntityByPath("/ui/ToastGroup")
-- 連接defaultUIGroup實體。
self.defaultUIGroup = _EntityService:GetEntityByPath("/ui/DefaultGroup")

-- 連接主標題面板實體。
self.mainTitlePanel =
_EntityService:GetEntityByPath("/ui/ToastGroup/UITitle")
-- 連接主標題畫面最高分文字元件。
self.titleBestScore =
_EntityService:GetEntityByPath("/ui/ToastGroup/UITitle/UITitleBestScorePanel/UITitleBestScore").TextComponent
-- 連接主標題畫面的開始遊戲按鈕實體。
self.titleGameStartButton =
_EntityService:GetEntityByPath("/ui/ToastGroup/UITitle/UIButtonGameStartBtn")

-- 連接角色選擇彈跳視窗實體。
self.characterSelectPopup =
_EntityService:GetEntityByPath("/ui/ToastGroup/UITitle/UICharacterSelect")

-- 連接遊戲中畫面上方UI hpSlider參照。
self.hpSlider =
_EntityService:GetEntityByPath("/ui/ToastGroup/InGameUI/UI_HPbar").SliderComponent
-- 連接遊戲中畫面上方UI ScoreText(分數值)參照。
self.inGameCurrentScoreText =
_EntityService:GetEntityByPath("/ui/ToastGroup/InGameUI/UIInGameScore").TextComponent
-- 連接遊戲中畫面上方UI ScoreText(僅顯示'Score'字樣)參照。

```

```

self.inGameScoreText =
_EntityService:GetEntityByPath("/ui/ToastGroup/InGameUI/UIScoreText")
-- 連接遊戲中技能圖示元件。
self.skillRenderer =
_EntityService:GetEntityByPath("/ui/ToastGroup/InGameUI/UI_Skill").SpriteGUI
IRendererComponent
-- 連接遊戲中開始時的開始等待計時文字。
self.counterText =
_EntityService:GetEntityByPath("/ui/ToastGroup/InGameUI/UICounterText").Text
Component
-- 連接遊戲中、行動裝置環境下，為了執行角色滑行功能的搖桿按鈕實體。
self.mobileJumpButton =
_EntityService:GetEntityByPath("/ui/DefaultGroup/Button_Jump")
-- 連接遊戲中、行動裝置環境下，為了執行角色跳躍功能的搖桿按鈕實體。
self.mobileSlideButton =
_EntityService:GetEntityByPath("/ui/DefaultGroup/Button_Slide")

-- 連接結果畫面彈窗實體。
self.resultPopup =
_EntityService:GetEntityByPath("/ui/ToastGroup/UIResultPopup")
-- 連接結果彈跳視窗內最高分文字實體。
self.resultBestScoreText =
_EntityService:GetEntityByPath("/ui/ToastGroup/UIResultPopup/UIBestScoreText/
UIBestScore")
-- 連接結果彈跳視窗內目前分數文字實體。
self.currentScoreText =
_EntityService:GetEntityByPath("/ui/ToastGroup/UIResultPopup/UIScoreText/UI
Score")
-- 連接結果彈跳視窗內選單按鈕實體。
self.resultToMenuButton =
_EntityService:GetEntityByPath("/ui/ToastGroup/UIResultPopup/UISMainMenuBtn"
)
end

@ExecSpace("ClientOnly")
method void UpdateTitleBestScore(number score)
-- 這是設定顯示在主標題畫面的最高分數值的方法。

-- 參數1 number score：以傳入的值設定標題畫面內的分數值。

-- 如果沒有score值
if score == nil or score <= 0 then
-- 將主標題畫面的最高分文字值設為「0」。
self.titleBestScore.Text = "0"

-- 如果有score值
else
-- 將主標題畫面的最高分登錄為傳入的值。利用math.floor捨去小數點以下部分。
self.titleBestScore.Text = tostring(math.floor(score))

end
end

@ExecSpace("ClientOnly")

```

```
method void ShowCharacterSelectPopup()
-- 這是顯示角色選擇視窗的方法。

-- 利用實體服務取得角色選擇彈窗。
local characterSelectPopup =
_EntityService:GetEntityByPath("/ui/ToastGroup/UITitle/UICharacterSelect")

-- 啟用彈窗實體。
characterSelectPopup.Enable = true
end

@ExecSpace("ClientOnly")
method void ShowMobileJoyStick(boolean active)
-- 這是在行動裝置環境中顯示用來控制角色的搖桿控制器的方法。

-- 參數1 boolean active: 若為true則啟用, 若為false則停用。

-- 如果目前平台環境為行動裝置環境
if _GameLogic.isMobile == true then
-- 以參數值控制存放搖桿控制器的UI畫布之啟用、停用。
self.defaultUIGroup.Enable = active
end
end

@ExecSpace("ClientOnly")
method void CountTimerStart()
-- 這是在正式遊戲執行前進行開始倒數的方法。

-- 啟用倒數文字物件。
self.counterText.Entity.Enable = true

-- 將倒數文字的值設為「3」。
self.counterText.Text = "3"
-- 邏輯執行等待1秒。
wait(1)
self.counterText.Text = "2"
wait(1)
self.counterText.Text = "1"
wait(1)

-- 因倒數已結束, 將倒數文字物件停用。
self.counterText.Entity.Enable = false
end

@ExecSpace("ClientOnly")
method void ShowPlayerInfo(boolean active)
-- 這是啟用、停用應提供給遊戲中玩家的UI元素的方法。

-- 參數1 boolean active: 若為true則啟用, 若為false則停用。

-- 將遊戲中hp滑桿的啟用狀態設為參數值。
self.hpSlider.Entity.Enable = active
-- 將遊戲中顯示目前分數值的文字物件啟用狀態設為參數值。
self.inGameCurrentScoreText.Entity.Enable = active
```

```
-- 將遊戲中「score」文字物件的啟用狀態設為參數值。
self.inGameScoreText.Enable = active
-- 將遊戲中技能圖示物件的啟用狀態設為參數值。
self.skillRenderer.Entity.Enable = active
end

@ExecSpace("ClientOnly")
method void RegistSkillIcon()
-- 這是在遊戲中登錄目前角色技能圖示的方法。

-- 技能圖示的RUID值，會根據登錄在skillLogic的技能資訊，從表格中取得RUID值。
self.skillRenderer.ImageRUID =
_DataSetLogic.skillInfoTable[_SkillLogic.skillId].UI_RUID
-- 為了顯示為可使用技能的狀態，將alpha值設為最大值。
self.skillRenderer.Color.a = 1
end

@ExecSpace("ClientOnly")
method void ShowResultPopup(string bestScore, string currentScore)
-- 這是顯示結果彈跳視窗的方法。

-- 登錄最高分文字。
self.resultBestScoreText.TextComponent.Text = bestScore

-- 登錄目前分數文字。
self.currentScoreText.TextComponent.Text = currentScore

-- 啟用結果彈跳視窗。
self.resultPopup.Enable = true;
end

@ExecSpace("ClientOnly")
method void ReturnToMenu()
-- 這是在遊戲結束後將狀態還原為主選單的方法。

-- 停用在遊戲中顯示的UI元素。
self:ShowPlayerInfo(false)

-- 停用角色選擇彈跳視窗。
self.characterSelectPopup.Enable = false
-- 停用遊戲結束後顯示的結果彈跳視窗。
self.resultPopup.Enable = false
-- 啟用應顯示在主選單標題上的面板物件。
self.mainTitlePanel.Enable = true
end

@ExecSpace("ClientOnly")
method void SetScoreText(number value)
-- 這是將作為參數傳入的值設為UI Score文字的方法。

-- 參數1 number value : 傳入要設為分數值的數值。

self.inGameCurrentScoreText.Text = tostring(math.floor(value))
end
```

```
@ExecSpace("ClientOnly")
method number SetHpSliderValue(number value)
-- 這是把hp滑桿的值設為參數值，並回傳目前滑桿Value值的方法。

-- 參數1 number value：傳入要設為滑桿的數值。

-- 將滑桿的值設為傳入的數值，並利用clamp設定為不得超過初始最大體力值。
self.hpSlider.Value = math.clamp(value, 0, _Config.maxHp)

-- 回傳目前滑桿的值。
return self.hpSlider.Value
end

end
```

最低製作標準

- 製作用來管理 使用之後要使用的DataSet的Logic。
 - 以範例世界 基準製作DataSetLogic以管理DataSet。
- 製作用來執行遊 進行與整體管理的Logic。
 - 以範例世界 基準，製作GameLogic來進行依遊 狀態而處理的內容。

應用與進階應用

- 將每秒增加分數的功能改成自己的分數給予公式，以改變遊 流程。
- 用Logic製作 使用用來管理預計不會變更之固定 的「Config」邏輯。
 - EX：透過變更 有設定的最大體力 來修改遊 難度。
 - EX：透過變更隨時間持續減少的體力來修改遊 難度。

[💡 基礎學習] 建構捲動系統

影片時間軸

- 可在課程影片中以下時間點學習。
- 建構捲動系統：00:25:46

學習目標

- 理解 了營造角色持續奔 效果，而讓背景與圖塊朝反方向移動的捲動系統核心原理。

- 使用OnUpdate方法與delta時間，撰寫不受幀數影響、能依時間以固定速度平順移動實體的本。
- 可透過控制背景面與地板圖塊的速度來設定遊難度。
- 理解依遊進行，關環境是如何動態變化的。

觀看該影片後可嘗試執行的內容

- 透過變更Map實體子層Background實體之BackgroundComponent的ScrollRate，設定適當的背景移動速度。

理解捲動系統的基本結構與原理

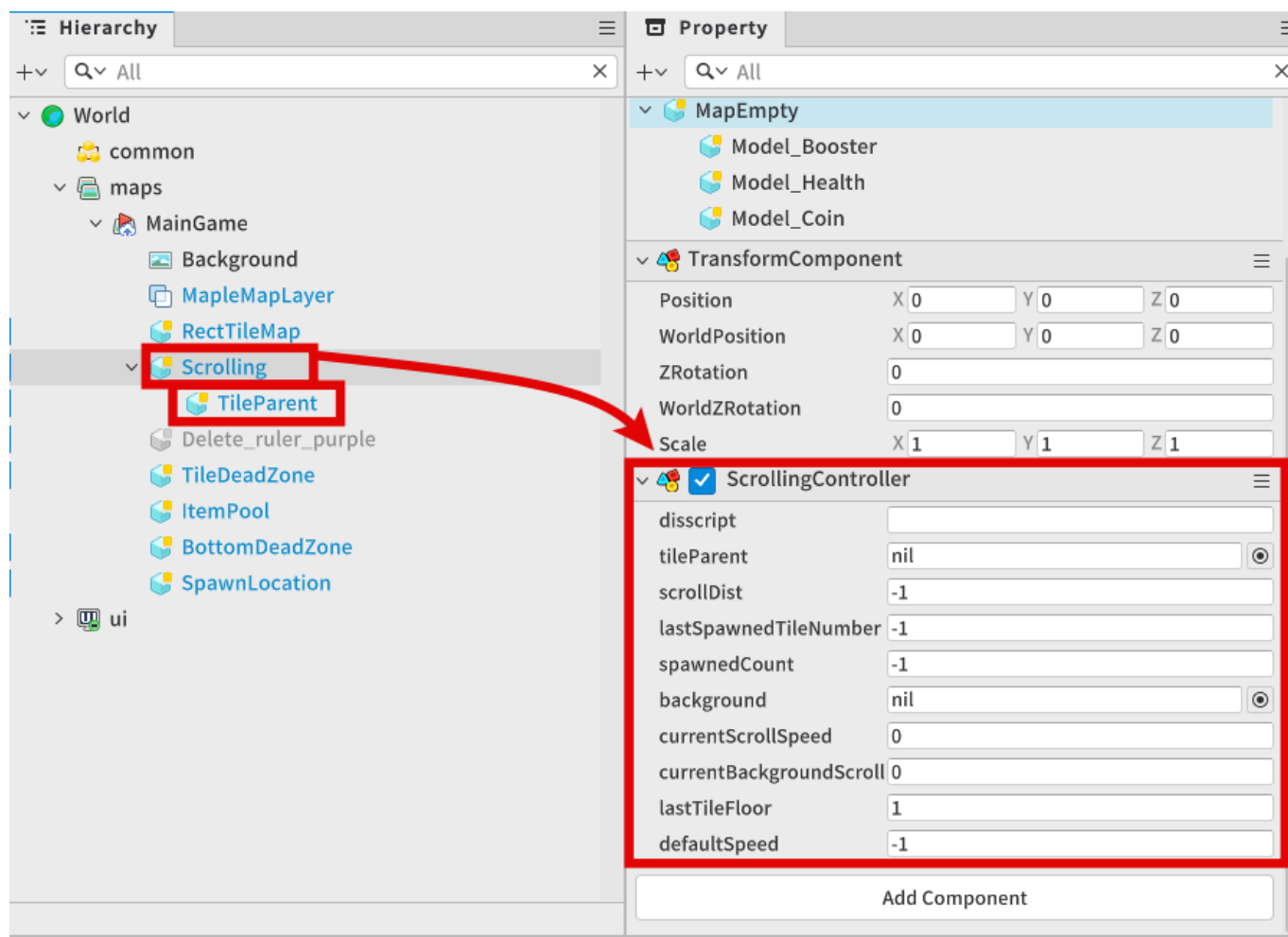
- 在酷遊中，若讓角色無限向右奔，將會遇到地圖必須無限延長製作的限制。
- 若角色座標無限變大，可能在系統上發生致命錯誤。
- 了克服物理限制穩定演出無限地圖，有必要導入捲動系統。

相對捲動



- 不移動攝影機與玩家角色，而是將其固定在面某個位置。
- 取而代之的是，將角色著的地板圖塊、障物、背景元素持續朝角色前進反方向，也就是左方移動。
- 透過這種方式，會在玩家視覺上生角色向前奔的錯覺效果。
- 如此一來，便無需擔心地圖大小或終點，也能製作原地持續奔的環境。

製作ScrollingController



- 在空白實體上 建立ScrollingController元件。
 - ScrollingController是管理遊 整體捲動環境的主體。
 - 將建立實體的TransformComponent之Position 設 (0,0,0)。
- 建立空白實體， 設定 附加ScrollingController元件之實體的子實體。
 - 該實體扮演移動整個圖塊集的父實體角色。
 - 將建立實體的TransformComponent之position 設 (0,0,0)，使其與父層位於同一位置。

建構ScrollingController Component

- 必須撰寫用來控制背景、地板圖塊的程式碼。
- 如下撰寫程式碼。
- 該程式碼是以Plain Text 基準撰寫。

```
@Component
script ScrollingController extends Component

property string discript = "" -- 這是用來在跑酷遊戲內負責整體橫向移動的邏輯。

property Entity tileParent = nil

property number scrollDist = -1 -- 這是用來儲存目前捲動距離的變數。// config中設定每個圖塊集距離為10

property number lastSpawnedTileNumber = -1 -- 這是最後建立的圖塊編號。
```

```
property number spawnedCount = -1 -- 這是用來記錄已建立多少個圖塊的變數。

property BackgroundComponent background = nil

property number currentScrollSpeed = 0 -- 這是用來儲存目前捲動畫面速度值的變數。

property number currentBackgroundScrollSpeed = 0 -- 這是用來儲存目前背景捲動畫面速度值的變數。

property number lastTileFloor = 1 -- 這是用來記錄最後建立的最後圖塊樓層的變數。

property number defaultSpeed = -1 -- 這是在選擇角色時設定的捲動畫面速度值。

@ExecSpace("ClientOnly")
method void Scroll(number moveDistance)
-- 將目前設定中的捲動畫面，依傳入的移動距離移動已啟用的圖塊集物件。

-- 移動父物件(TileParent)。
self.tileParent.TransformComponent.Translate(moveDistance, 0)
end

@ExecSpace("ClientOnly")
method void ConnectRefs()
-- 連接內部所需成員參照的方法。

-- 連接將成為所有圖塊集父層的物件實體。
self.tileParent =
_EntityService.GetEntityByPath("/maps/MainGame/Scrolling/TileParent")

-- 連接Background元件。
self.background =
_EntityService.GetEntityByPath("/maps/MainGame/Background").BackgroundComponent
end

@ExecSpace("ClientOnly")
method void Init()
-- 進行Scrolling Controller元件內部初始化的方法。

-- 進行內部參照連接。
self.ConnectRefs()

-- 將背景畫面捲動畫面速度值設為Config中設定的背景捲動畫面速度。
self.background.ScrollRate = _Config.backgroundScrollSpeed

-- 指定實際背景物件進行捲動畫面的速度值。
self.currentBackgroundScrollSpeed = _Config.backgroundScrollSpeed

-- 初始化已建立圖塊集數量。
self.spawnedCount = 0
end

@ExecSpace("ClientOnly")
```

```
method void OnUpdate(number delta)
-- 這是每一幀都會被呼叫的方法。

-- 如果主遊戲已結束
if _GameLogic.isEnd == true then
-- 中止Update方法運作。
    return
end

-- 依設定的捲動畫面速度執行地板物件移動。
self.Scroll(-1 * delta * self.currentScrollSpeed)
end

@ExecSpace("ClientOnly")
method void SpawnRandomTile()
-- 建立隨機圖塊集的方法。

-- 取得與最後建立圖塊編號不重複的隨機編號。
local randomNumber = self.GetRandomTileNumber()

-- 建立隨機編號的圖塊。
self.SpawnTile(randomNumber)
end

@ExecSpace("ClientOnly")
method number GetRandomTileNumber()
-- 用來取得隨機圖塊編號的方法。

-- 這是取得一個表格，其中包含可與最後建立圖塊集終點樓層相連的圖塊集編號之流程。
-- ex) 終點樓層：2樓 / 包含從2樓開始的圖塊集編號的表格 0
-- ex) 終點樓層：1樓 / 包含從2樓開始的圖塊集編號的表格 X
-- 這是為了建立可相連的圖塊集。

-- 這是程序化地圖建立的核心邏輯。

-- 假設最後建立圖塊的結束點樓層為N樓時，
-- 取得包含可從N樓開始的圖塊集編號的表格。
local tileSetFloorInfoTable =
_DataSetLogic.tilesFloorInfoTable[self.lastTileFloor]

-- 確認該表格的總數。
local tileSetTableTotalCount = #tileSetFloorInfoTable

-- 以1到圖塊種類數為範圍取得隨機編號。
local randomNumber = math.random(1, tileSetTableTotalCount)

-- 在只包含目標樓層的表格中，抽取第「randomNumber」個圖塊集編號。
local selectedTileNumber = tileSetFloorInfoTable[randomNumber]

-- 用來防止連續建立相同圖塊的邏輯。(若希望允許連續建立相同圖塊集，則不需執行下列語句。)

-- 如果任意抽出的數字編號與之前建立的圖塊編號相同(為防止選到重複圖塊而再次執行隨機抽取。)
```

```
if selectedTileNumber == self.lastSpawnedTileNumber then

    -- 再次抽取任意隨機數字。(在符合該樓層的圖塊集要素中抽取隨機要素。)
    local tempNumber = math.random(1, tileSetTableTotalCount)

    -- 如果臨時數字又再次相同
    if tempNumber == randomNumber then
        -- 將臨時抽出的數字值加1後利用餘數運算。
        tempNumber = (tempNumber + 1) % tileSetTableTotalCount + 1

        -- 因資料集的起始值是從1開始，若餘數運算結果為0
        if tempNumber == 0 then
            -- 則將tempNumber值設為1。
            tempNumber = 1
        end
    end

    -- 在只包含目標樓層的表格中，選擇任意抽出的第'tempNumber'個要素。
    selectedTileNumber = tileSetFloorInfoTable[tempNumber]
end

-- 回傳該圖塊集編號。
return selectedTileNumber
end

@ExecSpace("ClientOnly")
method void SetScrollSpeed(number speed)
    -- 調整地板圖塊物件移動速度的方法。

    -- 參數1 number speed : 傳入欲設定的速度值。

    -- 將目前速度值設為作為參數傳入的速度值。
    self.currentScrollSpeed = speed
end

@ExecSpace("ClientOnly")
method void SetBackgroundScrollSpeed(number speed)
    -- 調整背景畫面捲動畫面速度的方法。

    -- 參數1 number speed : 傳入欲設定的速度值。

    -- 將背景的捲動畫面速度設為參數值。
    self.background.ScrollRate = speed
end

@ExecSpace("ClientOnly")
method void ReturnAllTiles()
    -- 將所有已建立圖塊集物件全部回收(停用)的方法

    -- 從'tileParent'的子層中以遞迴方式存取，將'TileSet'元件存入表格。
    local tileSetTable =
    self.tileParent:GetChildComponentsByTypeName("TileSet", true)

    -- 依存有'TileSet'元件的表格數量進行巡覽。
```

```

for i = 1, #tileSetTable, 1 do

    -- 若各要素物件目前處於開啟(啟用)狀態時(為了只回收已開啟的物件。)
    if tileSetTable[i].Entity.Enable == true then

        -- 存取各要素的圖塊集元件。
        local tempTileset = tileSetTable[i].Entity.TileSet
        -- 將配置為圖塊集子層的道具全部回收。
        tempTileset:ReturnAllItems()
        -- 將該圖塊集再次回收至物件池。
        _ObjectPoolLogic:ReturnTile(tempTileset.Entity)
    end

end

end
end

method void SpawnTile(number tileSetNumber)
-- 建立圖塊集物件的方法。

-- 參數1 number tileSetNumber: 為建立該參數編號的圖塊集而傳入。

-- 從物件池取得「參數」編號的圖塊物件。
local spawnedTile = _ObjectPoolLogic:GetTile(tileSetNumber)

-- 記錄從物件池生成的圖塊集的結束點樓層。
self.lastTileFloor =
_DataSetLogic.tileSetInfoTable[tileSetNumber].LastFloor

-- 設定圖塊集相對於作為基準父物件的x軸距離。
-- spawnedCount(既有生成的圖塊集數) * 10(圖塊集Prefab的基準距離單位)
spawnedTile.TransformComponent.Position = Vector3(self.spawnedCount *
10,0,0)

-- 將已建立圖塊集數加1。
self.spawnedCount += 1
-- 將最後建立的圖塊集編號替換為從物件池取得的圖塊集編號。
self.lastSpawnedTileNumber = tileSetNumber

-- 進行從物件池取得的圖塊集初始化。
spawnedTile.TileSet:Init()
end

@ExecSpace("ClientOnly")
method void Reset()
-- 初始化遊戲進行所需成員要素的方法。

-- 將已建立的圖塊數初始化為0。
self.spawnedCount = 0
-- 將成為所有圖塊集物件父層的「tileParent」物件位置初始化為原點。
self.tileParent.TransformComponent.Position = Vector3(0,0,0)
-- 將背景畫面捲動畫面速度設為Config中作為基準設定的數值。
self:SetBackgroundScrollSpeed(_Config.backgroundScrollSpeed)

```

```

-- 開始遊戲時依序建立1到5號圖塊集。

-- 執行從1到5的迴圈時
for i = 1, 5, 1 do
    -- 建立第i號圖塊。
    self:SpawnTile(i)
end
end

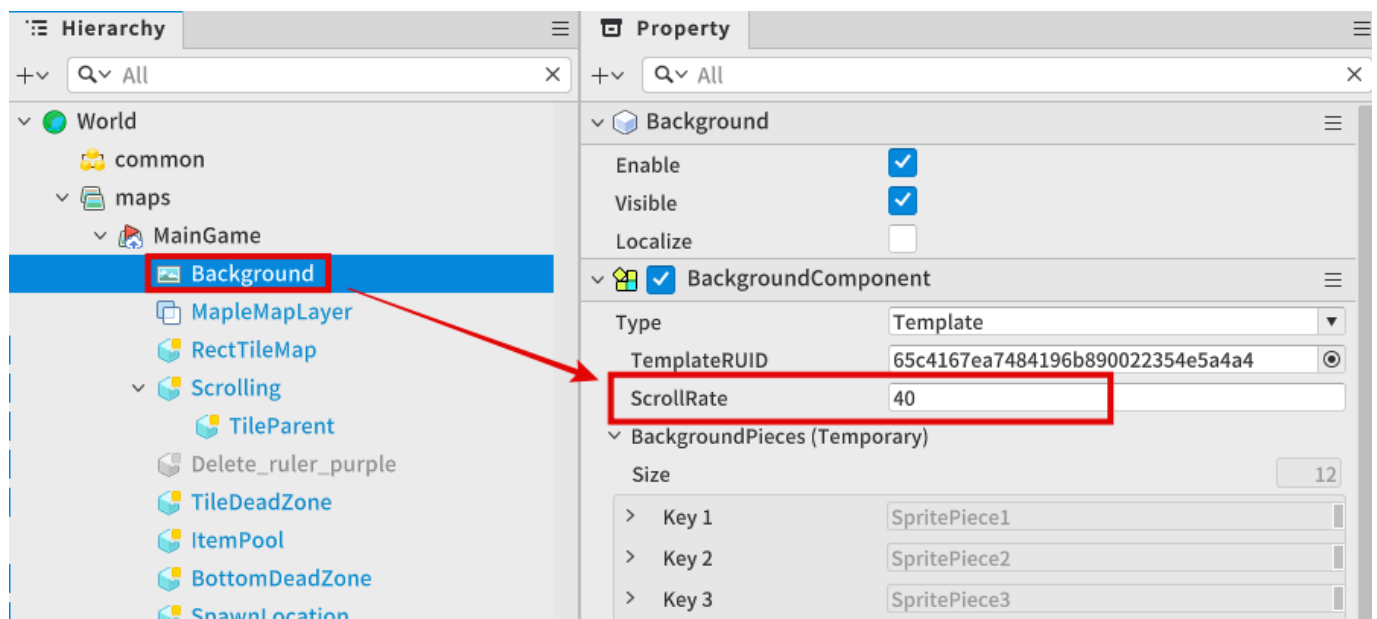
@ExecSpace("ClientOnly")
method void ChangeBackground()
-- 變更背景畫面的方法。

-- 參照登錄在遊戲邏輯中的目前階段編號，將背景畫面設為資料集中儲存的背景RUID值。
self.background:ChangeBackgroundByTemplateRUID(_DataSetLogic.stageRenderInfoTable[_GameLogic.currentStage].BackgroundRUID)
end

end

```

BackgroundComponent的ScrollRate



- 了控制背景圖片的捲動 面速度，需要存取Background實體。
- 設定BackgroundComponent的ScrollRate 。
- 使編輯器不是播放狀態，也可以直接在Scene分頁 面中確認。

以程式碼變更ScrollRate

- 這是在遊 進行途中 時控制背景捲動 面速度的方法。
- 參考登錄在ScrollingController 本中的SetBackgroundScrollSpeed方法。

```

void SetBackgroundScrollSpeed (number speed)
{

```

```
self.background.ScrollRate = speed
}
```

時變更階段的方法



- 設定 當遊 進行 經過一定時間後，背景、地板圖塊、障 物的外型會發生變化。

登錄各階段將套用的外型DataSet

Scene
Play (Preview)
StageRenderInfo ×

텍스트 입력

	1	2	3	+
ID	TileUID	ObstacleUID	BackgroundRU	
1	0376d76138514	b17ee15ba7eb4	65c4167ea7484	
2	5710f95597364	c9273530b80b4	01b9e67532df4	
3	9c67b9c20e434	061993f170e64	03a5acc0f4634	
+				

Workspace

+
Q All

- DefaultPlayer
- WorldConfig
- BaseEnvironment
- MyDesk
 - Components
 - DataTable
 - CharacterInfo
 - ItemInfo
 - SkillInfo
 - StageRenderInfo
 - StringTable
 - TileSetInfo

名稱	說明
TileUID	與角色互動的地板圖塊資源ID值。

名稱	說明
ObstacleRUID	與地板圖塊一同出現的障礙物元素資源ID值。
BackgroundRUID	作為backgroundComponent的templateRUID值，用來套用為背景的資源ID值。

- 需要一個用來儲存各階段應套用RUID 的DataSet。
- 在範例世界中，如圖所示分成1、2、3三個階段， 儲存各階段地板圖塊的RUID、障 物的RUID、背景圖片的RUID 。

階段變更方式

- 管理階段編號的資料，是以GameLogic的currentStage這個number型態變數來管理。
- 在GameLogic的OnUpdate方法中，經過一定時間後會呼叫變更階段的功能。
- 在範例世界中，雖然總共只登錄了3個階段，但利用%運算子，製作成1、2、3 -> 1、2這樣循環階段的結構。
- 地板圖塊與障 物則是在實體被 用時，確認GameLogic的currentStage編號， 套用符合目前編號的RUID 。
- 以下程式碼是GameLogic程式碼中的一部分。

```
GameLogic
number currentStage = 1 -- 這是用來記錄目前階段資訊的變數。
number stageTimer = 0 -- 這是用來記錄距離階段切換還剩幾秒的變數。

void OnUpdate(number delta) -- 每一幀都會呼叫的方法。
{
    if(stageTimer > 0) then -- 若計時器仍有剩餘時間
        self.stageTimer -= delta -- 計時器減去delta值

    else -- 若計時器沒有剩餘時間
        self.StageChange() -- 呼叫StageChange方法。
        self.stageTimer = _Config.stageChagneTime -- 將計時器初始化為初始時間值。
}

void StageChange() -- 變更階段的方法。
{
    -- 製作成可讓階段編號持續以1、2、3、1循環的結構。
    self.currentStage = (self.currentStage % (totalStageCount+1))+1
    scrollingController:ChangeBackground()
}
```

套用地板圖塊外型範例

- 在範例世界提供的地板圖塊功能所使用元件中，會在實體 用時，在Init方法內套用外型。
- TileSet元件 本會在下一單元透過程序化建立方式管理地圖中進行具體學習。

TileSet

```

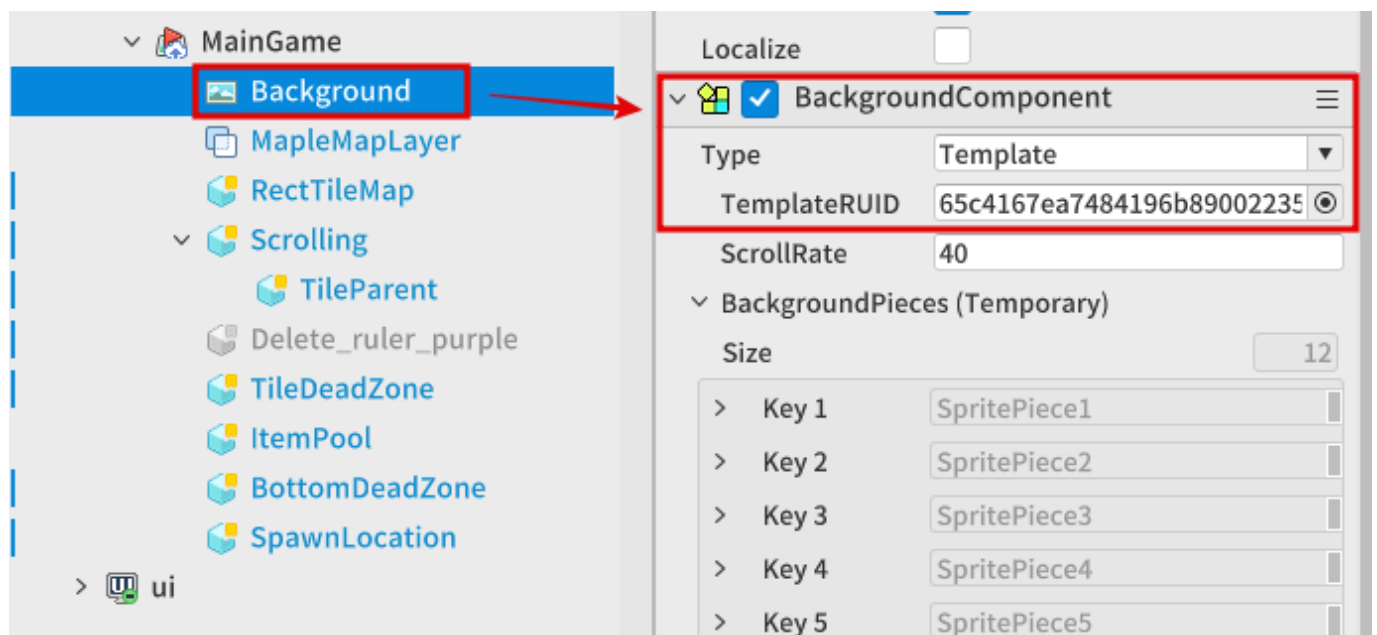
void Init()
{
    local renderers =

    self.Entity:GetChildComponentsByTypeName("SpriteRendererComponent", false)

    renderer.SpriteRUID =
        _DataSetLogic.stageRenderInfoTable[_GameLogic.currentStage].TileID
}

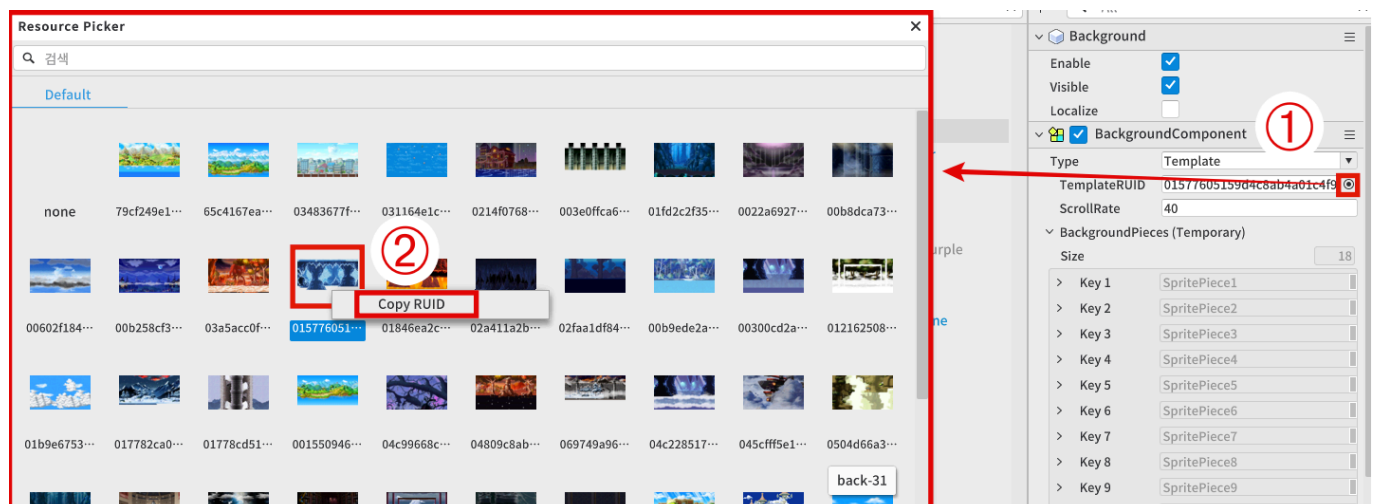
```

變更背景外型



- 若要變更管理背景圖片的BackgroundComponent的RUID，必須使用BackgroundComponent所提供的方法。
- 確認目前Background Type。
- 目前使用的型態是Template，因此用BackgroundComponent的ChangeBackgroundByTemplateRUID方法。
 - 楓之谷世界官方文件，BackgroundComponent [링크](#)

確認backgroundTemplateRUID



1. 了確認background的TemplateRUID，開 BackgroundComponent的RUID的資源選擇器視。
2. 在想要的資源圖像上按右鍵後，透過Copy RUID取得其。

變更backgroundTemplateRUID

- 當GameLogic呼叫階段變更功能時，會在ScrollingController元件內透過程式碼變更背景圖片。
- 確認已登錄背景的型態，對應BackgroundComponent提供的RUID變更方法。

```
ScrollingController

BackgroundComponent background

void ChangeBackground()
{
    self.background:ChangeBackgroundByTemplateRUID(
        _DataSetLogic.stageRenderInfoTable[_GameLogic.currentStage].BackgroundRUID)
}
```

最低製作標準

- 製作基礎地板圖塊的建立與地板圖塊移動功能。

應用與進階應用

- 可在管理地板圖塊、障物、背景RUID的DataSet內新增RUID，以組成更富的階段。
- 可調整階段切換計時器，在想要的時機切換階段。

#[💡 基礎學習]透過程序化建立方式管理地圖

影片時間軸

- 可在課程影片中以下時間點學習。
- 透過程序化建立方式管理地圖：00:33:49

學習目標

- 理解無限延展地圖的程序化地圖生成念與實務上的必要性。
- 學習將地圖切分固定大小的區塊，透過各區塊的起始樓層與結束樓層資料配對，讓地圖自然銜接的邏輯。
- 透過本無限生成地圖，製作在地板圖塊生成位置生新地板圖塊實體的方法。

觀看該影片後可嘗試執行的內容

- 在DataSetLogic管理的樓層管理表中新增資料，以加入新的圖塊資訊。
- 按下播放按 確認是否會隨機選取符合條件的下一個圖塊集， 正確生成在最後的X座標。
- 在遊 中實際測試生成的地圖平台高低是否自然、圖塊之間是否沒有空隙。

理解程序化地圖生成的 念

- 所謂程序化地圖生成，是指與其從頭到尾手動配置地形，不如依照 定規則與演算法，在遊 進行時 時生成地圖的系統。
- 這是在必須不斷奔 的無限 酷類型中，用來無限制延長地圖長度， 在每次遊玩時建立多樣化地形樣式所需的核心理論技術。

克服無限地圖的限制

- 在 酷遊 中，若要把玩家奔 的地圖從頭到尾都在編輯器中事先做得 長，不只工作量龐大，在系統記憶體層面也有大限制。
- 了解決這個問題，便導入一種會依照遊 進行狀況 時建立新地圖碎片的「程序化建立」。
- 套用這種功能後，就像在奔 列車前方持續鋪設鐵軌一樣， 能無限延長地圖，也能有效率地管理系統資源。

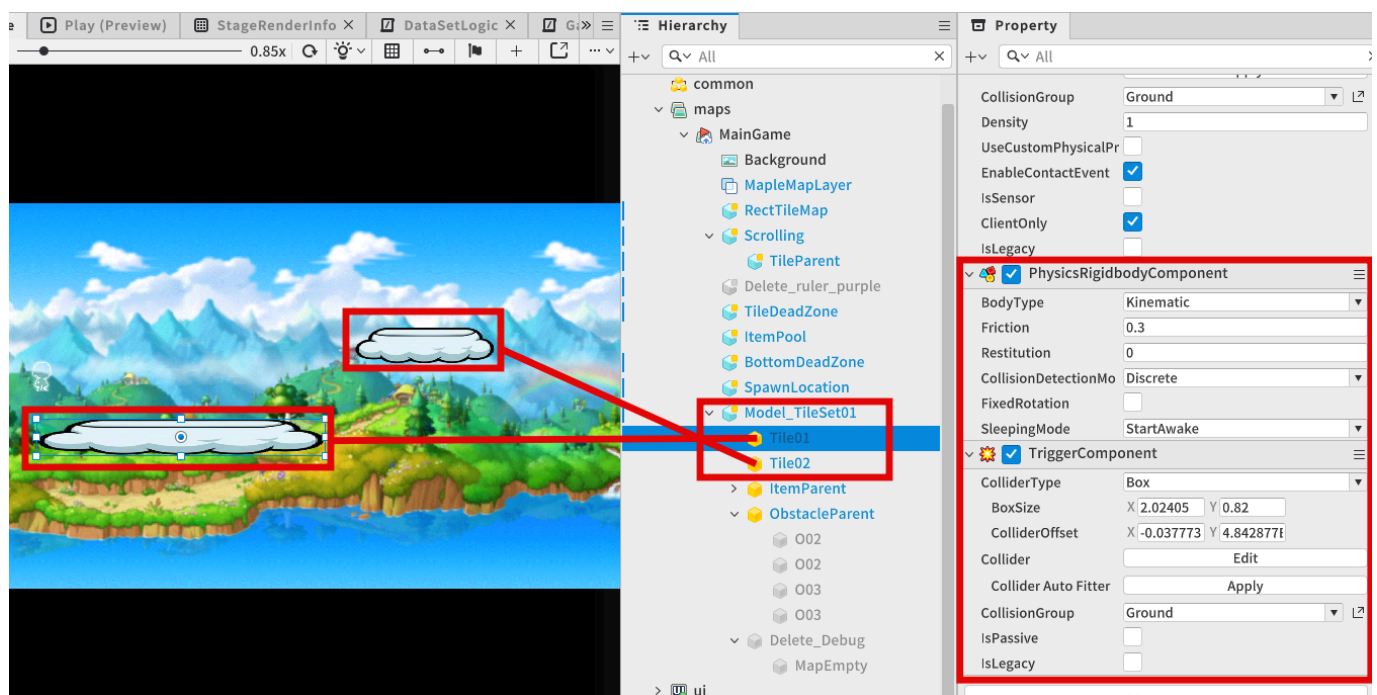
以圖塊集(區塊) 單位的分割管理與隨機性

- 時生成地圖時， 不是隨機一格一格像素或圖塊地打出來，而是先製作 管理多種固定長度的地圖碎片(區塊)樣式。
- 本會從這些預先做好的碎片中，依照規則隨機選擇一個，再像樂高積木一樣接到前一塊的尾端。
- 透過這種方式，不但能維持地圖結構的穩定性，也能讓玩家每次遊玩時遇到不同類型的地形，進而大幅提升遊 重玩性。

製作區塊地圖

- 所謂區塊，是指在程序化地圖建立系統中，用來銜接地形的預先製作固定大小地圖碎片。

建立地板圖塊實體



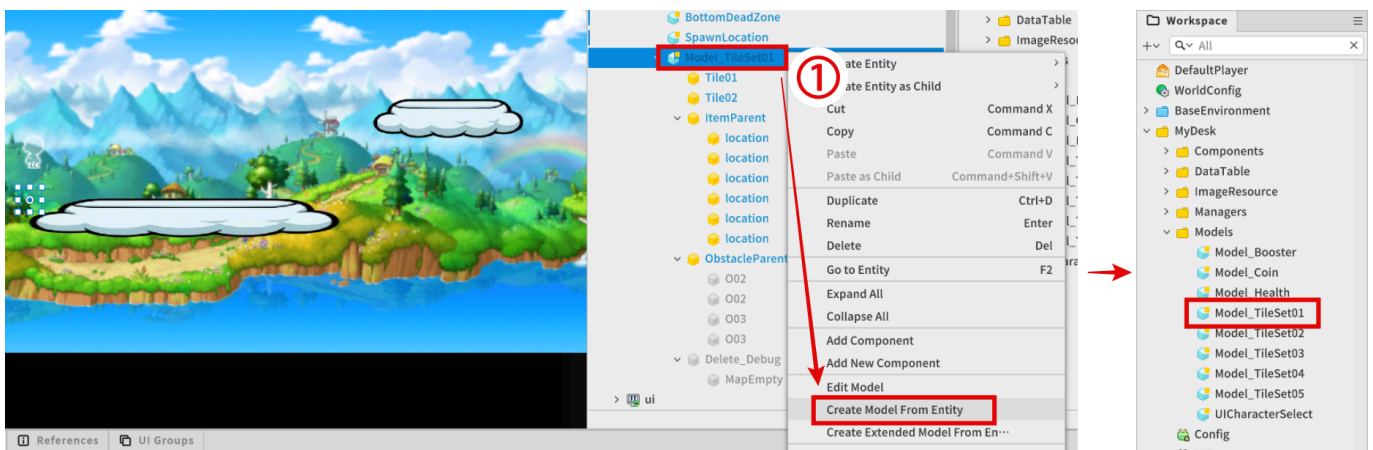
- 了建立一個單位的區塊，先建立一個作 圖塊父層的空白實體。

- 此時，為了讓在編輯器中配置圖塊時作業更順利，請將父實體座標設為原點(0,0,0)。
- 登錄多個子實體，附加SpriteRendererComponent。
- 設定SpriteRendererComponent的RUID以登錄外型。
- 除了父實體外，對各地板實體登錄PhysicsRigidbody、PhysicsCollider元件，設定成可與角色互動。
- 透過PhysicsCollider元件中的collider edit按鈕，修改碰撞器的位置與尺寸。



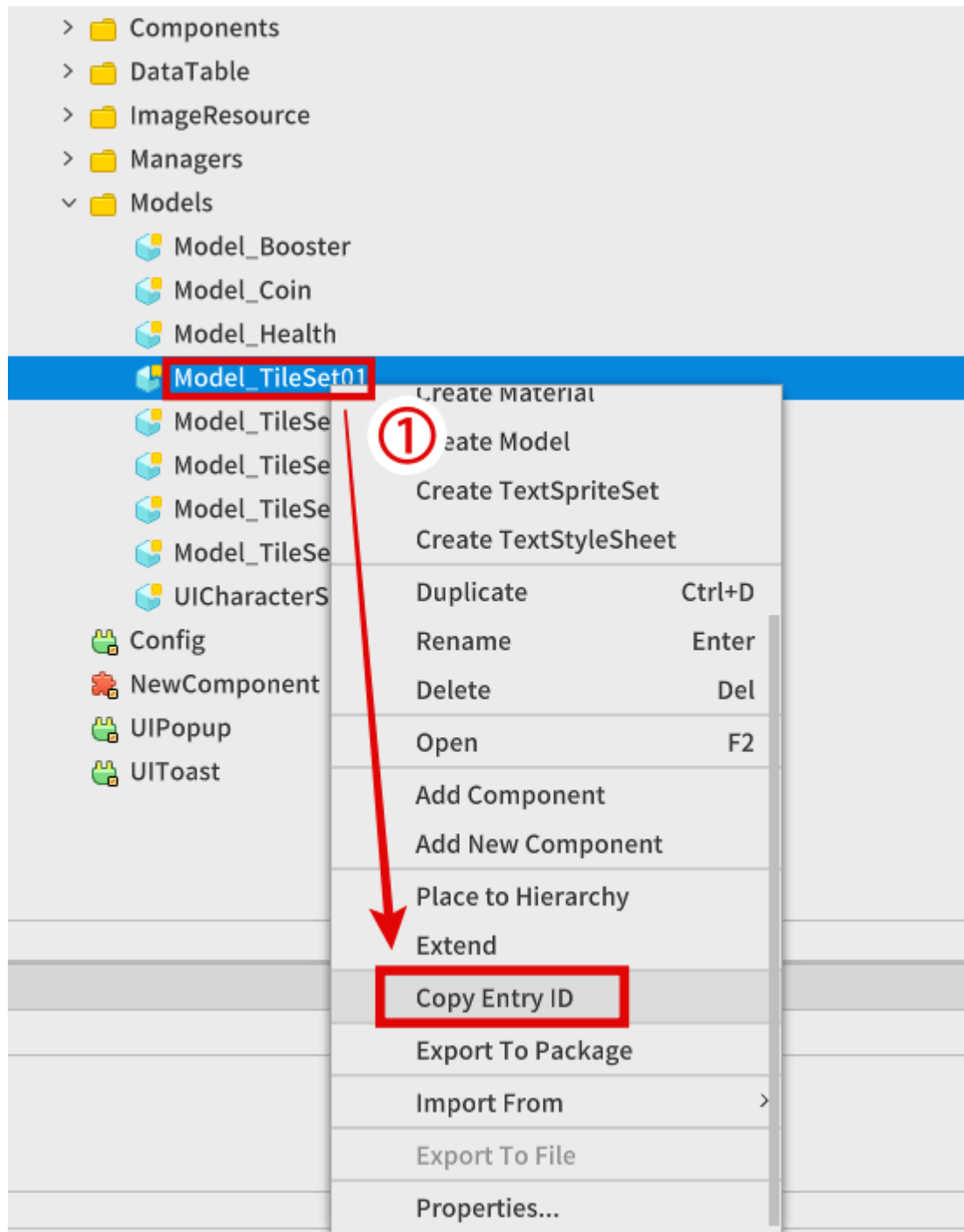
⚠ 由於所有區塊都會以固定週期生成在相同位置，若每個區塊地圖的長度、高度不同，就可能看起來像地圖斷裂的樣子。在範例世界的製作中，一個圖塊集的x軸間距是以10為基準製作的。

登錄製作好的地板圖塊集模型



- 為了讓預先製作好的區塊地圖能在需要時透過生成呼叫工作實體配置，需要先登錄該實體的Model。
- 在完成製作的圖塊集實體上按右鍵後，選擇Create Model From Entity。
- 可在Workspace中確認該實體的模型已被建立。

建立製作好的模型



- 因 已經製作出一個原始地板圖塊，所以可以在Hierarchy中依需要輕 生成該圖塊。
- 在Workspace中對已建立的模型按右鍵後，選擇CopyEntry ID來複製Entry ID 。
- 在程式碼中使用EntryID與SpawnService，在世界中生成與模型相同的實體。
 - 楓之谷世界官方文件，SpawnService[링크](#)

建立範例

```
void Spawn()
{
    local entryID = "model://02e63fc1-cca0-4b98-ada3-d5a36e26f7de"
    _SpawnService.SpawnByModelId(entryID,"TempEntity",Vector3.zero,nil)
}
```

登錄各圖塊的起始點與結束點樓層資訊

	1	2	3
ID	FirstFloor	LastFloor	EntryID
1	1	2	model://02e631
2	2	1	model://d1ca0
3	1	2	model://ec391
4	2	1	model://d00b8
5	1	1	model://30079

- 製作資料表，以直觀管理各圖塊的起始與結束資訊。
- 表中記錄各圖塊起始部分的樓層、結束部分的樓層，也就是FirstFloor、LastFloor資訊，以及各Model的EntryID。

在DataSet中登錄各地板圖塊資料

①

ID	FirstFloor
1	1
2	2
3	1
4	2
5	1

②

TableName	FirstFloor	TilesNumber
tilesFloorInfoTable	[1]	1,3,5
	[2]	2,4

- 這是實際儲存的DataSet(TileSetInfo)結構的一部分。
 - 這是顯示DataSetLogic將以何種方式儲存到表格中的表。
- 範例世界的地板圖塊製作成1樓與2樓兩種。因此tileFloorInfoTable也分成[1]與[2]兩個陣列，分別管理與存取。

DataSetLogic的tileFloorInfoTable儲存程式碼

```
table tilesFloorInfoTable = {[1]={}, [2]={}}
```

```
void InitTileSetInfoTable()
{
```

```
    -- 用來儲存圖塊物件相關資料集的方法。
```

```
    -- 用來在本地端存取儲存在工作空間資料集名稱的變數。
```

```
    local dataSetName = "TileSetInfo"
```

```

-- 目標資料集的Row數。
local rowCount = self:GetDataSetRowCount(dataSetName)

-- 儲存圖塊集種類數。
self.totalTileSetCount = rowCount

-- 用來儲存各列資料的變數。
local rowData

-- 從1開始依資料集Row數進行巡覽。
for i = 1 , rowCount , 1 do
    rowData =
    {
        -- 圖塊集物件第一個圖塊的樓層。
        FirstFloor =
tonumber(_DataService:GetCell(dataSetName,i,"FirstFloor")),
        -- 圖塊集物件最後一個圖塊的樓層。
        LastFloor =
tonumber(_DataService:GetCell(dataSetName,i,"LastFloor")),
        -- 用來建立圖塊集物件的原始實體模型(Prefab)Id。
        EntryID = _DataService:GetCell(dataSetName,i,"EntryID")
    }

    -- 將以rowData儲存的資料存入表格。
    self.tileSetInfoTable[i] = rowData

    -- 將剛登錄的圖塊集樓層存入變數。
    local firstTileFloor = self.tileSetInfoTable[i].FirstFloor

    -- /分支1/ 若第一個圖塊樓層為1樓
    if firstTileFloor == 1 then
        -- 存入1樓表格清單。
        table.insert(self.tilesFloorInfoTable[1],i)
    -- /分支2/ 若第一個圖塊樓層不是1樓
    else
        -- 存入2樓表格清單。
        table.insert(self.tilesFloorInfoTable[2],i)
    end
end
}

number GetDataSetRowCount(string Name)
{
    -- 回傳目標資料集的Row數。

    -- 參數1 string Name : 傳入目標資料集名稱。

    -- 回傳確認過的資料集Row數。
    return _DataService:GetRowCount(Name)
}

```

使用登錄在DataSet中的各地板圖塊資料

- 儲存好的tilesFloorInfoTable中，各樓層的表格可分類如下。
 - tilesFloorInfoTable[1]：存有從1樓開始的地板圖塊集編號的表格
 - tilesFloorInfoTable[2]：存有從2樓開始的地板圖塊集編號的表格
- 確認最後建立的地板圖塊終點樓層， 從各樓層表格中抽出隨機圖塊編號。
- 登錄最後圖塊樓層的方法，是更新最後建立的圖塊集的lastFloor 。

ScrollingController中建立圖塊方法的一部分

```
number lastTileFloor
```

```
void SpawnTile(number tileSetNumber)
{
    -- 從物件池取得「參數」編號的圖塊物件。
    local spawnedTile = _ObjectPoolLogic:GetTile(tileSetNumber)

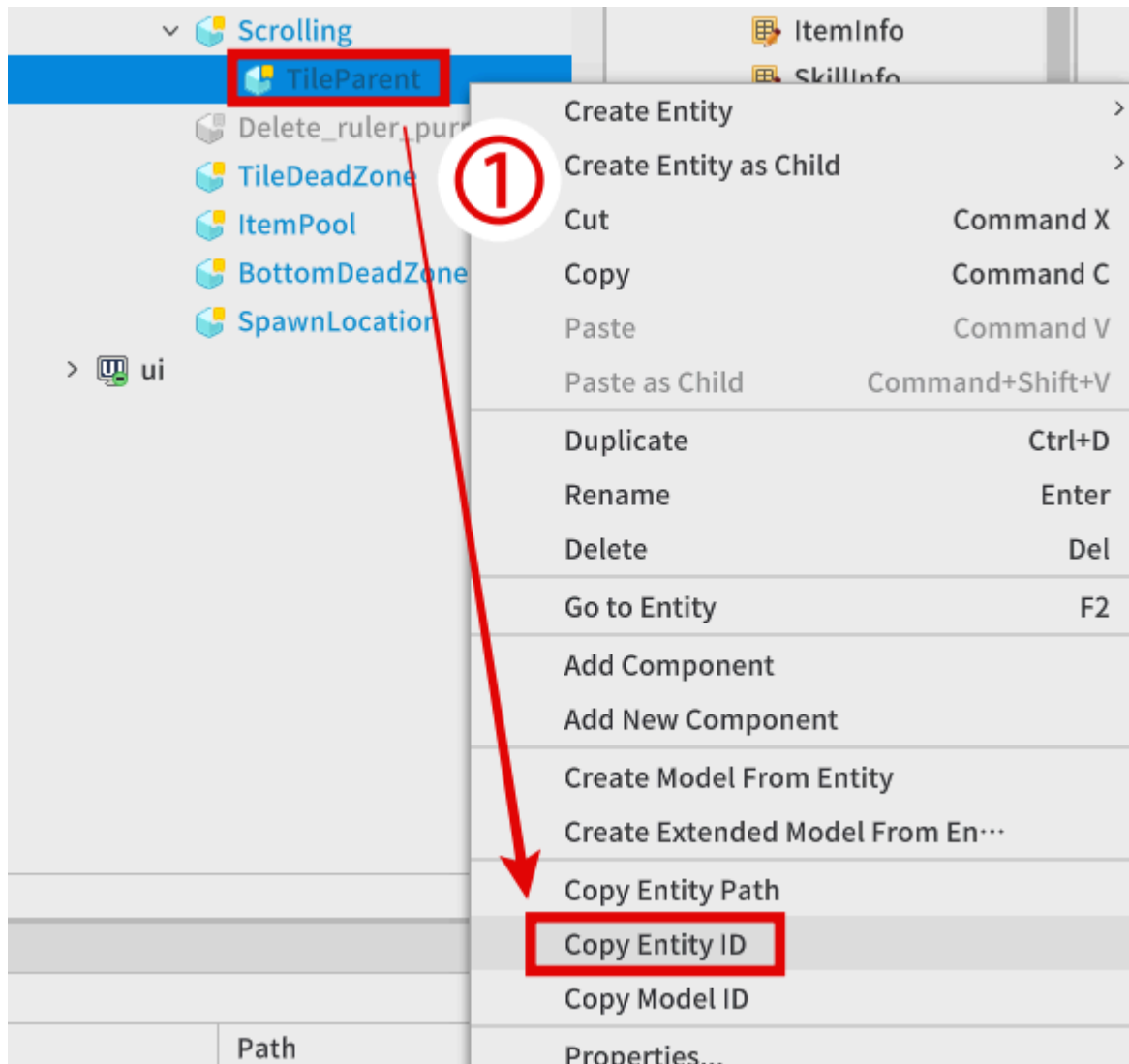
    -- 記錄從物件池生成之圖塊集的結束點樓層。
    self.lastTileFloor =
_DataSetLogic.tileSetInfoTable[tileSetNumber].LastFloor
}
```

多個區塊地圖的移動原理



- 在世界中，實際負責移動的是Scrolling的子實體TileParent。
- 不是每個區塊地圖圖塊集實體各自執行移動，而是把 們登錄 實際執行移動的TileParent的子實體，使其自動著移動。
- 也就是說，TileParent在正式開始遊 時會持續朝-x軸(由右往左)移動，而若登錄 TileParent實體的子實體，子實體也會自動同樣移動。

登錄 TileParent實體的子實體



- 可透過EntityService取得Hierarchy中的實體。
- 若要取得目標實體的Entity ID，可在Hierarchy中對該實體按右鍵後，透過Copy Entity ID複製ID。
- SpawnService的SpawnByModelId方法可在第4個參數(Parameter)指定建立物件的父層(Parent)實體。
- 從先前製作的地板圖塊集模型生成新實體時，設定其Parent。

使用SpawnService的SpawnByModelId方法範例

```
void SpawnTile()
{
    // 從DataSetLogic取得2號區塊地圖圖塊的EntryID
    local entryID = _DataSetLogic.tileSetInfoTable[2].EntryID

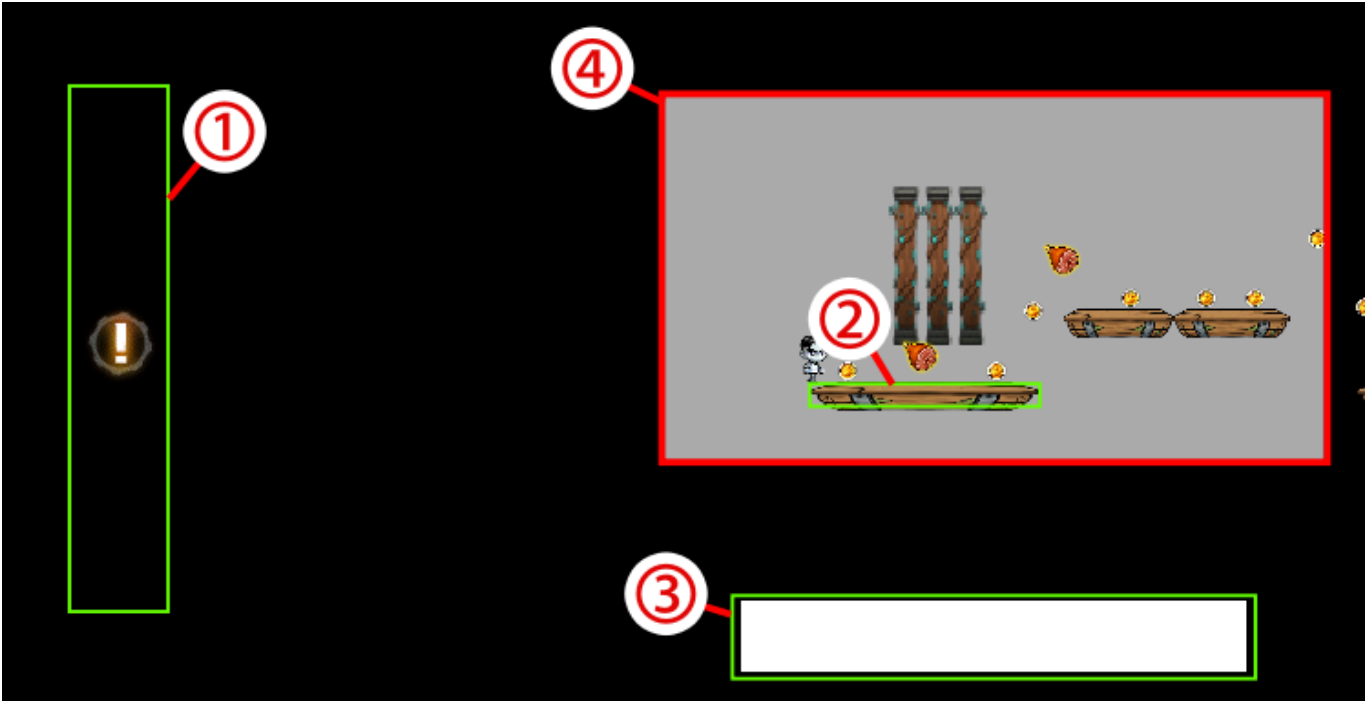
    // 將要生成的新實體名稱
    local entityName = "TempEntity"

    // 將要生成的座標值
    local spawnPosition = Vector3(50,0,0)

    // 新生成實體的父層實體
    local tileParentEntity = _EntityService:GetEntity("d24b5917-1821-4743-ae07-77cc9bfc2d0a")
}
```

```
_SpawnService:SpawnByModelId(entryID,entityName,spawnPosition,tileParentEntity)
}
```

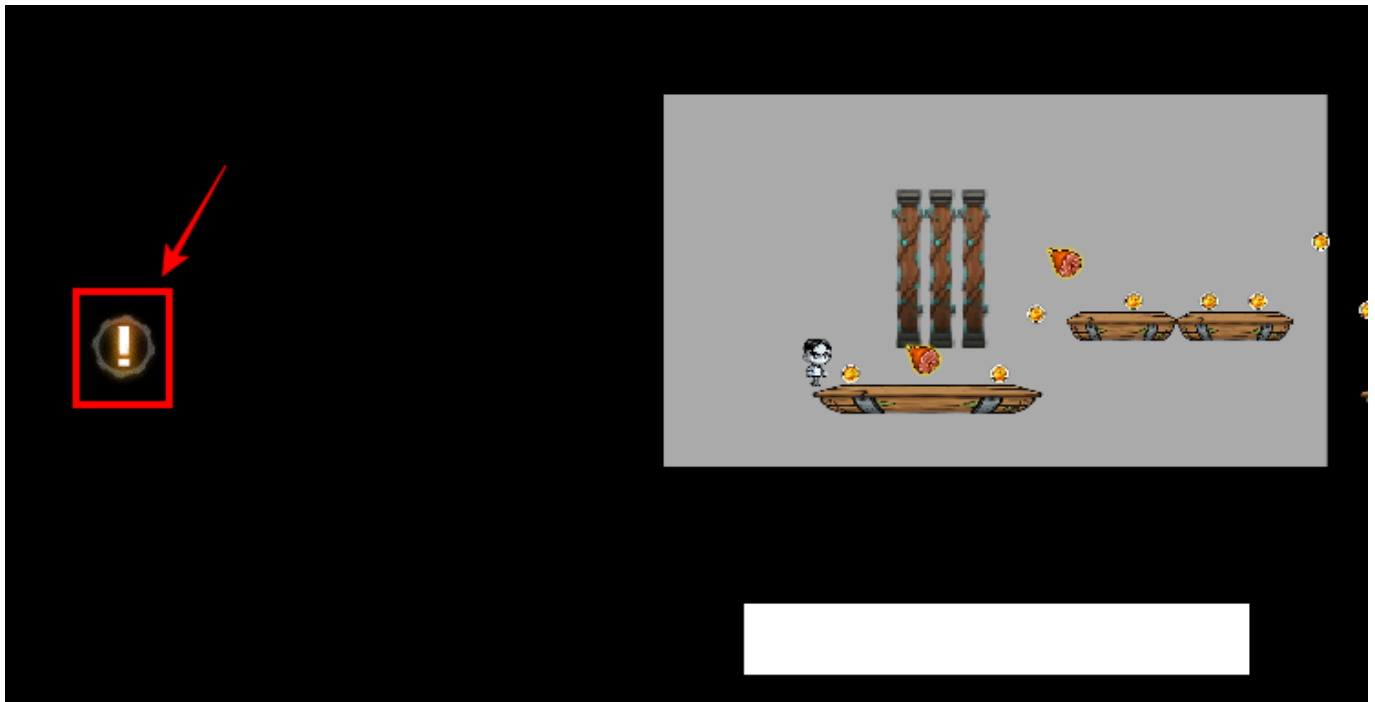
移除超出 面區域的圖塊集



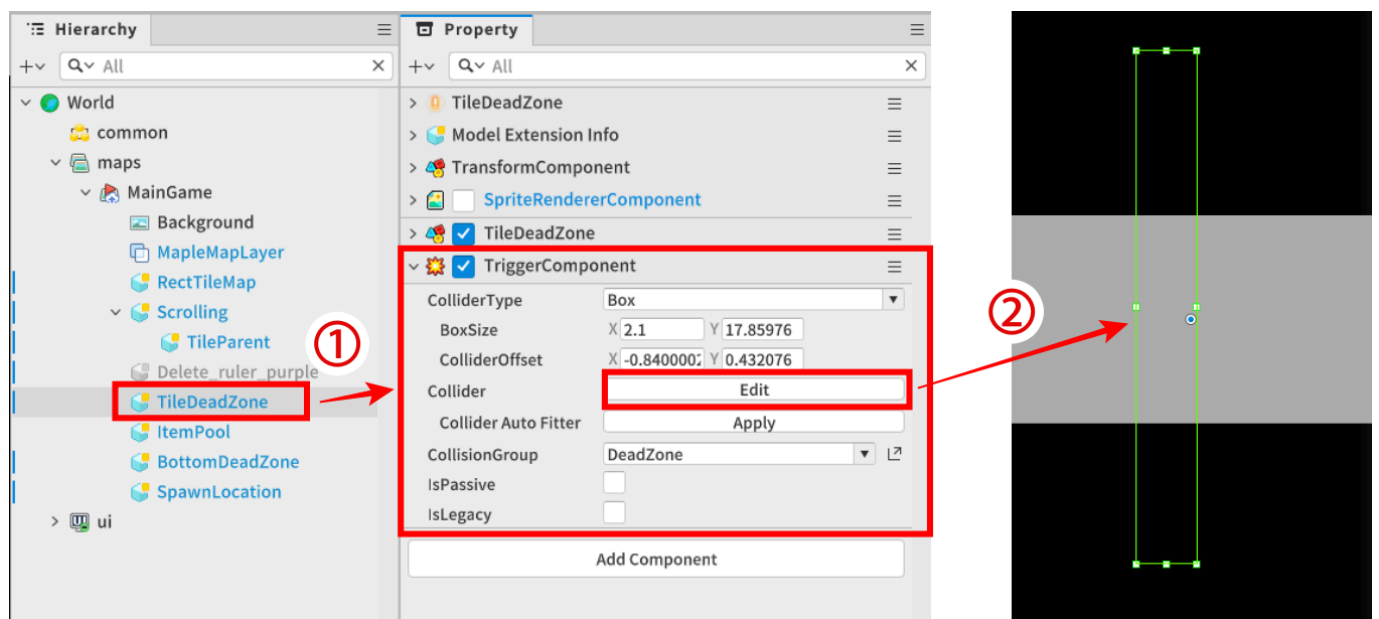
項目	名稱	說明
1	DeadZone	設置此元件，是為了偵測往左移動的地板圖塊的碰撞，並移除目標實體。
2	Tile Trigger	這是用來與DeadZone互動的Collider。
3	DeadZone	這是當角色朝下方掉落時，能立即結束遊戲的實體。
4	View Area	這是實際Play環境中可見的攝影機區域。

- 完全離開 面左側邊界的圖塊實體，在系統上已不需再渲染。
- 需要配置能與地板圖塊偵測 撞 移除目標圖塊集的實體。

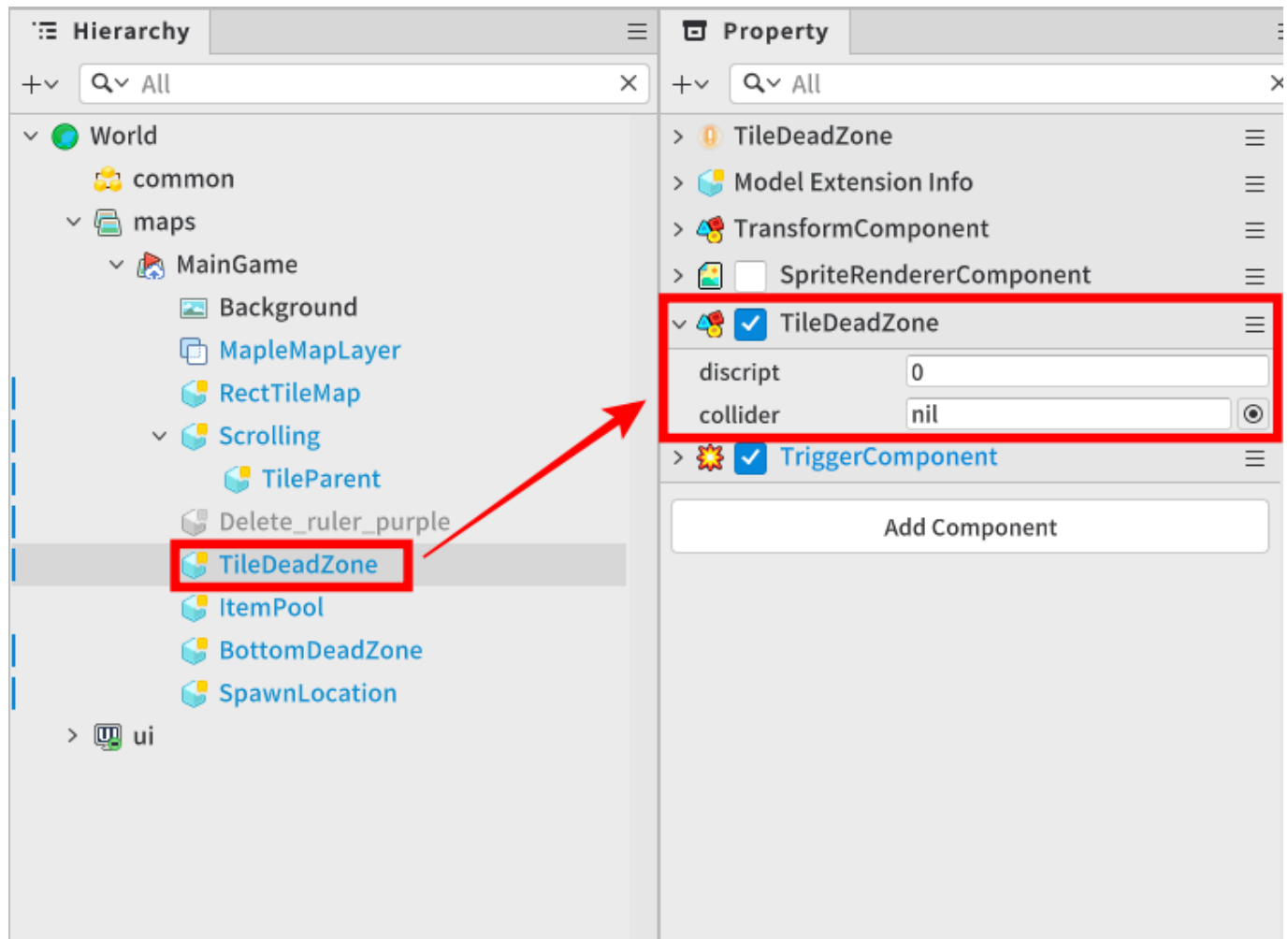
製作與Tile互動的DeadZone



- 了不讓玩家看見地板圖塊被移除的狀況，在離攝影機區域 遠的地方建立新實體。



1. 在作 TileDeadZone角色的實體上建立TriggerComponent。
2. 透過已附加TriggerComponent的Collider的Edit按 ，設定 撞偵測範圍。



1. 建立TileDeadZone元件 本 登錄至TileDeadZone。
2. 以TileDeadZone的Plain Text登錄程式碼內容。

```
@Component
script TileDeadZone extends Component

property number discript = 0 -- 這是負責圖塊物件銷毀(停用)的元件。

property PhysicsColliderComponent collider = nil

@ExecSpace("ClientOnly")
method void ConnectRefs()
-- 連接內部所需成員參照。

-- 連接用來執行碰撞偵測的PhysicsCollider元件。
self.collider = self.Entity.PhysicsColliderComponent
end

@ExecSpace("ClientOnly")
method void Init()
-- 進行TileDeadZone元件初始化的方法。

-- 執行以變數宣告的參照連接。
self:ConnectRefs()
end
```

```

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- 這是在世界執行時呼叫的方法。

-- 進行該元件初始化。
self:Init()
end

@EventSender("Self")
handler HandleTriggerEnterEvent(TriggerEnterEvent event)
-- 這是在偵測到觸發碰撞時發生的事件。

-- 儲存觸發碰撞的實體。
local TriggerBodyEntity = event.TriggerBodyEntity
-- 確認碰撞觸發的Collision群組的id值。
local TriggerGroup =
tostring(TriggerBodyEntity.TriggerComponent.CollisionGroup.Id)

-- 若將上述群組id值作為key代入Config中登錄的表格時，回傳的字串為「Ground」
-- 也就是說，若以觸發偵測到的碰撞體是Ground
if _Config.triggerGroup[TriggerGroup] == "Ground" then

    -- 就執行回收圖塊集物件。
    _ObjectPoolLogic:ReturnTile(TriggerBodyEntity.Parent)

    -- 要求捲動控制器生成接續在最後圖塊後面的隨機圖塊。
    _GameLogic.scrollingController:SpawnRandomTile()
end
end

end

```

最低製作標準

- 製作成起始樓層與結束樓層配對資料能正確運作，且圖塊之間不會斷裂。
- 製作成依照捲動 面移動速度，新的地圖碎片會及時生成，不會讓角色因缺少平台而掉出地圖外。

應用與進階應用

- 可登錄多種不同地板圖塊模型，藉由擴充或更改種類，調整隨機生成權重，使高難度地圖碎片更常出現。

[💡 基礎學習] 建立物件池

影片時間軸

- 可在課程影片中以下時間點學習。
- 建立物件池：00:44:52

學習目標

- 掌握當實體(物件)生成與銷 反覆發生時造成效能下降的原因， 理解 了解決此問題而需要物件池。
- 可在遊 開始時預先生成所需地圖區塊或障 物實體， 親自製作儲存在待機狀態中的池資料結構。
- 撰寫將超出 面外的實體不刪除而改 停用後回收到池中， 在需要時重設位置再取出使用的重複利用方法。

觀看該影片後可嘗試執行的內容

- 使用邏輯 本，在遊 初始化階段預先生成指定數量的圖塊集實體後，以停用狀態儲存起來。
- 實際測試 到 面左端DeadZone的圖塊是否不是被刪除，而是在屬性被初始化後再次回到物件池待機。
- 在程序化地圖生成時，不是每次都建立新的實體，而是確認會不會從物件池取出待機中的實體， 正常接續在前一個圖塊的終點座標。

什 是物件池？

- 在遊 進行時會使用大量實體。
- 其中，像子彈或平台這種會頻繁生成的要素，過一段時間後就必須從 面上消失 被銷 。
- 但是生成(Spawn)與銷 (Destroy)作業會伴隨記憶體配置與釋放，因此如果這個過程在短時間內反覆進行，就會對電腦運算造成 大負擔(掉幀)。
- 但若事先建立實體， 不是消滅而只是停用，就能減少電腦負擔。
- 因 是像開關一樣，只切換狀態 重複利用記憶體。
- 也就是說，**物件池(Object Pool)**是一種核心最佳化， 會把預先建立的實體放進「水池(Pool)」中，需要時再取出使用 回收，藉此將不必要的生成與銷 降到最低。

製作物件池

- 範例世界的物件池，是像GameLogic或DataSetLogic一樣，製作成可在任何地方以全域方式存取， 輕 借出與回收實體的集中管理(管理者)結構。
- 在Workspace中建立新的邏輯， 命名 ObjectPoolLogic。

撰寫ObjectPoolLogic 本

- 必須如下撰寫程式碼。
- 該程式碼是在Plain Text環境下撰寫。

```
@Logic
script ObjectPoolLogic extends Logic

property number discript = 0 -- 這是高效率管理頻繁發生建立與銷毀物件的邏輯。

property table tilePool = {} -- 這是用來扮演管理圖塊集物件的池角色的表格。

property table itemPool = {} -- 這是用來扮演管理道具物件的池角色的表格。
```

```

property Entity ItemPoolParent = "9e9e98bd-4f7b-46ed-9a93-f0e7d74fac1c"

property table itemDataSetIDTable = [{"Coin"] = 1 , ["Health"] = 2 ,
["Booster"] = 3} -- key : 道具名稱 , value : 用來儲存資料集Index編號的表格。

@ExecSpace("ClientOnly")
method void ReturnTile(Entity entity)
-- 這是將圖塊物件回收至物件池的方法。

-- 參數1 entity : 傳入欲回收(停用)的物件。
-- 參數2 tileNumber : 傳入圖塊編號。

-- 取出圖塊集實體的編號。(為了依照表格內編號管理圖塊集物件。)
local tileNumber = self.GetTileNumber(entity)

-- 檢查圖塊編號對應的池(表格)是否有效。
if invalid(self.tilePool[tileNumber]) == false or self.tilePool[tileNumber]
== nil then
    -- 若表格不存在,則新建新的表格空間。
    self.tilePool[tileNumber] = {}
end

-- 將目標實體(物件)儲存至目標表格。
table.insert(self.tilePool[tileNumber],entity)
-- 停用實體(物件)。
entity.Enable = false
end

@ExecSpace("ClientOnly")
method Entity GetTile(number tileNumber)
-- 從圖塊池取得指定編號圖塊實體的方法。

-- 參數1 number tileNumber : 傳入特定圖塊集編號。

-- 使用math.floor方法去除tileNumber的小數部分。 ex) 4.0 => 4
tileNumber = math.floor(tileNumber)

-- 進行目標池(表格)有效性檢查。

-- 若不存在管理傳入tileNumber圖塊物件的池(表格)
if invalid(self.tilePool[tileNumber]) == false or self.tilePool[tileNumber]
== nil then
    -- 則建立新的表格空間。
    self.tilePool[tileNumber] = {}
end

-- /分支1/ 若沒有則新建後回收。

-- 若目標池內已無物件(將從模型(Prefab)重新建立物件。)
if #self.tilePool[tileNumber] <= 0 then

    -- 確認表格集中儲存的EntryID。
    local entryID = _DataSetLogic.tileSetInfoTable[tileNumber].EntryID
    -- 以預先儲存的(Prefab)名稱類型儲存名稱。

```

```
    local name = "Model_TileSet0"..tileNumber

    -- 在臨時位置生成目標模型(Prefab)物件。
    local tempTile =
    _SpawnService:SpawnByModelId(entryID,name,Vector3.up*100,_EntityService:Get
    Entity("d24b5917-1821-4743-ae07-77cc9bfc2d0a"))

    -- 回收該物件。
    return tempTile
end

-- /分支2/ 若有則取出最後一個元素。

-- 確認目前目標表格數量。
local tableCount = #self.tilePool[tileNumber]

-- 取出目標表格最後一個元素，並將該實體從表格移除。
local tempTile = table.remove(self.tilePool[tileNumber],tableCount)

-- 啟用選取的物件。
tempTile.Enable = true

-- 回收物件。
return tempTile

end

@ExecSpace("ClientOnly")
method Entity GetItem(string itemName)
-- 從物件池取得道具物件的方法。

-- 參數1 string itemName: 傳入道具名稱。

-- 從道具池取得存放目標道具的表格。
local targetTable = self.itemPool[itemName]

-- 當表格內元素為空時
if #targetTable == 0 then
    -- 依照Config設定的生成數量追加生成目標道具。
    self:CreateItem(itemName,_Config.itemPoolInitCount)
end

-- 從欲取得的表格中載入實體。(表格最後一個元素)
---@type Entity
local targetEntity = targetTable[#targetTable]

-- 因已儲存實體，因此移除表格最後一個元素。
table.remove(targetTable,#targetTable)

-- 啟用實體(物件)。
targetEntity.Enable = true
```

```
-- 回收已儲存的實體。
return targetEntity
end

@ExecSpace("ClientOnly")
method void ReturnItem(Entity entity)
-- 將道具物件再次回收至物件池的方法。

-- 參數1 Entity entity : 傳入欲回收的道具物件實體。

-- 確認回收道具的名稱。
local itemName = entity.Item.name
-- 將實體放入目標表格。
table.insert(self.itemPool[itemName],entity)

-- 從圖塊集原本的Parent (為配置圖塊集道具而預先登錄的物件)
-- 再將Parent移回ItemPool。
entity:AttachTo(self.ItemPoolParent)
-- 停用實體 (物件)。
entity.Enable = false
end

@ExecSpace("ClientOnly")
method void CreateItem(string itemName, number createCount)
-- 從模型 (Prefab) 重新生成道具物件並放入目標池的方法。

-- 參數1 string itemName : 傳入欲生成的道具名稱。
-- 參數2 number createCount : 生成數量, 決定要放入多少物件到物件池。

-- 宣告臨時表格。
local tempTable

-- 設定資料集要參照的Id編號。
local dataSetID = self.itemDataSetIDTable[itemName]

-- 載入資料集中目標道具模型的EntryID值。
local entryID = _DataSetLogic.itemInfoTable[dataSetID].EntryID
-- 載入資料集中目標道具模型的Name值。
local name = _DataSetLogic.itemInfoTable[dataSetID].Name
-- 載入資料集中目標道具模型的Value值。
local value = _DataSetLogic.itemInfoTable[dataSetID].Value

-- 進行目標表格有效性檢查。

-- /分支1/ 若目標道具池內沒有目標池 (表格)
if isValid(self.itemPool[name]) == false or self.itemPool[name] == nil then
-- 建立新的臨時表格。
tempTable = {}
-- 將道具池中的內部池 (表格) 設為臨時表格。
self.itemPool[name] = tempTable

-- /分支2/ 若目標道具池內已存在目標池
else
-- 將臨時表格 (池) 設為既有使用中的表格。
```

```

        tempTable = self.itemPool[name]
    end

    -- 執行參數2 createCount次迴圈。
    for i = 1, createCount, 1 do
        -- 使用SpawnService建立物件。
        local entity = _SpawnService:SpawnByModelId(entryID,name,Vector3.up *
100, self.ItemPoolParent)

        -- 進行建立物件的設定作業。

        -- 設定建立物件的道具固有數值。
        entity.Item.value = value
        -- 設定建立物件的道具name變數。
        entity.Item.name = name

        -- 將建立的實體插入應存放的表格(池)。
        table.insert(tempTable,entity)

        -- 停用建立的實體物件。
        entity.Enable = false
    end
end

@ExecSpace("ClientOnly")
method void InitItemPool()
    -- 初始化道具池的方法。

    -- 從DataSetLogic載入存放道具資料集的表格。
    local itemDataTable = _DataSetLogic.itemInfoTable

    -- 依照Config設定的生成數量生成Coin道具。
    self:CreateItem("Coin",_Config.itemPoolInitCount)
    -- 依照Config設定的生成數量生成Health道具。
    self:CreateItem("Health",_Config.itemPoolInitCount)
    -- 依照Config設定的生成數量生成Booster道具。
    self:CreateItem("Booster",_Config.itemPoolInitCount)
end

@ExecSpace("ClientOnly")
method void Init()
    -- 執行ObjectPool Logic初始化的方法。

    -- 進行道具池初始化。
    self:InitItemPool()
end

@ExecSpace("ClientOnly")
method number GetTileNumber(Entity entity)
    -- 取出圖塊編號的方法。

    -- 參數1 Entity entity: 傳入目標實體。

    -- 取出實體名稱。

```

```
local tileSetName = entity.Name
-- 取出名稱最後字串。( 'Model_TileSet02' 字串中取出第14、15個字元 > 02)
local numberStr = string.sub(tileSetName,14,15)
-- 將字串切換為數值後回收。
return tonumber(numberStr)
end

end
```

- ObjectPoolLogic會管理週期性生成與銷 的地板圖塊集、各種道具元素。
- 已撰寫的GetTile方法會執行從圖塊池取出待機中的地板圖塊功能， 在ScrollingController元件需要生成新地圖碎片時被呼叫。
- ReturnTile方法會在超出 面的地板圖塊與DeadZone 撞時，由TileDeadZone 本使用，將該圖塊回收(停用)至池中，而非銷 。

最低製作標準

- 當地圖捲動進行時，圖塊集實體不會反覆生成與銷 (Destroy)，而是透過物件池系統正常停用與重新 用 循環使用。
- 所有圖塊集皆以從物件池生成或回收可使用實體的類型製作。

應用與進階應用

- 修改程式碼，讓其符合物件池預先生成後使用的原則。
- 依據修改後的程式碼，確認不只是地板圖塊，遊 中頻繁出現的障 物或道具元素，也能透過物件池正常重複使用。
- 非一開始無條件大量生成，而是在物件池內待機實體不足時，才動態擴大池大小(追加生成)，製作更佳記憶體最佳化的擴充邏輯。

[💡 基礎學習]製作玩家角色

影片時間軸

- 可在課程影片中以下時間點學習。
- 製作玩家角色：00:57:50

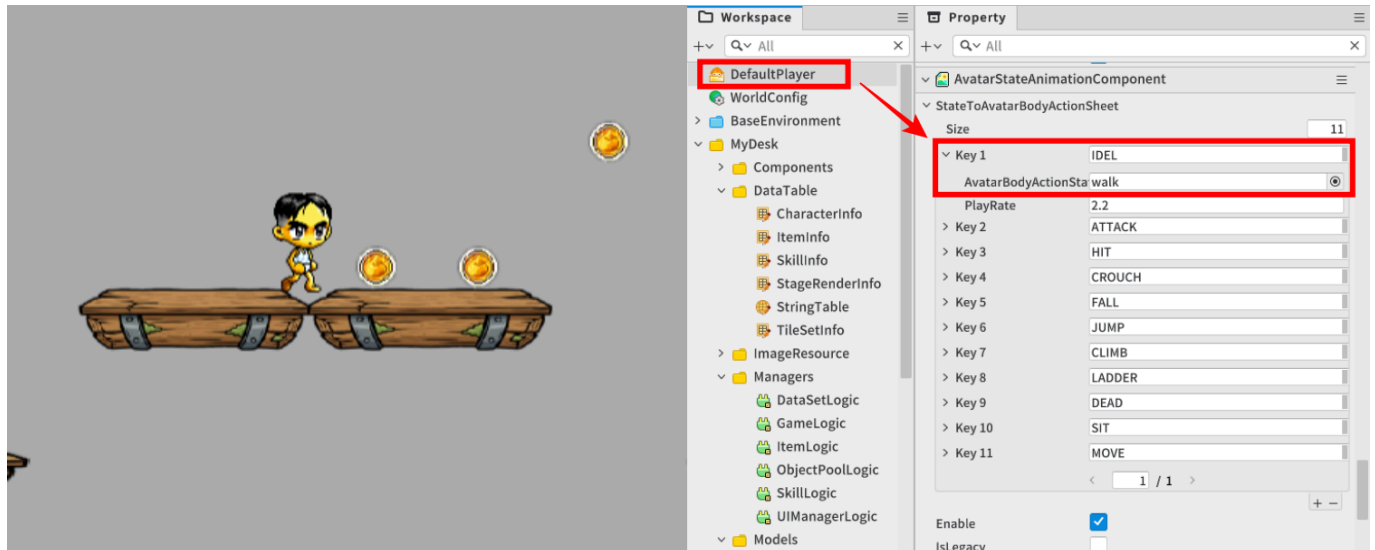
學習目標

- 可製作偵測鍵盤輸入 對角色施加物理力量的跳躍功能。
- 學習如何生成多種角色。
- 定義各角色可使用的技能， 學習使用技能時製作對應效果的方法。
- 可 玩家提供可選擇多種角色的環境。

觀看該影片後可嘗試執行的內容

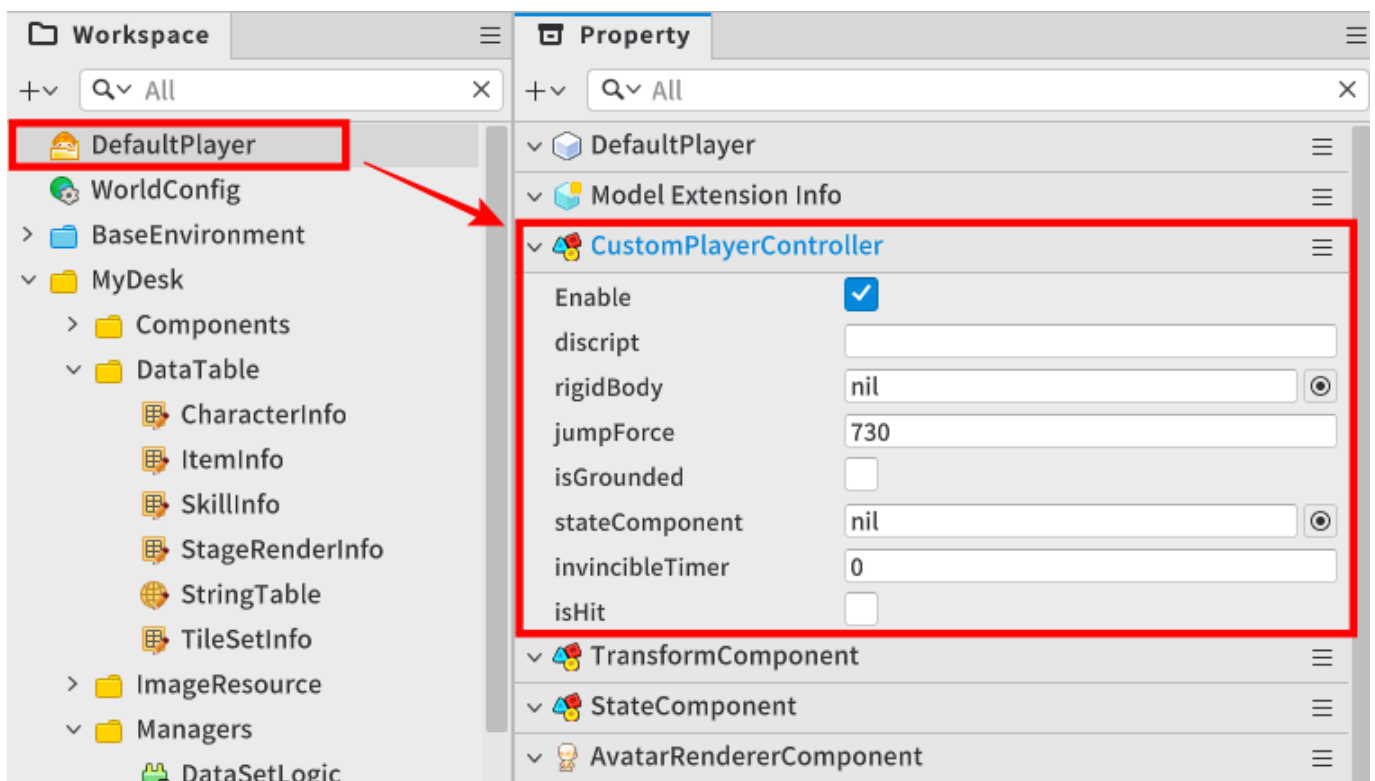
- 可在範例世界中設定除技能鍵外的其他輸入方式來發動技能。
- 可調整角色跳躍力量，變更想要的物理演出。
- 在CharacterInfo DataSet新增角色， 確認是否能以新角色在角色選擇視 中遊玩。

設定角色基本狀態



- 範例世界中的角色實際上不會移動，因此遊 進行時預設應播放奔 動 ，而非站立狀態。
- 將作 角色模型的DefaultPlayer元件中，AvatarStateAnimationComponent原本在站立狀態(IDLE)播放的動 ，改 行走(walk)動 。

控制角色功能



- 了直接以程式碼控制作 角色模型的DefaultPlayer各種功能，製作 附加CustomPlayerController元件 本。

撰寫CustomPlayer 本

- 必須如下撰寫程式碼。
- 該程式碼是在Plain Text環境下撰寫。

```
@Component
script CustomPlayerController extends Component

property string discript = "" -- 這是用來定義角色功能與狀態的元件。

property PhysicsRigidbodyComponent rigidBody = nil

property number jumpForce = 730 -- 角色執行跳躍時所需的力量值。

property boolean isGrounded = false -- 確認角色目前是否在地面上的變數。

property StateComponent stateComponent = nil

property number invincibleTimer = 0 -- 用來控制障礙物碰撞時無敵時間的計時器變數。

property boolean isHit = false -- 確認角色目前是否為障礙物受擊狀態的變數。

@ExecSpace("ClientOnly")
method void Init()
-- 執行CustomPlayerController元件初始化的方法。

-- 執行內部變數參照連接。
self:ConnectRefs()

-- 預設狀態設為MOVE狀態。
self:ChangeState("MOVE")

end

@ExecSpace("ClientOnly")
method void ConnectRefs()
-- 連接內部所需成員參照的方法。

-- 連接附加在角色上的Rigidbody元件。
self.rigidBody = self.Entity.PhysicsRigidbodyComponent
-- 連接附加在角色上的State元件。
self.stateComponent = self.Entity.StateComponent
end

@ExecSpace("ClientOnly")
method void Jump()
-- 執行角色跳躍動作的方法。

-- 若未偵測到地面
if self.isGrounded == false then
-- 中止跳躍執行。
return
```

```
end

-- 將角色y軸目前套用中的速度值設為0，以移除y軸運動。
self.rigidBody:SetLinearVelocityY(0)

-- 透過rigidbody在y軸方向施加Force值。
self.rigidBody:ApplyForce(Vector2.up * self.jumpForce)

-- 執行跳躍邏輯時若不是趴下狀態
if self.stateComponent.CurrentStateName ~= "CROUCH" then
    -- 切換為跳躍狀態。
    self:ChangeState("JUMP")
end

_SoundService:PlaySound("24ee4f2425924a1abd4f655e8faf2791",1)

-- 因執行跳躍，因此設定角色目前不是貼在地面的狀態。
self.isGrounded = false
end

@ExecSpace("ClientOnly")
method void ChangeState(string state)
    -- 將角色狀態變更為傳入狀態的方法。(用來控制角色動畫。)

    -- 參數1 string state: 傳入欲變更的字串狀態。

    -- 以成員宣告的state元件設定狀態。
    self.stateComponent:ChangeState(state)

    -- 調整觸發碰撞箱大小：用來區分角色趴下與站立狀態的碰撞箱狀態。

    -- 若傳入的狀態為CROUCH值
    if state == "CROUCH" then
        -- 將自身(角色)實體所包含的Trigger元件之觸發Box大小設為Config中設定的尺寸。
        self.Entity.TriggerComponent.BoxSize = _Config.crouchTriggerSize
        -- 設定Trigger元件的觸發Box距離Pivot(基準點)應偏移多少的offset值。
        self.Entity.TriggerComponent.BoxOffset = _Config.crouchTriggerOffset

    -- 若傳入的狀態為MOVE值
    elseif state == "MOVE" then
        -- 將自身(角色)實體所包含的Trigger元件之觸發Box大小設為Config中設定的尺寸。
        self.Entity.TriggerComponent.BoxSize = _Config.standTriggerSize
        -- 設定Trigger元件的觸發Box距離Pivot(基準點)應偏移多少的offset值。
        self.Entity.TriggerComponent.BoxOffset = _Config.standTriggerOffset
    end
end

@ExecSpace("ClientOnly")
method void ContactGround()
    -- 偵測到地面時執行的方法。

    -- /分支1/ 若目前已經接觸地面中
    if self.isGrounded == true then
        -- 不執行任何內容並結束方法。
    end
end
```

```
        return
    end

    -- /分支2/ 若原本未接觸地面狀態下接觸到地面

    -- 因目前為地面狀態，變更狀態。
    self.isGrounded = true

    -- 偵測到地面時播放移動動畫。
    self:ChangeState("MOVE")
end

@ExecSpace("ClientOnly")
method void ContactItem(Item item)
    -- 執行各道具應進行的動作。

    -- 參數1 Item item：傳入道具物件內包含的Item元件。

    -- 取得道具時播放效果音
    _SoundService:PlaySound("ef4235b158414f5283c068bad71f403a",1)

    -- 若傳入道具元件的name變數為Coin
    if item.name == "Coin" then
        -- 執行GameLogic的加分方法，並傳入該道具的Value值。
        _GameLogic:AddScore(item.value)
    end

    -- 若傳入道具元件的name變數為Health
    if item.name == "Health" then
        -- 執行遊戲邏輯的增加體力方法，並傳入該道具的Value值。
        _GameLogic:AddHP(item.value)
    end

    -- 若傳入道具元件的name變數為Booster
    if item.name == "Booster" then
        -- 呼叫道具邏輯中的使用加速器道具方法。
        _ItemLogic:UseBooster()
    end

    -- 取得道具時移動到遠處位置。
    -- （因從物件池重新生成時會有問題，所以暫時將道具物件移至看不見的位置。）
    item.Entity.TransformComponent.Position = Vector3.one * 100

    -- 執行將道具物件回收至物件池的方法。
    _ObjectPoolLogic:ReturnItem(item.Entity)
end

@ExecSpace("ClientOnly")
method void ContactObstacle()
    -- 接觸障礙物時執行的方法。

    -- 若無敵持續時間尚未結束
    if self.invincibleTimer > 0 then
        -- 不做任何事並結束。
    end
end
```

```
        return
    end

    -- 將Config定義的障礙物接觸傷害值傳給遊戲邏輯的增加體力方法。
    _GameLogic:AddHP(_Config.obstacleDamage)

    -- 設定為受擊中的狀態。
    self.isHit = true

    -- 將無敵持續時間計時器設為Config中設定的初始值。
    self.invincibleTimer = _Config.invincibleTime

    -- 播放受擊特效。
    self:PlayOnDamagedEffect()

    -- 設定紙娃娃渲染的透明度。(用來表現受擊狀態)
    self.Entity.AvatarRendererComponent:SetAlpha(0.5)
end

@ExecSpace("ClientOnly")
method void PlayOnDamagedEffect()
    -- 播放受擊時出現特效方法。

    -- 儲存受擊特效資源ID值。
    local effectRUID = "41e13add2ba746d9bbe624a31205a848"
    -- 取得地圖實體。
    local mapEntity = _EntityService:GetEntityByPath("/maps/MainGame")
    -- 設定特效出現位置。
    local position = self.Entity.TransformComponent.Position

    -- 使用特效服務播放特效。
    _EffectService:PlayEffect(effectRUID,mapEntity,position,0,Vector3.one * 3)
end

@ExecSpace("ClientOnly")
method void OnUpdate(number delta)
    -- 這是每一幀都會被呼叫的方法。

    -- 若為受擊中狀態
    if self.isHit == true then
        -- 若受擊計時器尚有剩餘持續時間
        if self.invincibleTimer > 0 then
            -- 減少受擊計時器的持續時間。
            self.invincibleTimer -= delta
        -- 若受擊計時器已無剩餘持續時間
        else
            -- 變更為解除受擊狀態。
            self.isHit = false
            -- 將紙娃娃渲染透明度值初始化為1。
            self.Entity.AvatarRendererComponent:SetAlpha(1)
        end
    end
end
end
```

```
@ExecSpace("ClientOnly")
method void ResetCharacter()
-- 遊戲重新開始時設定所需初始值的方法。

-- 將該實體位置絕對座標設為x = 0, y = 1。
self.Entity.TransformComponent.Position = Vector3(0,1,0)
-- 為了動態設定Rigidbody的BodyType, 設為Dynamic。
self.rigidBody.BodyType = BodyType.Dynamic

-- 讓角色顯示。
self.Entity.Visible = true
end

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- 這是在世界執行時呼叫的方法。

-- 執行該元件初始化。
self:Init()
end

@EventSender("Service", "InputService")
handler HandleKeyDownEvent(KeyDownEvent event)
-- 當使用者按下按鍵時呼叫的方法。

-- 若遊戲已結束狀態
if _GameLogic.isEnd == true then
-- 中止執行該方法。
    return
end

-- 儲存輸入的按鍵資訊。
local key = event.key

-- 按下空白鍵時
if key == KeyboardKey.Space then
    -- 執行跳躍動作。
    self:Jump()
end

-- 按下方向鍵下鍵時
if key == KeyboardKey.DownArrow then
    -- 將角色狀態設為 'CROUCH'。
    self:ChangeState("CROUCH")
end

-- 按下 'K' 鍵時
if key == KeyboardKey.K then
    -- 使用角色技能。
    _SkillLogic.UseSkill()
end
```

end

```
@EventSender("Service", "InputService")
handler HandleKeyUpEvent(KeyUpEvent event)
```

-- 當使用者結束按鍵輸入時呼叫的方法。

-- 若遊戲已結束狀態

```
if _GameLogic.isEnd == true then
```

-- 中止執行該方法。

```
    return
```

end

-- 儲存輸入的按鍵資訊。

```
local key = event.key
```

-- 放開方向鍵下鍵時

```
if key == KeyboardKey.DownArrow then
```

-- 再次解除Crouch狀態。

```
    self:ChangeState("MOVE")
```

end

end

```
@EventSender("Self")
```

```
handler HandleTriggerEnterEvent(TriggerEnterEvent event)
```

-- 當角色的觸發箱內與其他觸發器碰撞時呼叫。

-- 若遊戲已結束狀態

```
if _GameLogic.isEnd == true then
```

-- 中止執行該方法。

```
    return
```

end

-- 儲存觸發碰撞的目標實體。

```
local TriggerBodyEntity = event.TriggerBodyEntity
```

-- 儲存碰撞觸發器的群組名稱。

```
local TriggerGroup =
```

```
tostring(TriggerBodyEntity.TriggerComponent.CollisionGroup.Id)
```

-- 若目標觸發器為道具物件的觸發器

```
if _Config.triggerGroup[TriggerGroup] == "Item" then
```

-- 執行偵測到道具時的邏輯。

```
    self:ContactItem(TriggerBodyEntity.Item)
```

end

-- 若目標觸發器為障礙物元素物件的觸發器

```
if _Config.triggerGroup[TriggerGroup] == "Obstacle" then
```

-- 執行偵測到障礙物時的邏輯。

```
    self:ContactObstacle()
```

end

-- 若目標觸發器為DeadZone要素物件的觸發器

```
if _Config.triggerGroup[TriggerGroup] == "DeadZone" then
```

```

-- 目前角色與DeadZone相遇的條件為角色掉落至畫面下方時
-- 因此需要執行遊戲結束命令。

-- 將Rigidbody的BodyType設為Static，使物理運動不再進行。
self.rigidbody.BodyType = BodyType.Static
-- 執行遊戲邏輯的遊戲結束方法。
_GameLogic:EndMainGame()

end
end

@EventSender("Self")
handler HandlePhysicsContactBeginEvent(PhysicsContactBeginEvent event)
-- 偵測到Physics剛體物理碰撞時發生的事件。

-- 取得碰撞剛體的實體。
local ContactedBodyEntity = event.ContactedBodyEntity

-- 取得碰撞剛體父物件的圖塊集元件。
local tileSet = ContactedBodyEntity.Parent.TileSet

-- 若從接觸物件中偵測到圖塊集
if tileSet ~= nil then

    -- 取得接觸物件的Transform元件。
    local contactTransform = ContactedBodyEntity.TransformComponent
    -- 取得角色的Transform元件。
    local playerTransform = self.Entity.TransformComponent

    -- 若碰撞體目標物件的位置y值高於角色位置y值
    if contactTransform.Position.y >= playerTransform.Position.y then
        -- 中止執行該方法。
        return
    end

    -- 傳遞目前為站在地面的狀態。
    self.isGrounded = true
    -- 若目前狀態為 'JUMP'
    if self.stateComponent.CurrentStateName == "JUMP" then
        -- 切換為 'MOVE' 狀態。
        self:ChangeState("MOVE")
    end
end
end
end

end

```

理解跳躍功能執行過程

- 在已設定的環境中，可使用的跳躍功能之一，是利用登錄在角色上的PhysicsRigidbody物理功能。
- 在CustomPlayerController元件內，`self.rigidbody` 控制了角色的物理功能，會宣告PhysicsRigidbodyComponent型態的變數，`self.rigidbody` 連接元件參照。

- 若要對實體施加往上方向的力量，會使用PhysicsRigidbody的ApplyForce方法。
- 了讓人知道角色處於跳躍狀態，會播放跳躍動 。

角色跳躍功能製作範例

```
PhysicsRigidbodyComponent rigidBody;
StateComponent state;

number jumpForce = 730
boolean isGrounded = false

void OnBeginPlay()
{
    -- 連接要施加物理力量的PhysicsRigidbodyComponent
    self.rigidBody = self.Entity.PhysicsRigidbodyComponent
    -- 連接用來切換角色狀態的StateComponent
    self.state = self.Entity.StateComponent
}

void Jump()
{
    if self.isGround == false then
        return -- 若角色不在地面上，則不執行跳躍功能並結束。
    end

    -- 當開始執行跳躍時，消除角色目前受到的Y軸速度、力量。
    self.rigidBody:SetLinearVelocityY(0)

    -- 往Vector2.up角色的上方方向施加jumpForce大小的力量。
    self.rigidBody:ApplyForce(Vector2.up * self.jumpForce)

    -- 為了套用跳躍動畫，向StateComponent請求切換為Jump狀態。
    self.state:ChangeState("Jump")

    -- 因為要執行跳躍，所以切換狀態為角色目前不在地面上。
    self.isGround = false;
}

HandleKeyDownEvent(KeyDownEvent event)
{
    -- 這是在鍵盤按鍵被按下時呼叫的方法。

    -- 儲存輸入的按鍵資訊。
    local key = event.key

    -- 如果輸入的按鍵是Space鍵
    if key == KeyboardKey.Space then
        -- 則執行Jump方法。
        self:Jump()
    end
}
```

```
}
```

偵測到地面時切換角色狀態的方式

- 了防止角色連續跳躍，使用了boolean isGround狀態變數。
- 當角色開始跳躍時，會將該變數切換 false。
- 當跳躍完成，因重力落地到地面時，必須再將isGrounded狀態變數切換 true，恢復 可跳躍狀態。
- 可透過PhysicsCollider與EventHandler進行狀態切換。

地面碰撞狀態切換範例程式碼

```
StateComponent state
```

```
void OnBeginPlay()
```

```
{
    -- 連接用來切換角色狀態的StateComponent
    state = self.Entity.StateComponent
}
```

```
HandlePhysicsContactBeginEvent(PhysicsContactBeginEvent event)
```

```
{
    -- 這是在透過PhysicsCollider偵測到與其他碰撞體發生碰撞時呼叫的方法。

    -- 儲存碰撞體對象的實體。
    local ContactedBodyEntity = event.ContactedBodyEntity
    -- 從碰撞體對象取得圖塊集元件。
    local tileSet = ContactedBodyEntity.TileSet

    -- 若確認到圖塊集元件 (若確認為地板圖塊)
    if tileSet ~= nil then
        -- 則將isGround狀態切換為true。
        isGround = true
        -- 將原本為跳躍狀態的動畫重新切換為步行動畫。
        self.state.ChangeState("Move")
    end
}
```

製作多種角色

建立各角色資訊表

	1	2	3	4	+
ID	Name	Speed	SkillId	RUID	
1	TestA	3	1	ac753a90b9814	
2	TestB	4	2	b65a754c25174	
+					

DefaultPlayer
WorldConfig
> BaseEnvironment
v MyDesk
> Components
v DataTable
CharacterInfo
ItemInfo
SkillInfo
StageRenderInfo
StringTable
TileSetInfo
> ImageResource
v Managers
DataSetLogic

對象	說明
Name	這是提供給使用者的角色名稱。
Speed	這是為了讓各角色的捲動畫面速度不同而新增的值。
Skill id	這是各角色的專屬技能編號。
RUID	這是將登錄到角色CostumeManagerComponent的Body部位上的資源ID值。

- 製作好的CharacterInfo DataSet，會透過DataSetLogic中的InitCharacterInfoTable方法，將各角色資訊儲存成表格。

登錄技能資料

	1	2	3	4	+
ID	Name	Value	EffectRUID	UI_RUID	
1	Heal	20	db4e6e7fcb1b4	c99e612dbd54	
2	Bonus	500	db4e6e7fcb1b4	eb0b26a38476	
+					

WorldConfig
> BaseEnvironment
v MyDesk
v Components
> Character
> Item
> TileSet
> UI
v DataTable
CharacterInfo
ItemInfo
SkillInfo
StageRenderInfo
StringTable
TileSetInfo

對象	說明
Name	這是提供給使用者的技能名稱。

對象	說明
Value	這是各技能使用時依效果而定的固有數值。
EffectRUID	這是各技能使用時的特效資源ID值。
UI_RUID	這是各技能將顯示在UI上的Sprite資源ID值。

- 在範例世界中，當玩家輸入特定技能鍵時，在遊 進行時可發動的功能有額外獲得分數、恢復體力。
- 製作好的CharacterInfo DataSet，會透過DataSetLogic中的InitSkillInfoTable方法，將各技能資訊儲存成表格。

各角色技能連接方式

CharacterInfo

	1	2	3	4
ID	Name	Speed	SkillId	RUID
1	TestA	3	1	ac753a90b9814
2	TestB	4	2	b65a754c25174

SkillInfo

	1	2	3	4
ID	Name	Value	EffectRUID	UI_RUID
1	Heal	20	db4e6e7fcb1b4	c99e612dbd544
2	Bonus	500	db4e6e7fcb1b4	eb0b26a384764

- 登錄於CharacterInfo DataSet中的各角色SkillId，代表SkillInfo DataSet中的各ID編號。
 - 例如，若TestB角色的SkillId是2，則SkillInfo資料集內索引2對應的Bonus技能，就會被分配 該角色的技能。

Skill Logic的UseSkill運作原理

- 在CustomPlayerController的HandleKeyDownEvent方法中，當輸入技能發動鍵(K key)時，會呼叫SkillLogic的UseSkill方法。
- CustomPlayerController只負責接收按鍵輸入 向SkillLogic傳遞技能發動命令，不會處理具體的技能製作邏輯，以明確區分角色。
- Skill Logic會直接發動與自身登錄變數skillId編號對應的技能。
- skillId編號會在標題 面選擇角色時變更。

Skill Logic的UseSkill原理

```
number skillId;

void UseSkill()
{
    local skillName = _DataSetLogic.skillInfoTable[self.skillId].Name

    if skillName == "Heal" then
        -- 使用體力恢復技能時，定義具體動作過程
    elseif skillName == "Bonus" then
        -- 使用Bonus技能時，定義具體動作過程
    end
}
```

```
}
```

撰寫SkillLogic 本

- 必須如下撰寫程式碼。
- 該程式碼是在Plain Text環境下撰寫。

```
@Logic
script SkillLogic extends Logic

property number discript = 0 -- 這是為了在使用角色專屬技能時執行具體動作而宣告的邏輯。

property number skillId = 0 -- 這是用來設定角色專屬SkillId值的變數。

property boolean useable = true -- 這是用來儲存技能可使用狀態的狀態變數。

@ExecSpace("ClientOnly")
method void UseSkill()
-- 這是執行目前已登錄(SkillId)之具體動作的方法。

-- 若呼叫該方法時不是可使用的狀態
if self.useable == false then
    -- 就結束該方法。
    return
end

-- 則儲存目前角色技能名稱。
local skillName = _DataSetLogic.skillInfoTable[self.skillId].Name
-- 儲存目前角色技能固有數值。
local skillValue = _DataSetLogic.skillInfoTable[self.skillId].Value

-- 若技能名稱是 'GetLife'
if skillName == "Heal" then
-- 則依技能固有數值恢復體力。
    _GameLogic:AddHP(skillValue)

-- 若技能名稱是 'GetScore'
elseif skillName == "Bonus" then
-- 則依技能固有數值獲得額外分數。
    _GameLogic:AddScore(skillValue)
end

-- 播放技能特效。
self:PlaySkillEffect()

-- 播放技能效果音
_SoundService:PlaySound("56c1b5e3c7fc4abda561c1cc00813bda",1)
```

-- 因為已使用技能，所以切換為不可使用狀態。

```
self.useable = false
```

-- 將技能圖示UI渲染器的透明度值設低。

```
_UIManagerLogic.skillRenderer.Color.a = 0.3
end
```

```
@ExecSpace("ClientOnly")
```

```
method void Reset()
```

-- 這是初始化內部狀態的方法。

-- 切換為可使用技能。

```
self.useable = true
end
```

```
@ExecSpace("ClientOnly")
```

```
method void PlaySkillEffect()
```

-- 這是播放技能特效的方法。

-- 從SkillInfoTable取得儲存的技能特效RUID值。

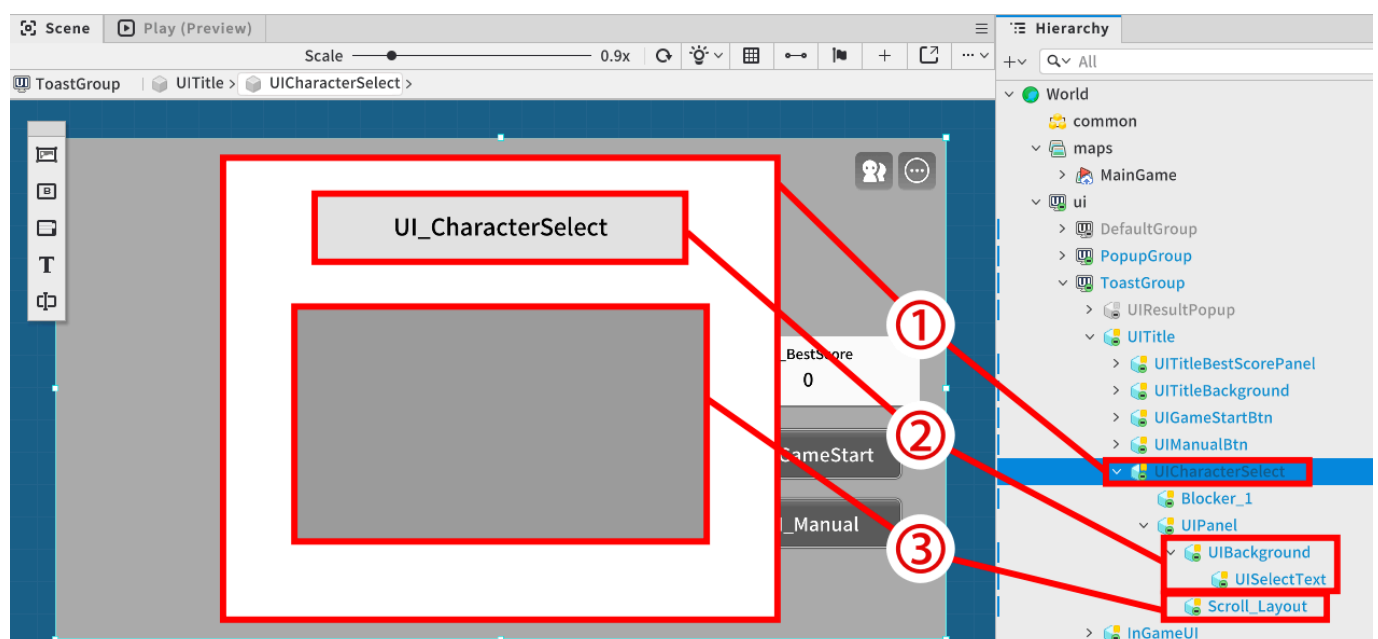
```
local effect = _DataSetLogic.skillInfoTable[self.skillId].EffectRUID
```

-- 利用EffectService附加到角色物件上並播放技能特效。

```
_EffectService.PlayEffectAttached(effect,_UserService.LocalPlayer,Vector3.zero,0,Vector3.one)
end
```

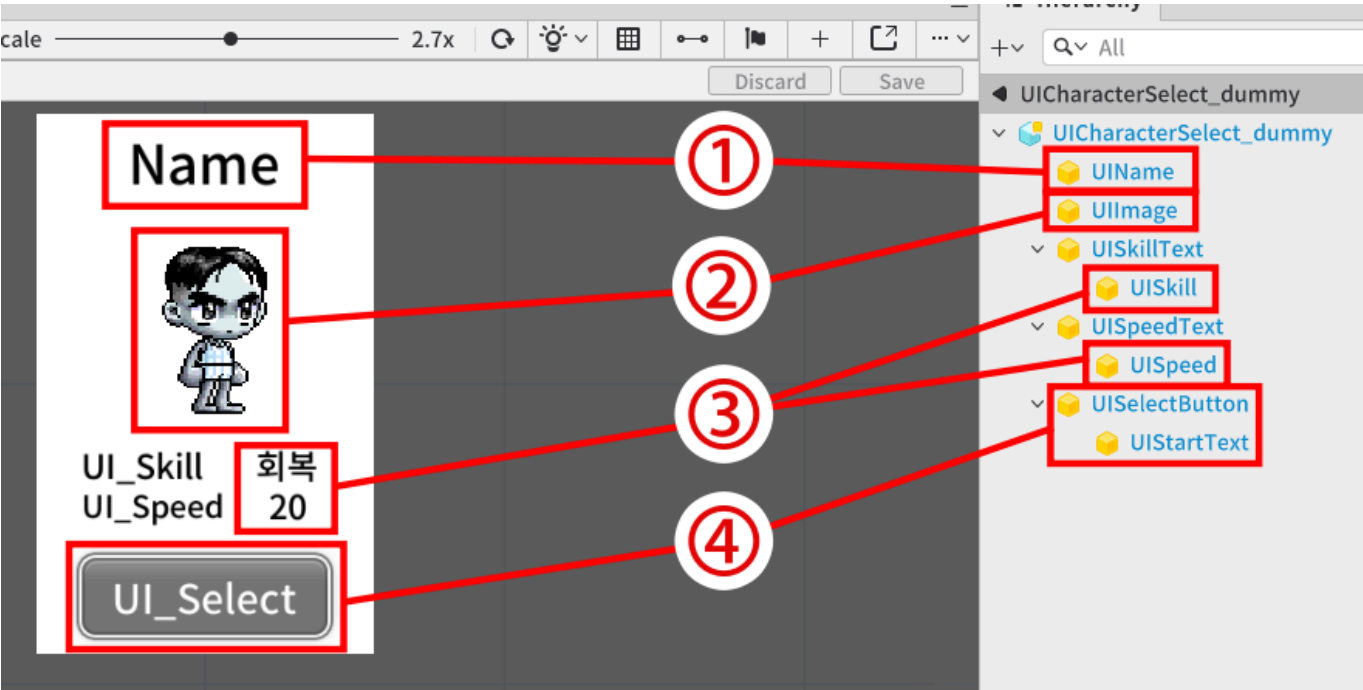
```
end
```

製作角色選擇



項目	實體	說明
1	UICharacterSelect	作為角色選擇視窗最上層父層的實體，為了顯示或隱藏角色選擇視窗，會控制該實體的Enable值。
2	UIBackground UISelectText	這是顯示該彈窗為角色選擇視窗的標題文字區域。
3	Scroll_Layout	這是顯示多種角色視窗的區域，用來排列'UICharacterSelect'槽位的空間。若道具元素超出該區域範圍，可透過滑鼠拖曳來左右控制檢視區域。

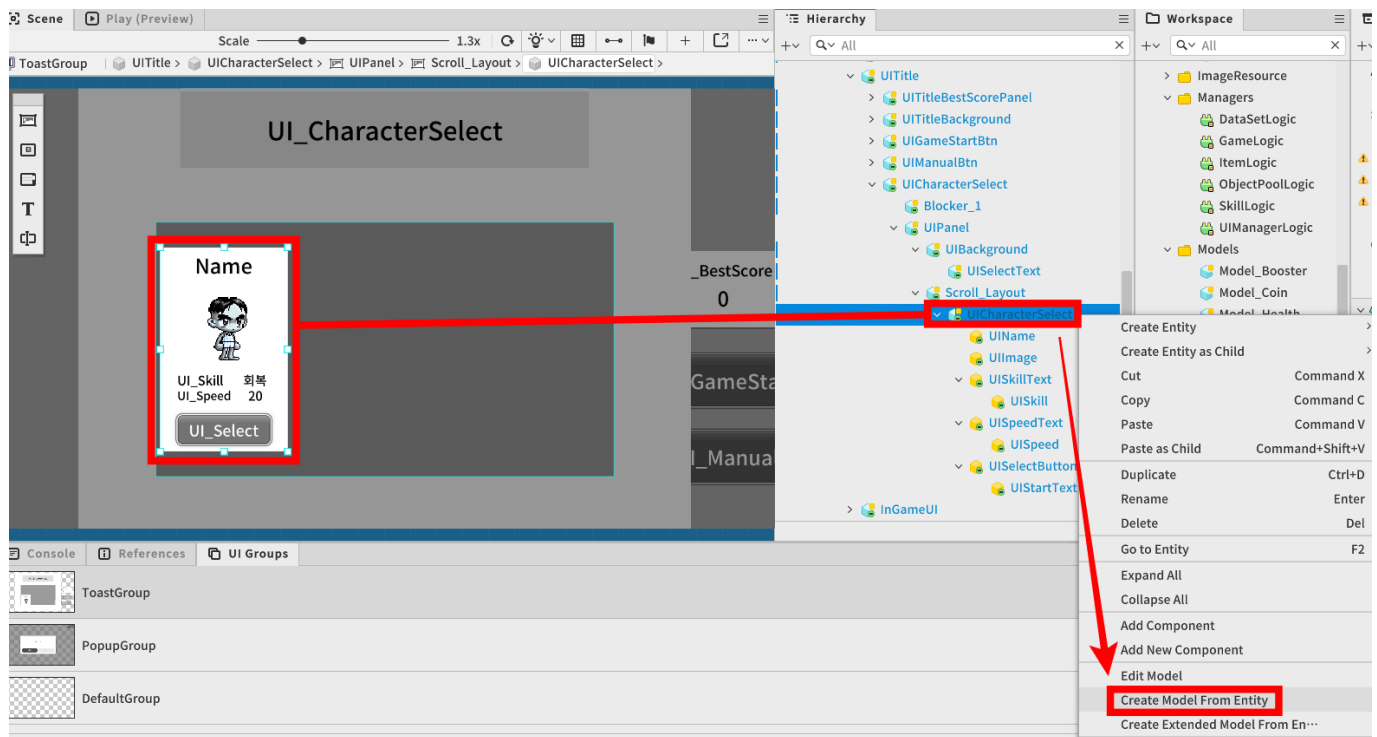
製作顯示角色資訊的UI Slot



項目	實體	說明
1	UIName	在Slot上方顯示角色名稱。
2	UIImage	提供讓玩家可以視覺確認角色外型的功能。
3	UISkill UISpeed	顯示各角色可使用的技能與各角色的捲動畫面速度。
4	UISelectButton	若想以登錄在Slot中的角色進行遊玩，則提供可用該角色正式開始遊戲的按鈕。

- 所謂UI Slot，是指在單一實體中，承載個別角色專屬資訊(頭像、名稱、能力 等) 顯示在 面的獨立UI單位。
- 當以 單類型排列多個角色時，會將這個UI Item作 「基本 架(模組)」，只替換資料 反覆重複使用。

製作Slot道具模型



- 在製作單一角色槽位時，建議採用直接在槽位內配置UI，一邊直觀確認實際UI配置或大小一邊製作的方式。
- 若已完成製作一個單位的Slot，則在該實體上按右鍵後選擇Create Model From Entity，將其登錄 Workspace中可重複使用的單一模型。

在角色Slot中登錄各角色資訊

- 在執行世界時，會預先生成所有角色的Slot道具。
- 各槽位必須輸出CharaterInfo DataSet的所有要素與角色的全部資料。
- 將角色資訊登錄到Slot的過程如下。
 - 世界執行 -> 設定 在自動呼叫的OnBeginPlay內呼叫RegisterCharacter方法。
 - RegisterCharacter方法的功能如下。

UICharacterSelectPopup的RegisterCharacter方法

Entity scrollViewEntity; -- Slot道具應配置的父實體

```
void RegisterCharacter()
{
```

-- 儲存登錄在Workspace中的Slot道具模型ID。

```
local modelId = _EntryService:GetModelIdByName("UICharacterSelect")
```

-- 為了讓呼叫角色資訊表的程式碼撰寫得更簡潔，將其儲存為區域變數。

```
local characterTable = _DataSetLogic.characterInfoTable
```

-- 依角色資訊表中登錄的數量進行巡覽。(CharacterInfo DataSet中登錄的角色數量)

```
for i = 1, #characterTable, 1 do
```

-- 使用SpawnService的SpawnByModelID方法生成一個Slot道具。

```
local characterInfo = _SpawnService:SpawnByModelID(
```

```
modelId,"characterInfoPopup",Vector3.zero,scrollViewEntity).UICharacterInfo
Component
```

```

-- 將顯示角色名稱的UI文字設定為角色名稱。
characterInfo.nameText.Text = characterTable[i].Name

-- 設定預先登錄在Slot中的角色外型(CustomBodyEquip)資源ID值。
characterInfo.costumImage.CustomBodyEquip = characterTable[i].RUID

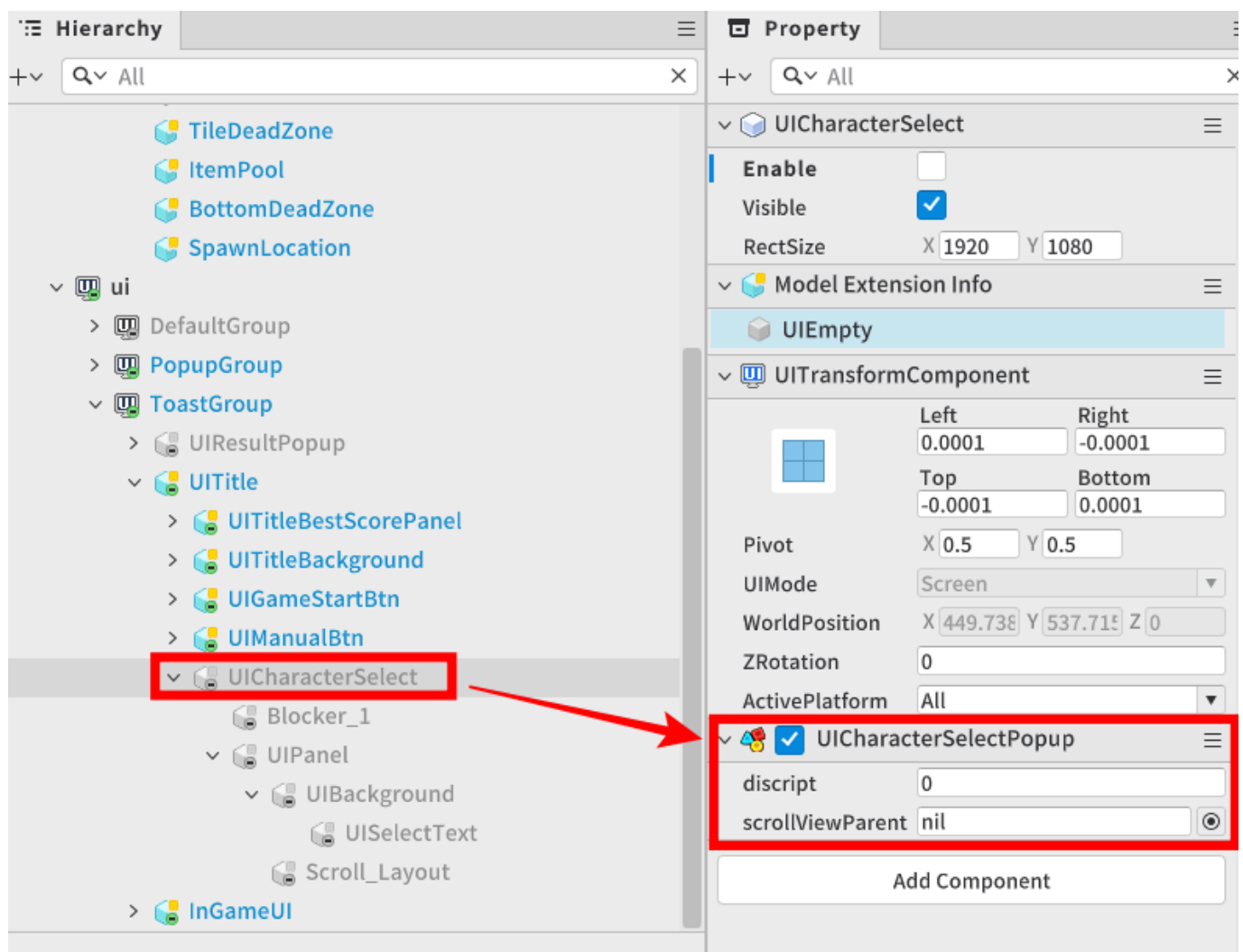
-- 取出儲存在CharacterInfo DataSet中的SkillId編號。
local skillId = tonumber(characterTable[i].SkillId)
-- 載入儲存在SkillInfo DataSet中的技能資訊，並登錄該技能的名稱。
characterInfo.skillText.Text =
_DataSetLogic.skillInfoTable[skillId].Name

-- 將顯示角色捲動畫面速度的UI文字設定為角色專屬捲動畫面速度。
characterInfo.speedText.Text = characterTable[i].Speed

end
}

```

製作角色選擇視



- 在角色選擇視 最上層實體上，登錄用來在世界執行時執行角色Slot生成功能的元件。

- 製作完成後，在玩家點擊開始遊 按 前，該彈 不應顯示，因此先將實體停用。
- 必須如下撰寫程式碼。
- 該程式碼是在Plain Text環境下撰寫。

```
@Component
script UICharacterSelectPopup extends Component

property number discript = 0 -- 這是定義顯示角色選擇彈跳視窗時所需功能的元件。

property Entity scrollViewParent = nil

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- 這是在世界開始時呼叫的方法。

-- 連接將作為從模型(Prefab)生成之物件父層設定的實體。
self.scrollViewParent =
_EntityService:GetEntityByPath("/ui/ToastGroup/UITitle/UICharacterSelect/UI
Panel/Scroll_Layout")

-- 進行角色資訊登錄。
self:RegisterCharacter()
end

@ExecSpace("ClientOnly")
method void RegisterCharacter()
-- 在角色選擇視窗內登錄角色資訊的方法。
-- 為了在從模型(Prefab)生成顯示角色資訊的彈窗時，設定內部要填入的資料值。

-- 取得基本模型(Prefab)要素的模型Id。
local modelId = _EntryService:GetModelIdByName("UICharacterSelect")
-- 從資料集取得角色資訊表。
local characterTable = _DataSetLogic.characterInfoTable

-- 依表格內登錄的角色數量執行迴圈。
for i = 1, #characterTable, 1 do

    -- 使用生成服務生成UI彈窗，並儲存角色資訊元件。
    local characterInfo =
_SpawnService:SpawnByModelId(modelId,"characterInfoPopup",Vector3.zero,self
.scrollViewParent).UICharacterInfoComponent

    -- 載入用來識別技能資訊表的skillId值。
    local skillId = tonumber(characterTable[i].SkillId)
    -- 設定要登錄在角色資訊內的角色技能編號。
    characterInfo.skillId = skillId

    -- 設定要登錄在角色資訊內的角色名稱。
    characterInfo.nameText.Text = characterTable[i].Name
    -- 設定要登錄在角色資訊內的服裝身體RUID值。
    characterInfo.costumeImage.CustomBodyEquip = characterTable[i].RUID
    -- 設定角色資訊內的技能名稱(根據儲存的SkillId編號從技能資訊表中取得技能名稱。)
```

```

characterInfo.skillText.Text =
_DataSetLogic.skillInfoTable[skillId].Name
-- 設定角色資訊內的速度文字。
characterInfo.speedText.Text = characterTable[i].Speed

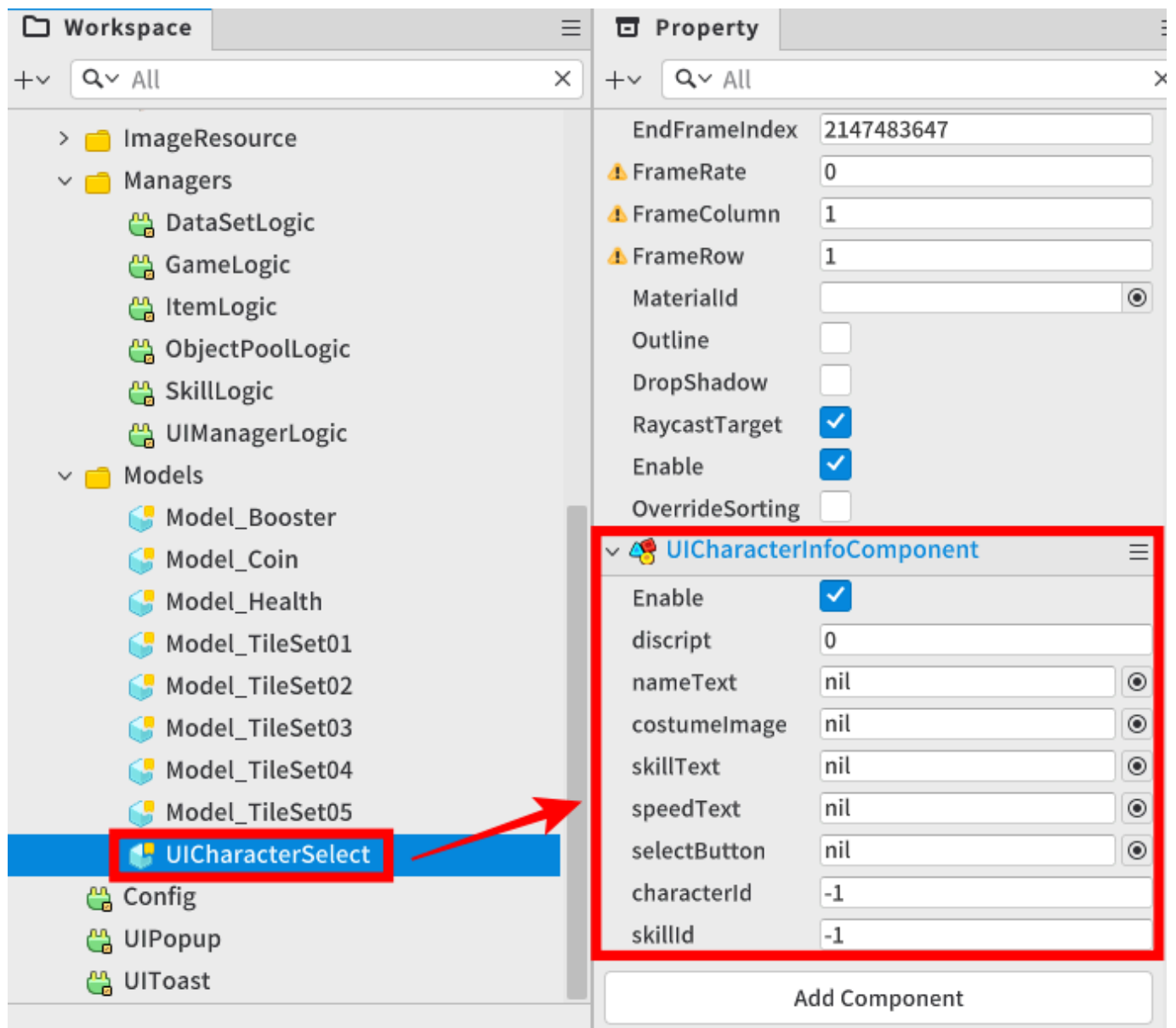
-- 設定角色ID(索引)值。
characterInfo.characterId = i
end

end

end

```

製作角色Slot



- 製作 本，以執行配置在Slot下方的角色選擇按 功能。
- 使按 事件可以呼叫CloseSelectPopup方法。

- CloseSelectPopup方法會執行以下動作。
 - 停用建構遊 開始時可見標題 面的UI元素。
 - 變更玩家所使用角色的外型。
 - 將該角色的技能編號傳遞給SkillLogic。
 - 向GameLogic傳遞目前角色是第幾號角色， 呼叫開始遊 功能。
- 必須如下撰寫程式碼。
- 該程式碼是在Plain Text環境下撰寫。

```
@Component
script UICharacterInfoComponent extends Component

property number discript = 0 -- 這是用來在角色選擇視窗中顯示各角色資訊的元件。

property TextComponent nameText = nil

property CostumeManagerComponent costumeImage = nil

property TextComponent skillText = nil

property TextComponent speedText = nil

property ButtonComponent selectButton = nil

property number characterId = -1 -- 這是用來區分角色索引的變數。

property number skillId = -1 -- 這是角色專屬技能編號。

@ExecSpace("ClientOnly")
method void OnBeginPlay()
-- 這是在世界開始時呼叫的方法。

-- 進行該元件初始化。
self:Init()
end

@ExecSpace("ClientOnly")
method void Init()
-- 執行CharacterInfo元件初始化的方法。

-- 執行模型(Prefab)子物件元件連接。

-- 從子物件取得用來填入角色名稱的文字元件。
self.nameText = self.Entity:GetChildByName("UIName").TextComponent
-- 從子物件取得用來登錄角色圖片的服裝元件。
self.costumeImage =
self.Entity:GetChildByName("UIImage").CostumeManagerComponent
-- 從子物件取得用來填入技能名稱的文字元件。
self.skillText = self.Entity:GetChildByName("UISkill",true).TextComponent
-- 從子物件取得用來填入速度值的文字元件。
self.speedText = self.Entity:GetChildByName("UISpeed",true).TextComponent
-- 從子物件取得用來偵測滑鼠點擊的按鈕元件。
self.selectButton =
```

```
self.Entity.GetChildByName("UISelectButton").ButtonComponent

-- 登錄在點擊角色選擇按鈕時將執行的方法(CloseSelectPopup)。
self.selectButton.Entity.ConnectEvent(ButtonClickEvent, self.CloseSelectPopu
p)

end

@ExecSpace("ClientOnly")
method void SetInfo(string name, string ruid, string skill, string speed)
-- 用來設定角色資訊視窗所需資訊、內容的方法。

-- 參數1 string name: 傳入角色名稱。
-- 參數2 sting ruid: 傳入角色身體RUID值。
-- 參數3 string skill: 傳入角色技能名稱。
-- 參數4 string speed: 傳入角色捲動畫面速度值。

-- 設定角色名稱文字。
self.nameText.Text = name
-- 設定服裝身體部位資源。
self.costumeImage.CustomBodyEquip = ruid
-- 設定技能名稱文字。
self.skillText.Text = skill
-- 設定捲動畫面速度值文字。
self.speedText.Text = speed
end

@ExecSpace("ClientOnly")
method void CloseSelectPopup()
-- 這是停用角色選擇彈跳視窗的方法。

-- 取得遊戲開始前的標題面板(開始按鈕、操作方式、角色選擇視窗的父層)實體。
local TitlePanel = _EntityService:GetEntityByPath("/ui/ToastGroup/UITitle")

-- 停用面板物件。
TitlePanel.Enable = false

-- 設定角色的服裝身體資源值。
_UserService.LocalPlayer.CostumeManagerComponent.CustomBodyEquip =
self.costumeImage.CustomBodyEquip

-- 設定遊戲中要使用的角色之專屬技能Id。
_SkillLogic.skillId = self.skillId

-- 請求遊戲邏輯以所選角色Id編號進行遊戲。
_GameLogic:GameStart(self.characterId)
end

end
```

最低製作標準

- 玩家角色必須能正常偵測與地板圖塊的碰撞，且不能在空中無限跳躍，而是僅限於地面狀態下才能跳躍，此限制功能必須正常運作。
- 在角色選擇視窗中，按下登錄在角色Slot中的按鈕輸入時，必須製作成可以向GameLogic呼叫GameStart。

應用與進階應用

- 可在CharacterInfo DataSet中，除了各角色的捲動速度外，再新增跳躍力、最大體力、無敵時間等細部能力欄位，藉此提供各角色不同的遊玩風格。
- 嘗試企劃新增各種動作技能，例如一邊破壞障礙物一邊奔跑的「無敵突進」，或一次取得周圍所有金幣等。
- 在選擇角色時，不只是替換一個身體服裝，而是可將頭髮、面部、披風等多種部位資料設定變更，進一步美化本。

[💡 基礎學習]製作Life系統

影片時間軸

- 可在課程影片中以下時間點學習。
- 製作Life系統：01:20:58

學習目標

- 理解OnUpdate方法與delta時間的概念，可製作不受裝置幀數影響、會隨時間穩定減少體力的邏輯。
- 學習當角色體力降至0以下時，偵測執行Game Over處理的狀態控制流程。
- 透過在UI實體附加SliderComponent，製作可視化體力條，掌握讓內部資料與面上的UI同步的方法。

觀看該影片後可嘗試執行的內容

- 變更Config Logic 本中設定的decreaseHpValue變數進行測試，設定體力要多快減少。
- 在遊戲中按下播放按鈕進行測試，確認隨著時間經過，UI體力計量條是否平順下降，以及體力變成0時是否正常切換至結果視窗。

製作隨時間減少體力功能



- 體力控制功能不只包含取得道具與 撞障 物時的 時變化，也一 製作了會隨時間經過而週期性減少體力的功能。
- 以程式碼減少體力的功能，是定義在GameLogic的AutoDecreaseHp與AddHP、SetHP方法中，各功能會在GameLogic的OnUpdate方法內執行。
- 若需要GameLogic程式碼內容，可參考「2.理解Logic」單元中附上的GameLogic 本內容。

體力設定功能

- 增加或減少體力的功能，會使用GameLogic的AddHP方法。

GameLogic內的AddHP方法

```
number hp;

void AddHP(number addValue)
{
    -- 將參數傳入的值原樣加到目前體力上或從中扣除。
    -- 呼叫AddHP方法時，可傳入正數或負數，以靈活執行恢復與減少功能。

    self.hp += addValue;

    -- 因體力值發生變動，因此請求更新UIManagerLogic所管理的滑桿資訊。
    _UIManagerLogic.SetHpSliderValue(self.hp)

    -- 當體力降至0以下時(滿足Game Over條件時)
    if self.hp <= 0 then
        -- 執行遊戲結束功能
    end
}
```

製作持續減少體力功能

- 使用每一幀呼叫的OnUpdate方法，就能製作體力緩慢減少的演出。
- 但此時若不乘上幀與幀之間的時間變化量delta，會發生依裝置幀率(FPS)不同而導致體力異常快速蒸發的致命問題。
- 若每秒幀率 60FPS，則幀間時間變化量delta $1\text{Sec}/60\text{FPS} = 0.0167\text{Sec}$ 。
- 若想每秒減少5點體力，則每一幀計算 $5 * \text{delta}$ 減少 可。無論裝置以60幀還是30幀運作，經過1秒後的累積減少量都會精準 5。

GameLogic的OnUpdate方法中體力減少功能部分範例

```
void OnUpdate(number delta)
{
    -- Config：若是設定後不會變更的值，則會登錄在該Logic腳本中並加以管理。
    -- 更新設定好的每秒體力減少值 * delta 的體力量
    self:AddHP(delta * _Config.decreaseHpValue)
}
```

```

void AddHP(number addValue)
{
    -- 將作為參數傳入的值原樣加到體力值上或從中扣除。
    -- 使用AddHP方法時，可傳入正數值、負數值，以依照意圖執行功能。

    self.hp += addValue;

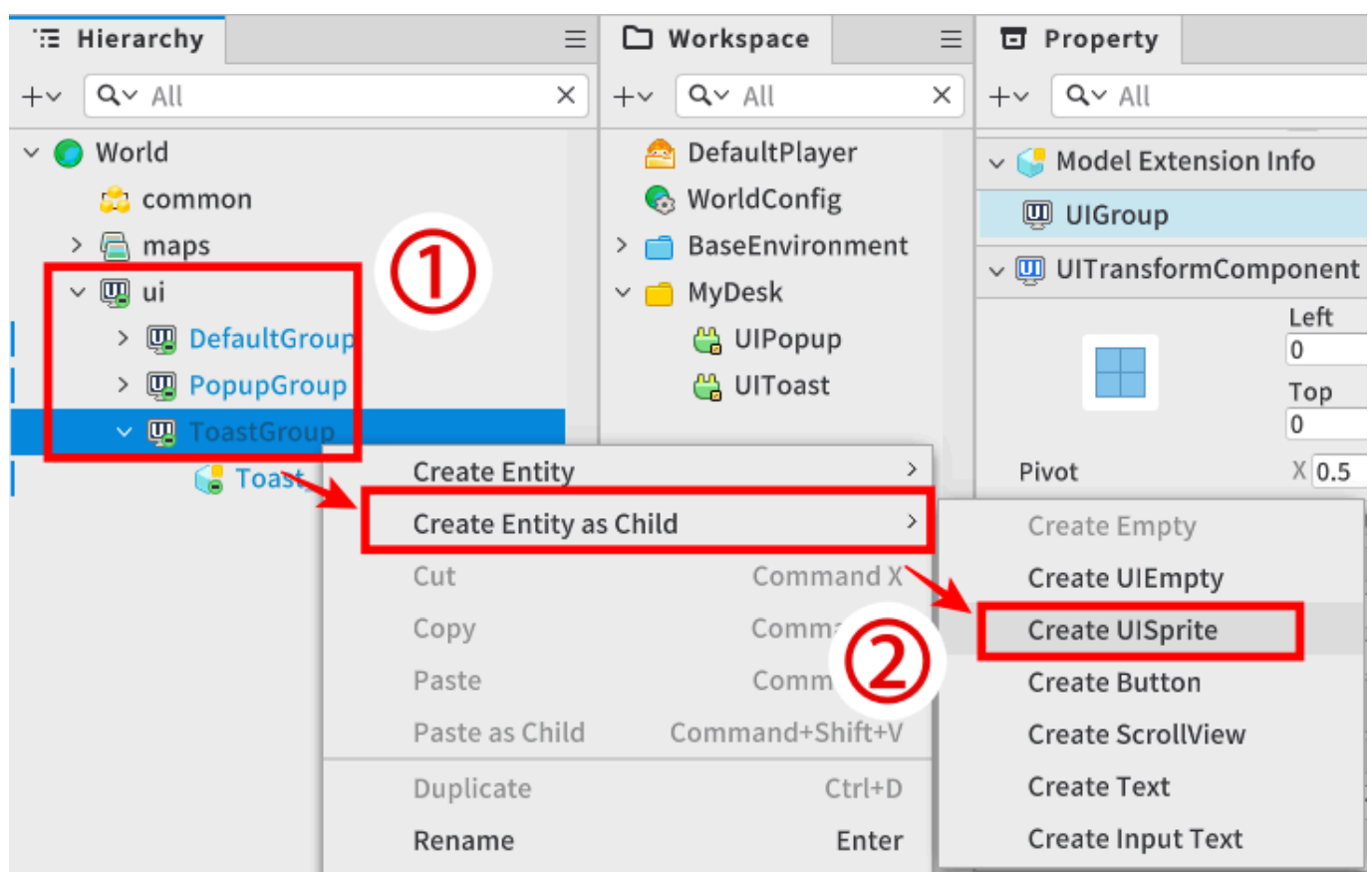
    -- 因體力值發生變動，因此請求更新UIManagerLogic所管理的滑桿資訊。
    _UIManagerLogic.SetHpSliderValue(self.hp)

    -- 當體力值變成0時（無法再繼續進行遊戲時）
    if self.hp <= 0 then
        -- 執行遊戲結束功能
    end
}

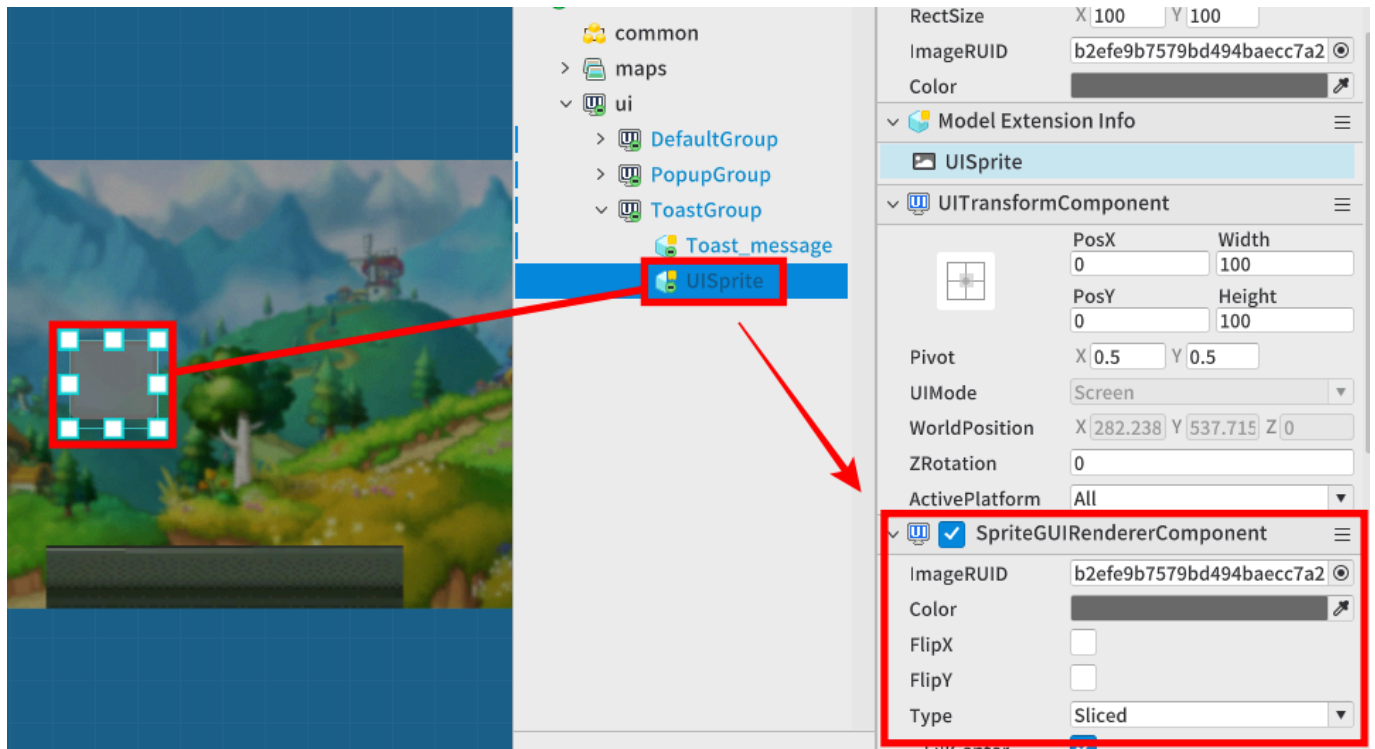
```

使用SliderComponent連動體力條(UI)

製作體力條UI

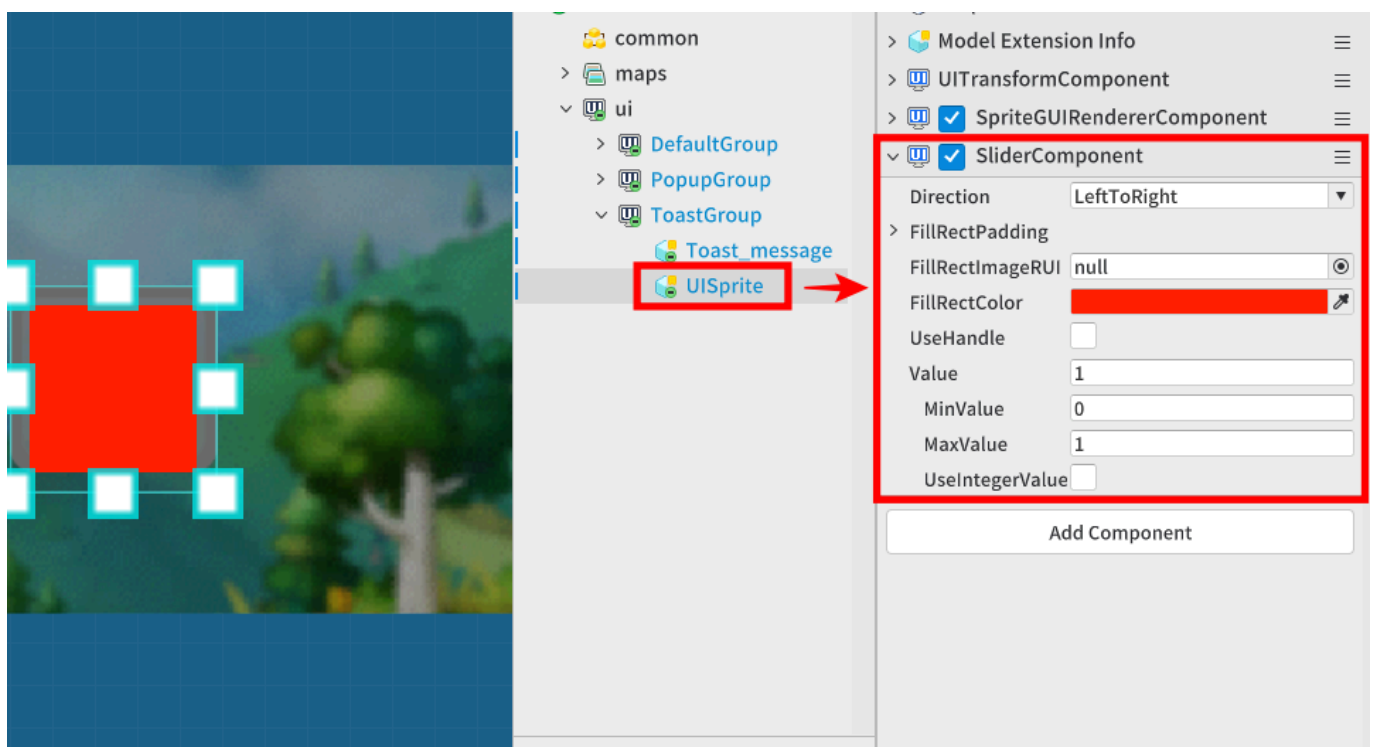


1. 在Ui子項目中選擇扮演 布角色的實體。
2. 在Create Entity as Child中選擇Create UISprite，建立將作 Sprite使用的UI。
3. 該UI是作 體力條背景的Sprite圖片。



- 在生成的實體中，在SpriteGUIRendererComponent設定作 體力條背景使用的圖片RUID與 色。

✅ 如果看不到生成的UI實體，請確認是否是實體預設登錄的Component，也就是UITransformComponent的座標設定錯誤。



- 為了能覆蓋在體力條背景上，建立SliderComponent。
- 設定FillRectColor 紅色， 確認是否正確覆蓋背景圖片。
- 嘗試變更元件內部屬性的Value，確認是否正確執行計量條功能。
- 以範例世界 例，是在最大體力 100的設定下製作。

透過UIManagerLogic控制體力條UI

- 作 計量條功能的實體已登錄在UIManagerLogic中。
- 若需要UIManagerLogic程式碼內容，可參考「2.理解Logic」單元附上的UIManagerLogic 本內容。
- 連接附加了SliderComponent的實體參照， 在GameLogic中每當體力變化時呼叫此邏輯，將最新體力 時覆寫到Slider的Value屬性。

UIManagerLogic的SetHpSliderValue方法製作範例

```
SliderComponent hpSlider;

void SetHpSliderValue(number value)
{
    -- 利用math.clamp方法設定將要設定的滑桿值，使其不會低於0或超過100。
    self.hpSlider.Value = math.clamp(value, 0, 100)
}
```

最低製作標準

- 本內部的體力資料(hp)變化，必須透過UIManagerLogic傳遞， 時同步到 面上的SliderComponent計量條UI。
- 當體力一低於0時，必須立刻偵測到，避免體力繼續降到負 ， 正常呼叫Game Over處理。

應用與進階應用

- 當體力降到一定數 以下(例如：20%)的危險狀態時，可以把SliderComponent的計量條 色改成紅色閃 ，藉此提供玩家視覺上的緊張感與警告回饋。

[💡 基礎學習]建構UI

影片時間軸

- 可在課程影片中以下時間點學習。
- 建構UI：01:31:40

學習目標

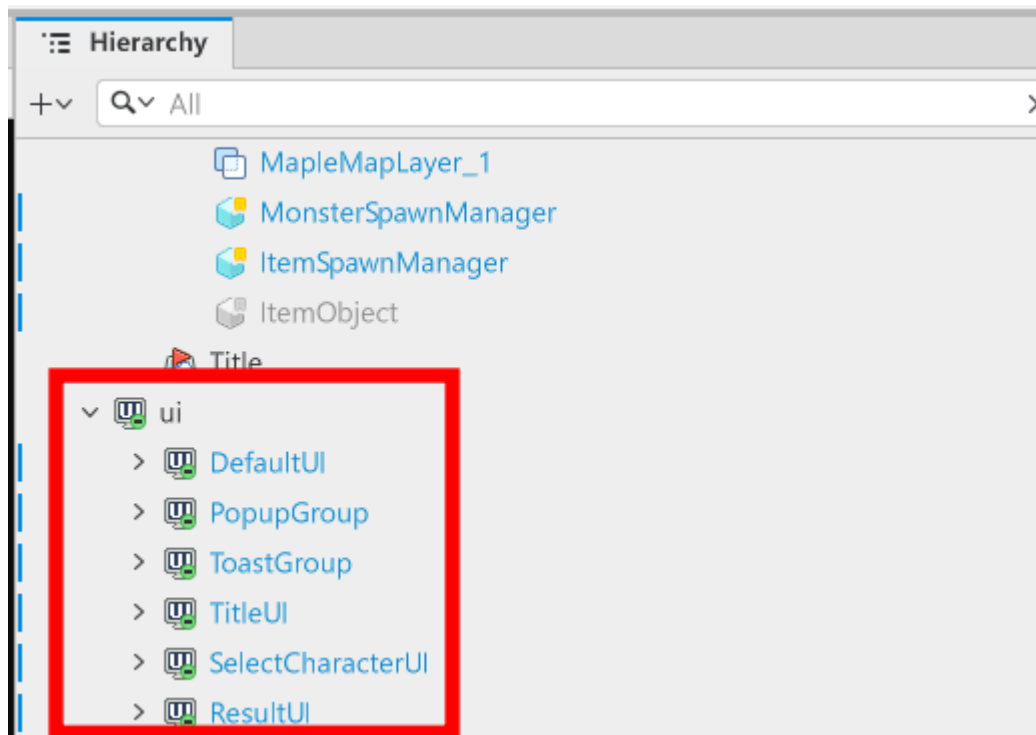
- 理解不是逐一控制個別UI元素，而是以單一UI實體單位加以分組， 依遊 狀態(標題、遊 中、結果)有效率切換 面的結構。

- 製作在遊 結束時呼叫的結果 面(Result Popup)， 學習如何連動GameLogic與UIManagerLogic，將目前分數與最高分數同步到UI文字。

觀看該影片後可嘗試執行的內容

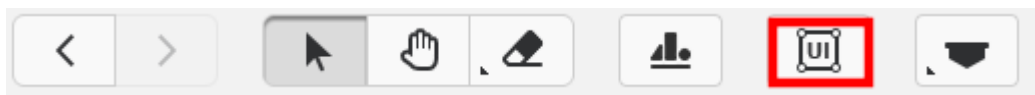
- 在Hierarchy視 中，將標題、遊 中、結果 面所需的UI元素，各自分組到對應的父實體(UI Group)底下， 透過開關最上層實體的Enable屬性，測試 面是否正常切換。
- 在遊 中故意撞上障 物讓遊 結束後，親自確認結果彈 中，剛才獲得的分數與 有最高分是否正確顯示在文字UI上。

理解UI Group

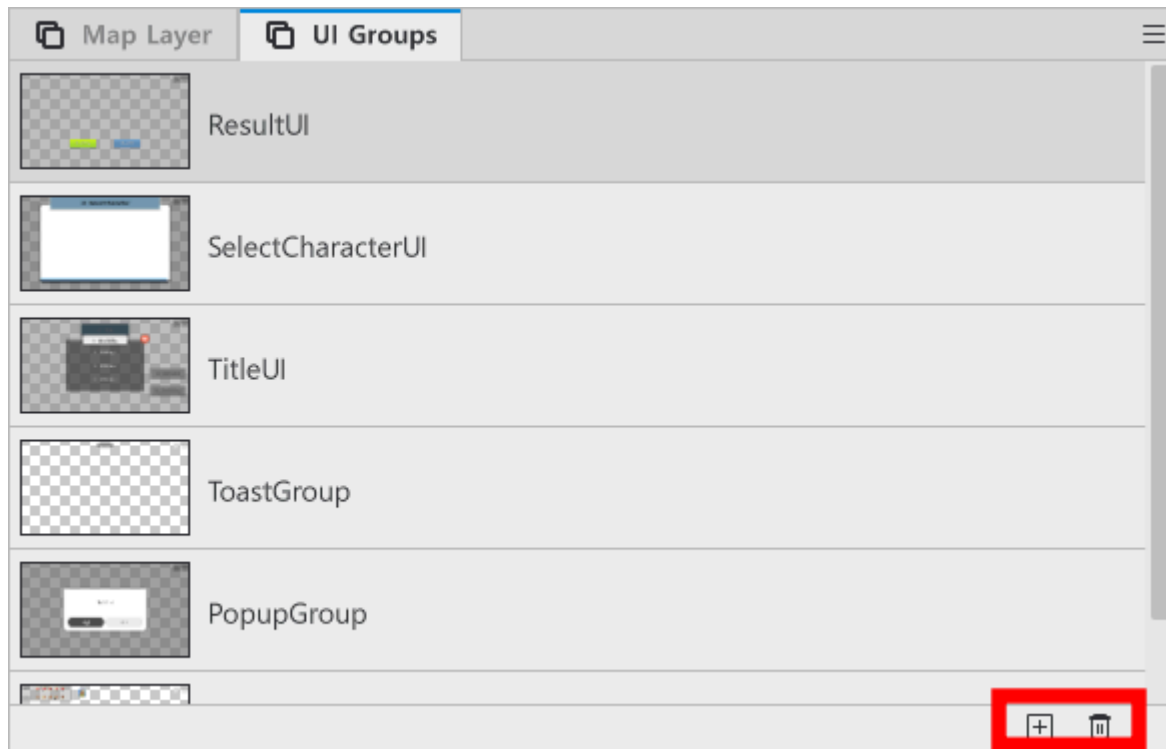


- 看Hierarchy視 下方時，可以看到以ui這個名稱建構的實體。
- 該實體是用來輸出 UI 的 UI Group。
- 可透過修改ui group來變更或製作要輸出的ui。

建立UI Group



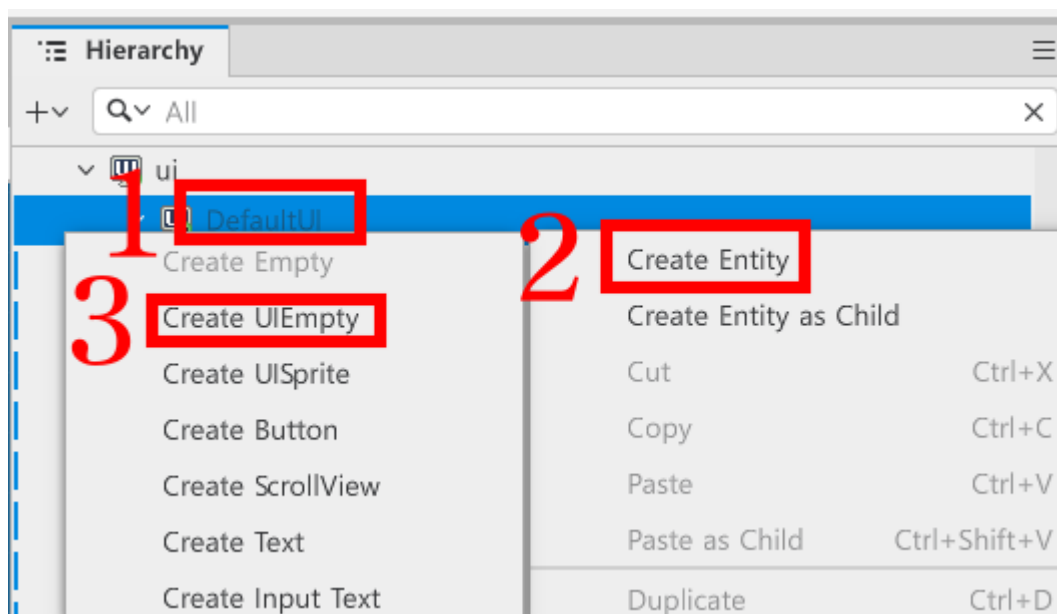
- 點擊編輯器上方的UI按 ，切換 UI編輯狀態。



- 可在編輯器視 下方的UI Groups中點擊 + 按 ，新增新的UI Group。
- 若有需要刪除的UI Groups，可點擊要刪除的UI Group後，再點擊 桶按 進行刪除。

製作空的UI實體

- 在UI中也可以像製作一般實體一樣製作實體。
- 差異在於，建立空實體時不是使用Create Empty，而是使用Create UIEmpty。



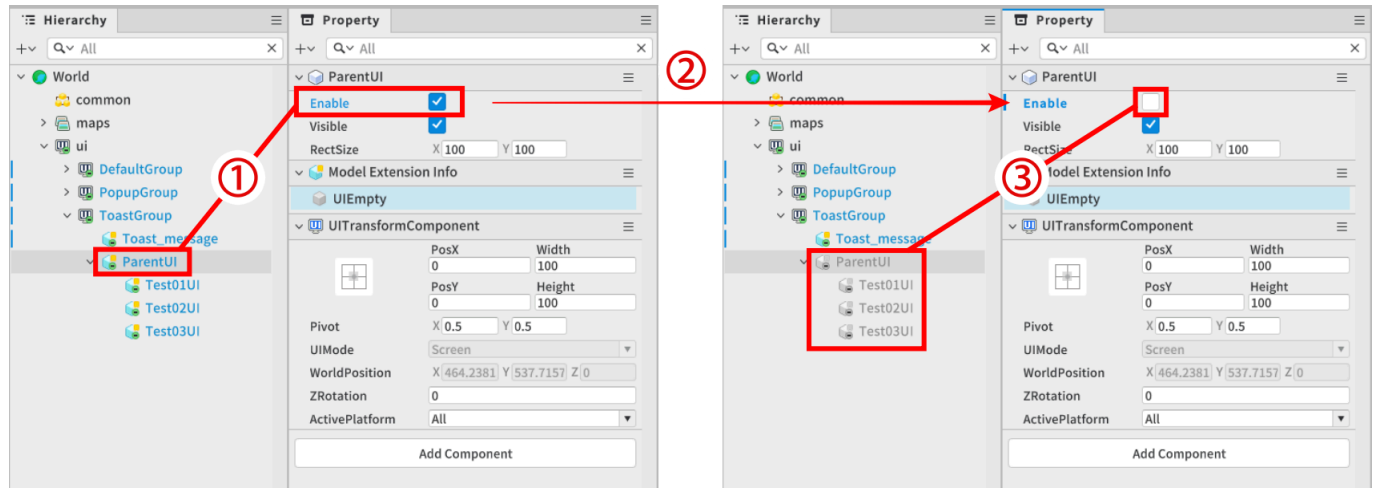
- 在Hierarchy視 中，對想新增實體的ui group按右鍵。
- 點擊Create Entity。點擊Create UIEmpty 可建立空的UI實體。

UI實體賦予Component

- UI實體也和一般實體一樣，可以賦予Component來執行額外功能。
- 賦予Component的方法與一般實體使用Add Component的方式相同。

- 在本單元中，將說明常用的Component。
 - **SpriteGUIRendererComponent**：這是用來在UI中顯示圖片的Component。在UI中輸出圖片時，比起SpriteRendererComponent，更建議使用該Component。
 - **TextGUIRendererComponent**：這是用來在UI中顯示文字的Component。
 - **ButtonComponent**：當UI需要執行按 功能時所使用的Component。

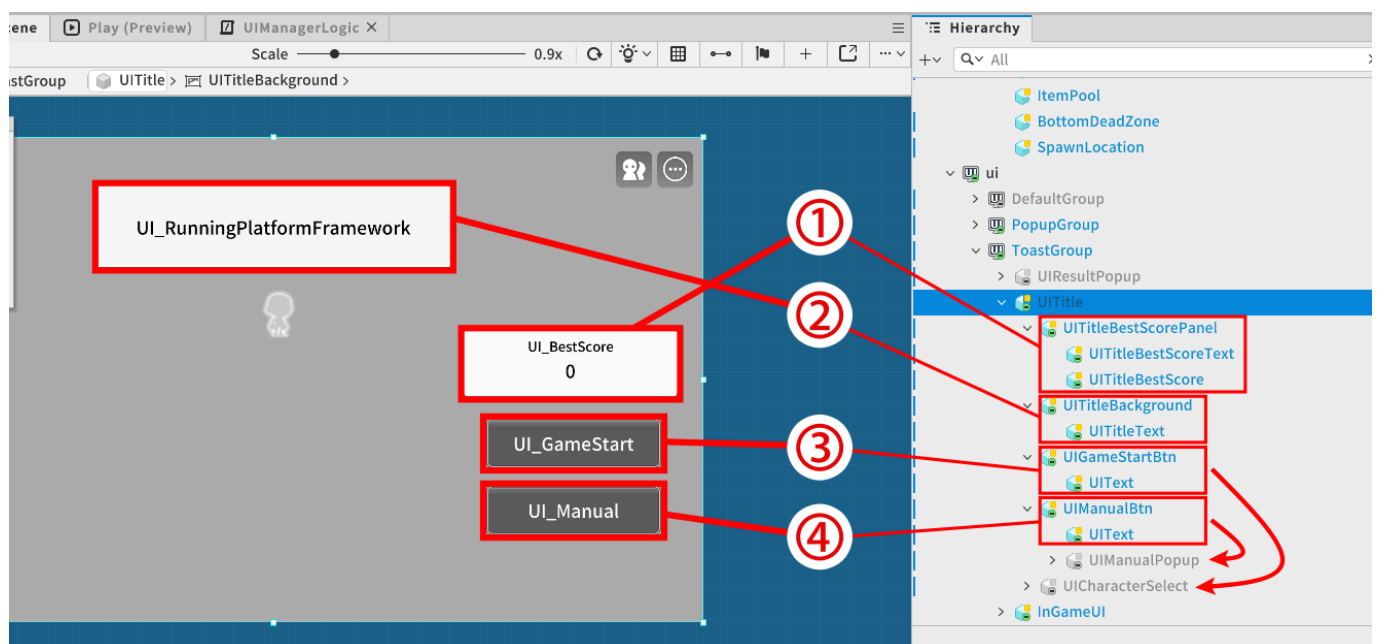
使用UI Groups進行群組單位控制



1. 當某個群組最上層的實體要素Enable true時，子實體可以個別切換Enable true或false來開 或關閉。
2. 將最上層實體要素的Enable狀態取消勾選 false。
3. 可確認當父實體停用時，子實體也會自動一 停用。

- 依實體特性，若停用父實體，屬於該實體的子實體也會自動一 停用。
- 使用這個特性，就不需要針對各個要素逐一控制不同遊 狀況下應顯示的UI，而是只要 用、停用群組單位的最上層實體，就能輕 控制。

製作Title UI Group



項目	群組	說明
1	UITitleBestScorePanel	這是在世界開始時顯示玩家最高分數值的區域。UITitleBestScoreText - 面板上方顯示「最高分數」的文字。UITitleBestScore - 僅用來顯示實際最高分數值的文字。
2	UITitle	這是在世界開始時顯示遊戲名稱、標題的區域。UITitleText - 輸出遊戲名稱的文字。
3	UIGameStartBtn	若玩家想開始遊戲，可透過該按鈕進行遊戲。該按鈕已連接為啟用角色選擇彈跳視窗(UICharacterSelect)。UIText - 顯示「開始遊戲」的文字。
4	UIManualBtn	當玩家想學習遊戲操作方式時，可透過該按鈕確認操作方式。該按鈕已連接為啟用操作說明彈窗(UIManualPopup)。UIText - 顯示「操作方式」的文字。

- TitleUI群組在世界開始時必須讓玩家可立 看到，因此預設Enable設 true。
- 直接 用TitleUI群組的時機，是正式遊 結束後，從結果 面再次切換回主選單的狀態。
- 直接停用TitleUI群組的時機，是玩家選擇角色 進入正式遊 狀態時。

在角色選擇視窗中，透過角色Slot(UICharacterSelect)的角色選擇按鈕開始遊戲的情況
UICharacterInfoComponent的CloseSelectPopup方法

```
void CloseSelectPopup()
{
    -- 儲存Title UI群組的最上層實體。
    local TitlePanel =
_EntityService:GetEntityByPath("/ui/ToastGroup/UITitle")

    -- 停用最上層實體。
    TitlePanel.Enable = false

    -- 請求GameLogic呼叫開始遊戲。
    _GameLogic:GameStart(self.characterId)
}
```

在遊戲結果視窗中，玩家透過「主選單」按鈕回到標題畫面的情況
UIManagerLogic的ReturnToMenu方法

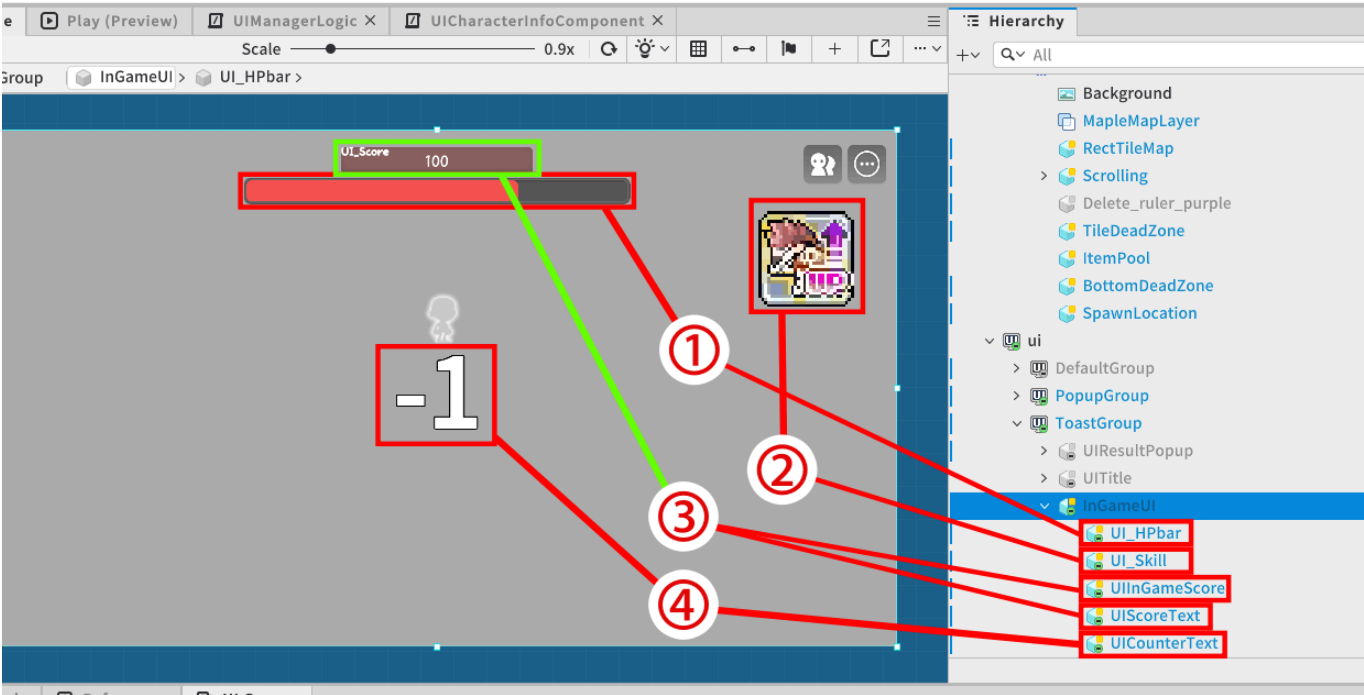
```
Entity mainTitlePanel
Entity resultPopup
Entity resultToMenuButton

void OnBeginPlay()
{
    -- 世界執行時，登錄結果彈跳視窗的「主選單」按鈕事件。

    self.resultToMenuButton:ConnectEvent(ButtonClickEvent,self.ReturnToMenu)
}
```

```
void ReturnToMenu()
{
    -- 停用結果彈跳視窗。
    self.resultPopup.Enable = false
    -- 啟用TitleUI最上層實體。
    self.mainTitlePanel.Enable = true
}
```

製作InGame UI Group



項目	群組	說明
1	UI_HPbar	這是玩家可確認角色目前體力狀態的UI。
2	UI_Skill	這是可確認角色能使用什麼技能，以及技能是否已使用的UI。
3	UI_Score	UIInGameScore - 這是會隨著遊戲進行，顯示玩家實際取得分數值的UI。 UIScoreText - 這是顯示該面板左上方「分數」文字的UI。
4	UICounterText	在正式遊戲開始前，會提示玩家N秒後捲動畫面即將開始。

- 直接 用InGameUI群組的時機，是玩家完成角色選擇 正式開始遊 時。
- 直接停用InGameUI群組的時機，是角色死亡後，從結果視 切換到主選單時。

玩家完成角色選擇後，呼叫遊戲中所需UI的情況
 UIManagerLogic的ShowPlayerInfo方法

```
SliderComponent hpSlider
Entity inGameCurrentScoreText
Entity inGameScoreText
SpriteGUIRendererComponent skillRenderer

void ShowPlayerInfo(boolean active)
{
    -- 將顯示體力計量條的實體切換為傳入的active狀態。
    self.hpSlider.Entity.Enable = active
    -- 將顯示玩家分數值的實體切換為傳入的active狀態。
    self.inGameCurrentScoreText.Entity.Enable = active
    -- 將顯示「分數」文字的UI元素切換為active狀態。
    self.inGameScoreText.Enable = active
    -- 將顯示角色技能資訊的實體切換為傳入的active狀態。
    self.skillRenderer.Entity.Enable = active
}
```

GameLogic的GameStart方法

```
void GameStart()
{
    _UIManagerLogic:ShowPlayerInfo(true)

    -- 執行遊戲中開始前所需的初始化作業。
}
```

在遊戲結果視窗中，玩家透過「主選單」按鈕回到標題畫面的情況
 UIManagerLogic的ReturnToMenu方法

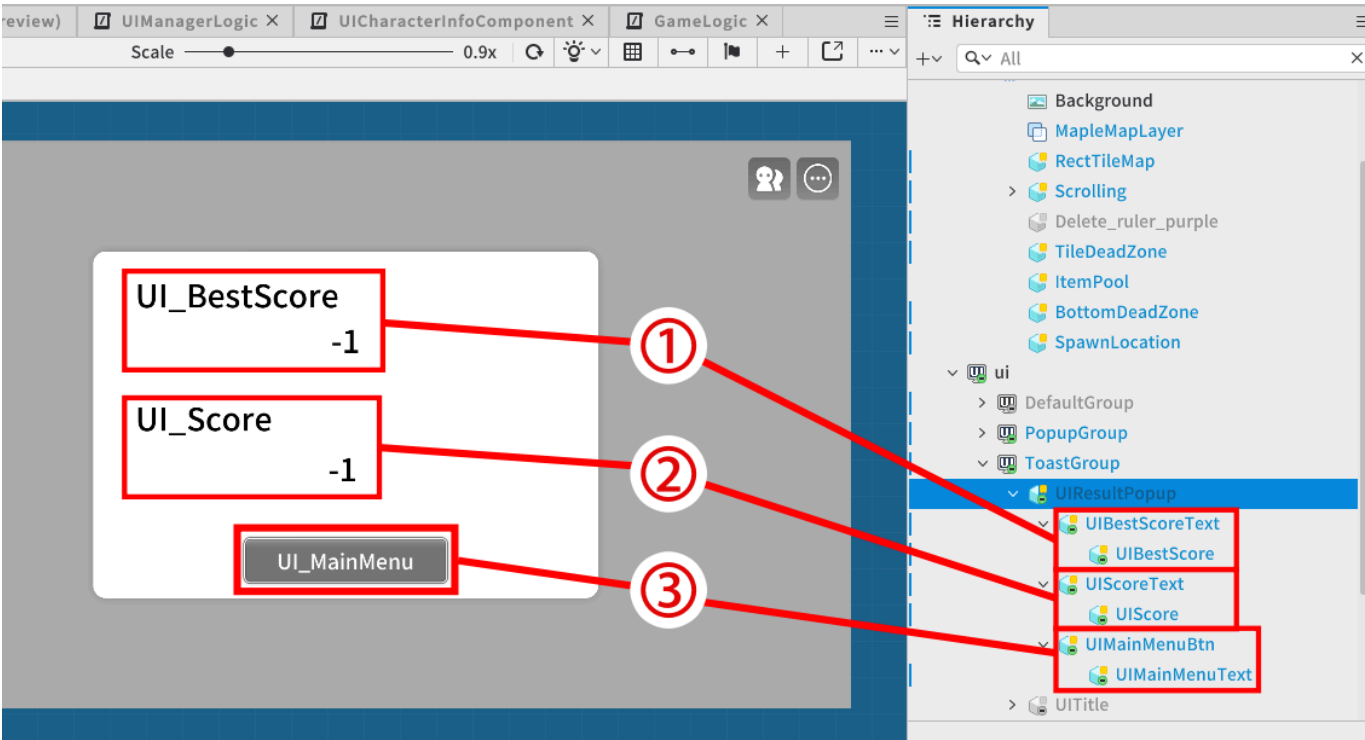
```
Entity mainTitlePanel

void OnBeginPlay()
{
    -- 世界執行時，登錄結果彈跳視窗的「主選單」按鈕事件。

    self.resultToMenuButton:ConnectEvent(ButtonClickEvent,self.ReturnToMenu)
}

void ReturnToMenu()
{
    -- 將遊戲中所需的UI元素全部停用。
    self:ShowPlayerInfo(false)
    -- 啟用TitleUI最上層實體。
    self.mainTitlePanel.Enable = true
}
```

製作Result Popup



項目	群組	說明
1	UIBestScoreText	這是結果視窗內顯示「最高分數」文字的UI。UIBestScore：這是用來顯示玩家原有最高分數值的文字UI。
2	UIScoreText	這是結果視窗內顯示「分數」文字的UI。UIScore：這是用來顯示玩家目前結束時點分數的文字UI。
3	UIMainMenuBtn	這是關閉結果彈跳視窗並回到Title狀態的按鈕UI。UIMainMenuText：這是顯示按鈕「主選單」文字的UI。

- 直接 用ResultPopup群組的時機，是正式在遊 中進行遊 後，角色已無法繼續遊玩的狀況。
- 直接停用ResultPopup群組的時機，是在結果視 內選擇「主選單」按 切換 Title狀態時。

遊戲中結束並顯示結果彈跳視窗的情況

UIManagerLogic的ShowResultPopup方法

— 登錄結果視窗內實際原有最高分數值的Text實體

Entity resultBestScoreText

— 登錄結果視窗內實際目前分數值的Text實體

Entity currentScoreText

```
void ShowResultPopup(string bestScore, string currentScore)
{
```

```

-- 將最高分數值文字變更為透過參數傳入的文字。
self.resultBestScoreText.TextComponent.Text = bestScore

-- 將目前分數值文字變更為透過參數傳入的文字。
self.currentScoreText.TextComponent.Text = currentScore

-- 啟用結果視窗。
self.resultPopup.Enable = true;
}

```

角色死亡時點呼叫的GameLogic之EndMainGame方法

```

number bestScore
number currentScore

void EndMainGame()
{
    -- 執行遊戲結束時所需功能

    _UIManagerLogic:ShowResultPopup(tostring(self.bestScore),tostring(self.currentScore))
}

```

在遊戲結果視窗中，玩家透過「主選單」按鈕回到標題畫面的情況
UIManagerLogic的ReturnToMenu方法

```

Entity resultPopup
Entity mainTitlePanel

void OnBeginPlay()
{
    -- 世界執行時，登錄結果彈跳視窗的「主選單」按鈕事件。

    self.resultToMenuButton:ConnectEvent(ButtonClickEvent,self.ReturnToMenu)
}

void ReturnToMenu()
{
    -- 停用結果彈跳視窗。
    self.resultPopup.Enable = false
    -- 啟用TitleUI最上層實體。
    self.mainTitlePanel.Enable = true
}

```

最低製作標準

- 必須讓可切換TitleUI、InGameUI、ResultPopup三種狀態的各按鈕 UI，以及開始遊戲、結束遊戲、回到主選單等功能都能正常運作。
- 在GameLogic中計算出的currentScore與bestScore，必須正確傳遞到各UI文字元件。

應用與進階應用

- ResultPopup出現時，不只是單純顯示，而是可加入像是尺寸變大或透明度變化、逐漸顯示的效果。
- 點擊按鈕時或結果視窗出現時，可播放效果音以提升操作手感。

[📖進階學習]製作障礙物物件

影片時間軸

- 可在課程影片中以下時間點學習。
- 製作障礙物物件：01:46:25

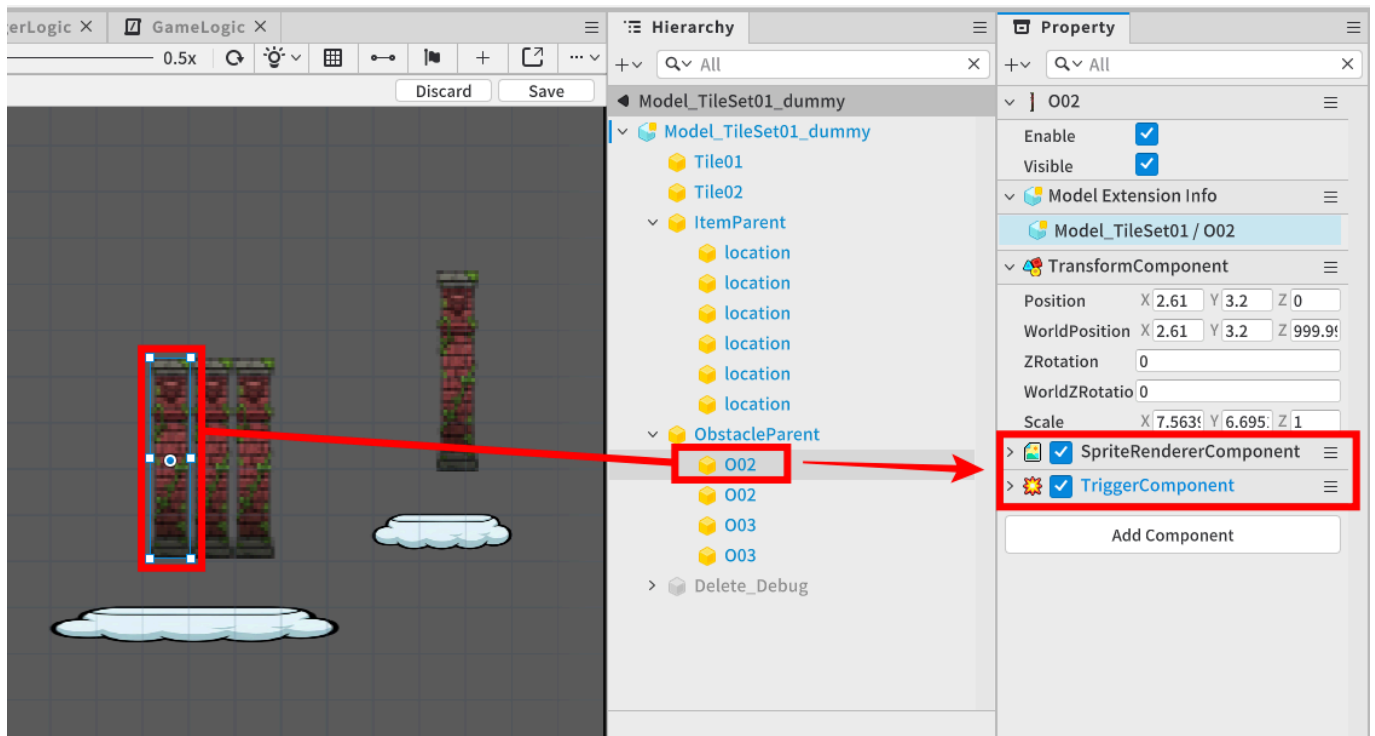
學習目標

- 理解預先在圖塊集內配置障礙物，在遊戲進行時依機率使用，以有效率管理資源的方式。
- 可製作在偵測障礙物撞時減少角色體力，讓角色進入一段時間無敵狀態的功能。

觀看該影片後可嘗試執行的內容

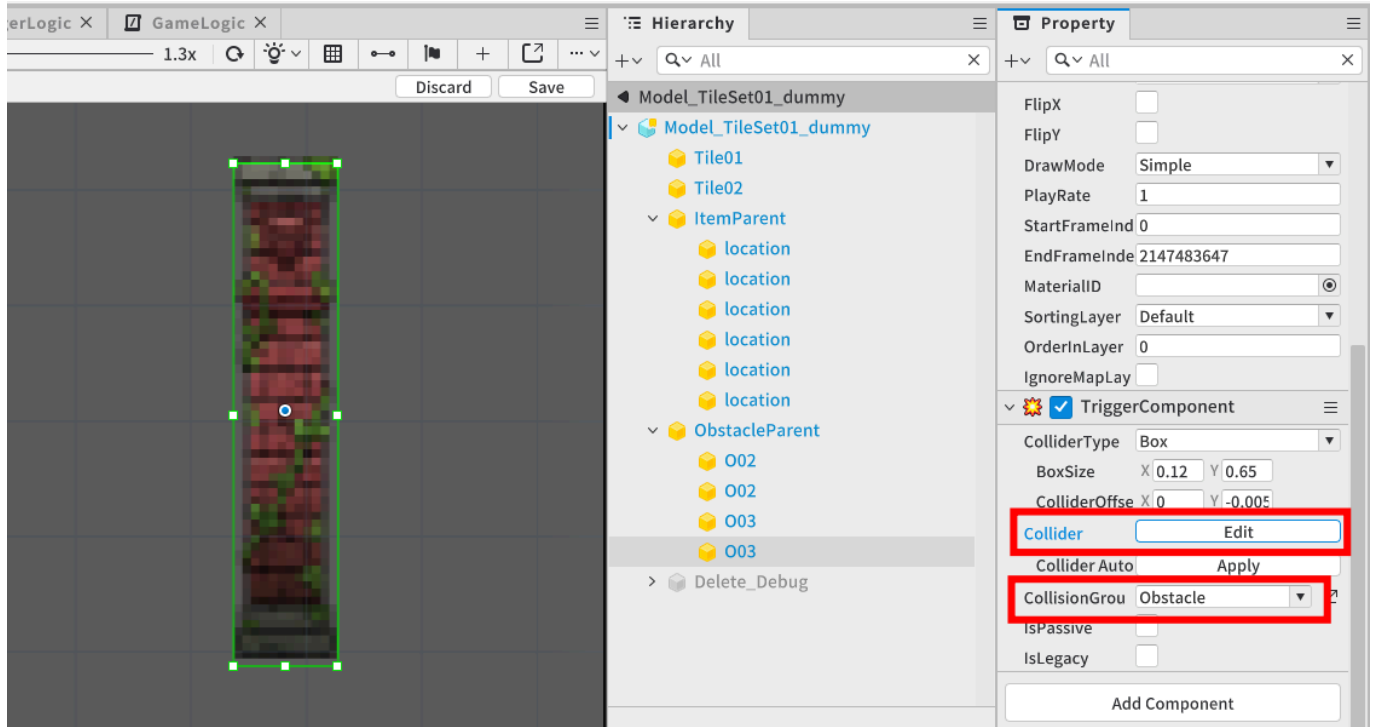
- 可將障礙物實體配置圖塊集的子實體，確認當圖塊移動時，障礙物也會在不需額外程式碼的情況下正常一起移動。
- 測試角色與障礙物撞時體力是否減少，以及在設定的無敵時間內是否不會受到額外傷害。

製作障礙物



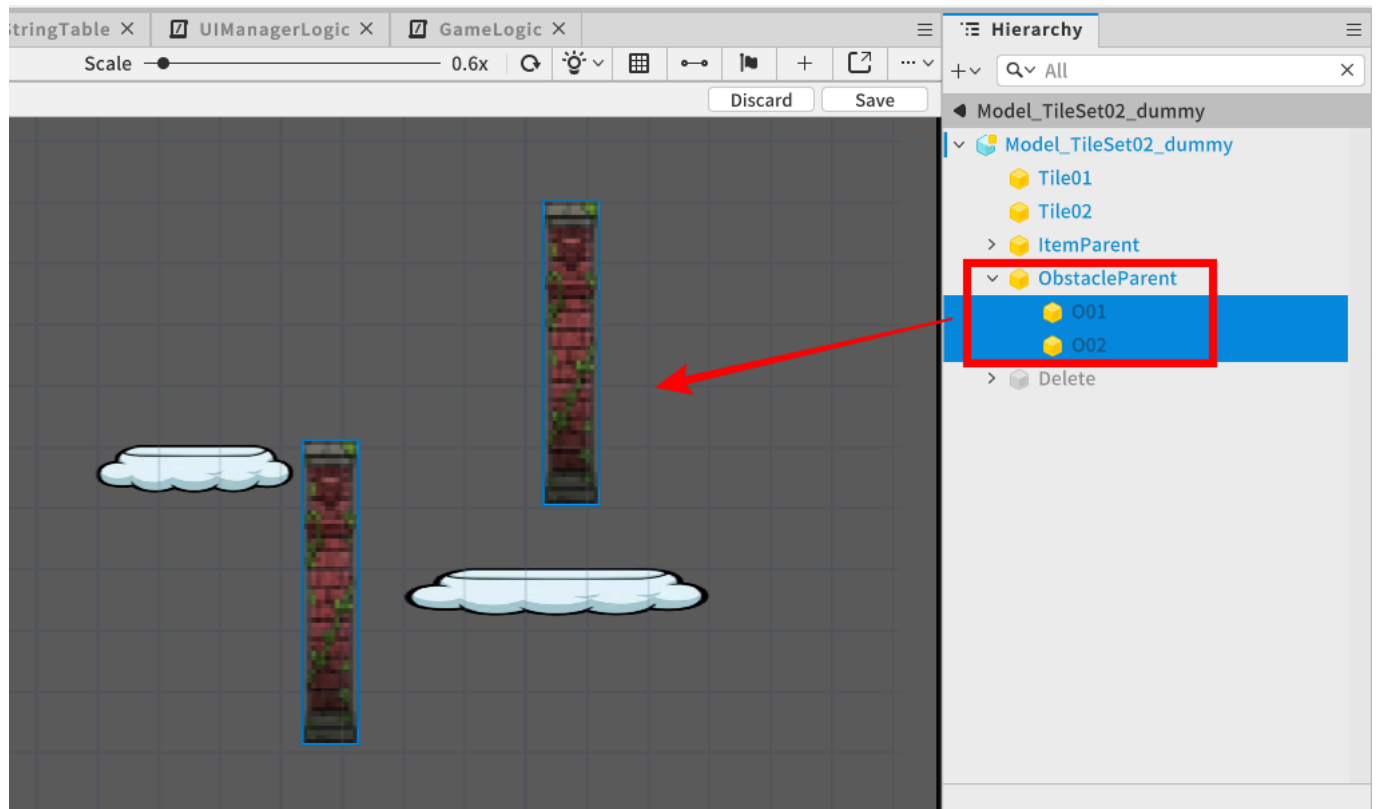
- 障 物實體所需元件 SpriteRendererComponent、TriggerComponent。
- SpriteComponent是用來顯示障 物外型。
- TriggerComponent是用來偵測與角色的 撞， 用來避免角色被阻、推開等實際物理 撞發生。

設定TriggerComponent



- 設定登錄在障 物實體上的TriggerComponent之Collider範圍。
- 在CollisionGroup中新增新的 撞群組「Obstacle」， 設定矩陣，讓他只偵測到與「Player」的 撞。
- 可藉由擴大或縮小Collider範圍來調整遊 難度。

障 物配置方式



-因 難以透過程式碼控制障 物位置，所以會採用事先配置障 物實體的方式製作。

- 會一邊測試玩家是否能閃避障 物，一邊配置與修改。
- 了讓障 物實體管理更順暢，會先準備障 物的父實體。

隨機配置障 物

- 如果每個圖塊都固定配置準備好的障 物，遊 會變得單調，因此會使用亂數(Random)功能，製作成讓障 物以不規則樣式出現。
- 將預先配置好的障 物隨機配置的功能，會在登錄在地板圖塊集的TileSet元件 本中，在圖塊集 用時進行。
- 隨機配置障 物功能可依下列程式碼處理。

TileSet腳本中的SpawnObstacle方法

-- 這是障礙物實體們的父實體。

Entity obstacleParent

```
void SpawnObstacle()
{
```

-- 確認圖塊集內預先安裝的障礙物物件數量。

-- 確認存放obstacle的父實體之子實體(障礙物實體)數量。

```
local spawnLocationCount = self.obstacleParent.Children.Count
```

-- 巡覽obstacleParent物件的子物件時

```
for index , child in pairs(self.obstacleParent.Children) do
```

-- 產生0到1之間的隨機數字。(製作意圖：若為0則配置障礙物0，若為1則配置障礙物X，機率50%)

```
local randomNumber = math.random(0,1)
```

-- /分支1/ 若數字為0

```

    if randomNumber == 0 then
        -- 啟用障礙物物件。
        child.Enable = true
        -- 覆蓋為符合階段編號的資源圖片。
        child.SpriteRendererComponent.SpriteRUID =
        _DataSetLogic.stageRenderInfoTable[_GameLogic.currentStage].ObstacleRUID

        -- /分支2/ 若數字不為0
    else
        -- 停用障礙物物件。
        child.Enable = false
    end
end
}

```

障 物受擊處理



- 若角色的TriggerComponent與障 物的TriggerComponent皆已正常登錄，則可使用彼此之間的Trigger事件。
- 在 了直接控制角色功能與動作而製作的CustomPlayer 本中，使用HandleTriggerEnterEvent事件方法。
- 若需要CustomPlayer程式碼內容，可參考「6.製作玩家角色」單元附上的CustomPlayer 本內容。
- 撞處理功能可依下列程式碼處理。

使用CustomPlayer事件處理器的碰撞處理

```

HandleTriggerEnterEvent(TriggerEnterEvent event)
{
    -- 儲存碰撞對象的實體資訊。
    local TriggerBodyEntity = event.TriggerBodyEntity

    -- 取出目標實體在CollisionGroup中設定的碰撞體群組ID編號。
}

```

```

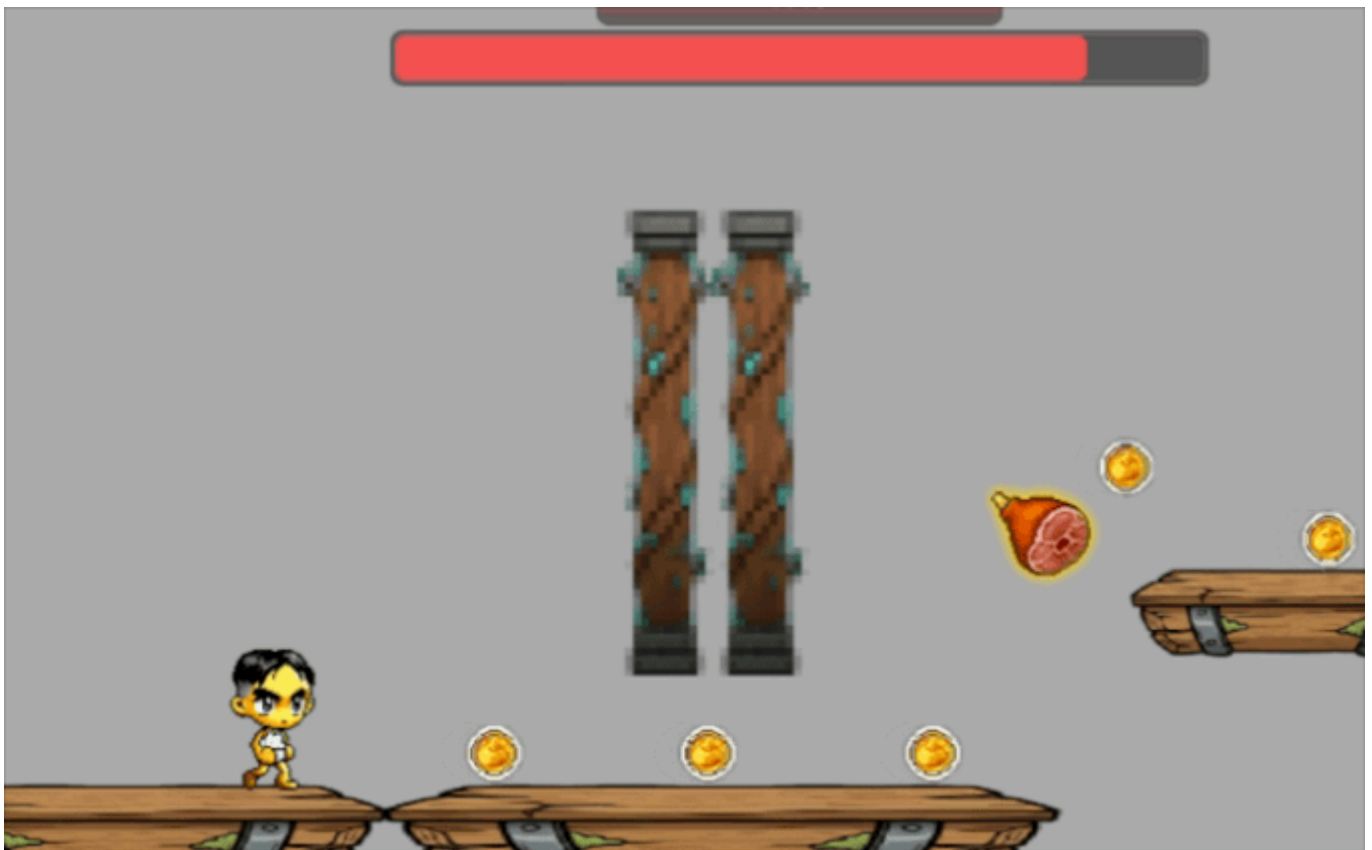
local TriggerGroup =
tostring(TriggerBodyEntity.TriggerComponent.CollisionGroup.Id)

-- -- 若該碰撞體的群組名稱與「Obstacle」相同
if _Config.triggerGroup[TriggerGroup] == "Obstacle" then
    -- 則執行碰撞判定處理。
    self:ContactObstacle()
end
}

void ContactObstacle()
{
    -- 要求GameLogic依照Config設定的障礙物受擊傷害值來減少體力。
    _GameLogic:AddHP(_Config.obstacleDamage)
}

```

新增角色無敵功能



- 若未給予一定的無敵時間，而是以HandleTriggerStayEvent事件方法製作障礙物受擊處理，則可能會每一幀都受到傷害，導致體力瞬間歸零。
- 透過在撞到障礙物後短時間內阻礙額外傷害，可提供玩家認知狀況重新調整動作的緩衝時間。
- 一定時間內無視受擊功能可依下列程式碼處理。

使用CustomPlayer的invincibleTimer

```

-- 宣告暫時無敵計時器變數。
number invincibleTimer
-- 宣告表示受擊狀態的變數。
boolean isHit

void ContactObstacle()
{
    -- 若暫時無敵計時器變數值大於0(暫時無敵計時器運作中的狀況)
    if self.invincibleTimer > 0 then
        -- 則不執行因障礙物造成的受擊處理並結束方法。
        return
    end

    -- 若上述條件式未被執行(暫時無敵計時器未運作的狀況)

    -- 則減少體力。
    _GameLogic:AddHP(_Config.obstacleDamage)

    -- 將受擊狀態切換為true。
    self.isHit = true;

    -- 為了讓玩家知道自己處於無敵狀態，將角色外型的alpha值減半，設為半透明狀態。
    self.Entity.AvatarRendererComponent:SetAlpha(0.5)

    -- 重新設定讓暫時無敵計時器可開始運作的無敵時間值。
    self.invincibleTimer = _Config.invincibleTime
}

void OnUpdate(number delta)
{
    -- 使用isHit值，避免在非isHit狀況下執行下方程式碼。
    if self.isHit == true then

        -- 若暫時無敵計時器值大於0，
        if self.invincibleTimer > 0 then
            -- 則將計時器值依delta值持續減少。
            self.invincibleTimer -= delta

            -- 若暫時無敵計時器已無剩餘持續時間，
        else
            -- 則為了讓玩家知道無敵狀態結束，將角色外型的alpha值恢復為原始值(1)，設為不透明狀態。
            self.Entity.AvatarRendererComponent:SetAlpha(1);
        end
    end
}

```

最低製作標準

- 障 物實體必須與地板圖塊同樣移動。
- 當玩家角色與障 物 撞時，必須發生受擊判定，且角色不能因障 物而被往後推。
- 因障 物受擊時，暫時無敵功能必須 用，且在計時器運作期間內不得受到傷害。

應用與進階應用

- 除了固定外型的障礙物外，也可新增帶有原地上下突起之動態動 的主動型障礙物。
- 可依遊 進行度(時間流逝)提高或改變障礙物生成機率，以調整遊 難度。

[📖進階學習]製作道具

影片時間軸

- 可在課程影片中以下時間點學習。
- 製作道具：01:55:02

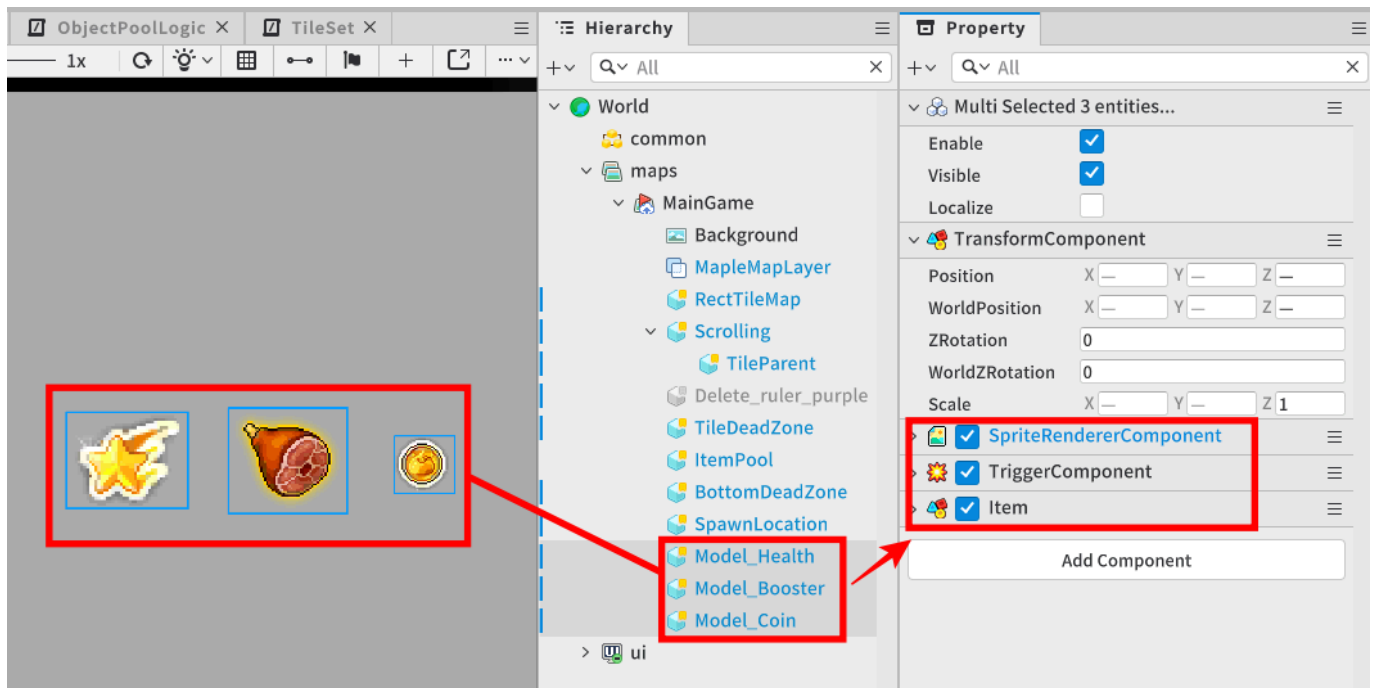
學習目標

- 理解在DataSet中定義道具固有資訊(效果、生成機率等)， 使用這些資訊而轉 表格形式管理的過程。
- 可使用物件池化系統，透過亂數運算依機率生成道具， 製作讓其依附在地圖圖塊的方法。
- 學習偵測角色與道具 撞， 透過ItemLogic依取得道具種類發動個別效果(增加分數、恢復體力、加速器等)的流程。

觀看該影片後可嘗試執行的內容

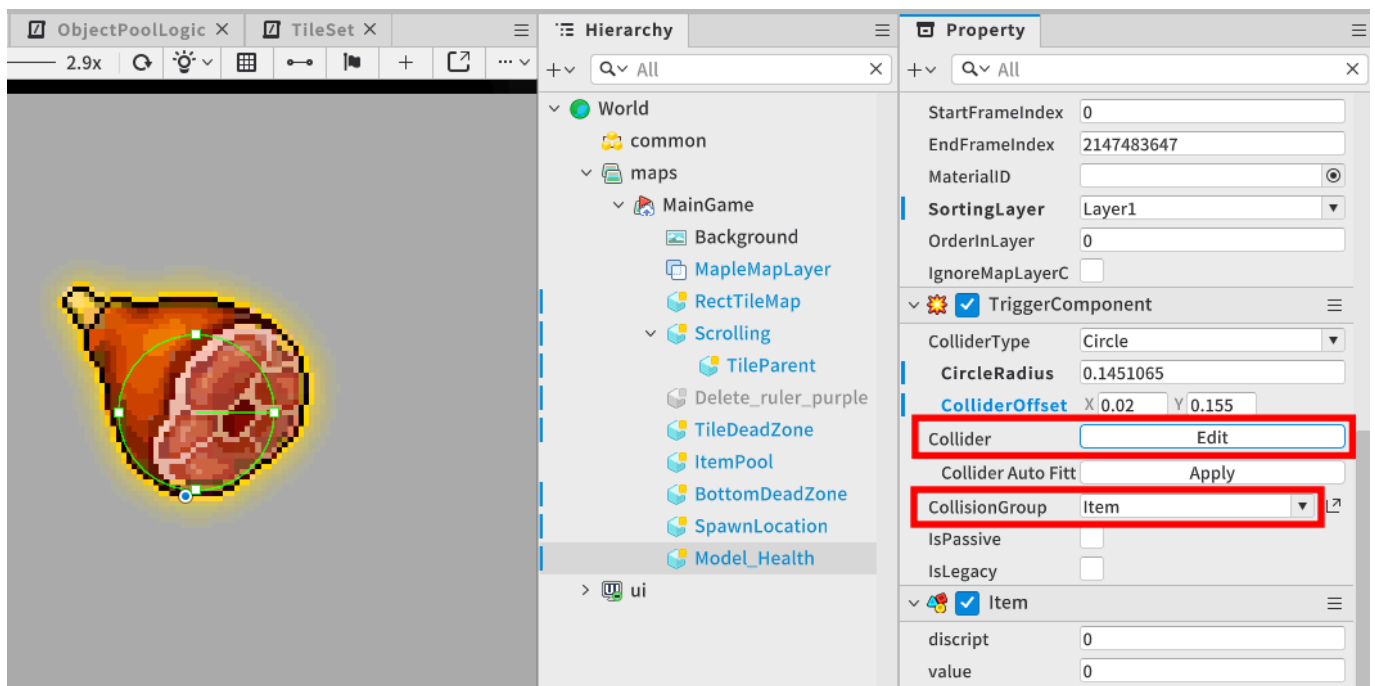
- 在ItemInfo DataSet中新增具新功能的道具(例如：無敵道具)資料， 確認世界執行時是否能透過邏輯正常登錄成表格。
- 嘗試極端提高或降低道具生成機率(SpawnRate) ， 測試在遊 中道具是否依預期頻率生成。
- 透過遊玩確認取得的道具數 是否正常更新到UI與系統中。

製作道具



- 在範例世界中，道具登錄的元件有SpriteRendererComponent、TriggerComponent、Item。
- SpriteRendererComponent是用來登錄道具外型的元件。
- TriggerComponent是用來偵測與角色的 撞， 用來防止角色被推開等實際物理 撞發生。
- Item元件是自行製作的 本，是在偵測到與角色 撞時，用來區分該道具是什 道具的元件。

設定TriggerComponent



- 若要變更道具 撞箱形狀，可使用ColliderType的下拉選單。
- 了與角色互動，在登錄於道具實體上的TriggerComponent中，使用Collider的Edit功能設定 撞範圍。
- 在CollisionGroup中新增新的 撞群組Item， 設定只與Player發生 撞。

製作Item 本

- Item 本只會單純登錄道具名稱與道具固有效果 變數。

- 如下撰寫程式碼。
- 該程式碼是以Plain Text 基準撰寫。

```
@Component
script Item extends Component

property number discript = 0 -- 這是包含道具基本功能、變數的元件。

property number value = 0 -- 這是道具所擁有的固有數值。

property string name = "" -- 這是用來儲存道具名稱的變數。

end
```

道具資料集

The screenshot shows a software interface with a table of item data and a workspace panel on the right. The table has columns for ID, Name, RUID, Value, SpawnRate, and EntryID. The workspace panel shows a hierarchy of UI elements, with 'ItemInfo' highlighted under 'DataTable'.

ID	Name	RUID	Value	SpawnRate	EntryID
1	Coin	5b9a6180cb99	50	1	model://babb1
2	Health	fe85b89044804	20	0.03	model://938f1e
3	Booster	88b07fd668e34	1.4	0.05	model://8c7e9l

對象	說明
Name	定義道具名稱。
RUID	記錄道具外型、資源ID值。
Value	定義各道具的固有效果值。
SpawnRate	定義並記錄道具生成機率，填寫時登錄0~1之間的值。
EntryID	儲存透過SpawnService重新生成新道具實體時所需原始(模型)的EntryID值。

- 學習如何將道具名稱、固有效果、生成機率、外型資料 入DataSet， 加以系統化管理。

登錄道具資料集表格

- 以DataSet製作的各道具資訊，會登錄到DataSetLogic的itemInfoTable中。
- 登錄成資料表的功能，是透過DataSetLogic 本的InitItemInfoTable方法流程進行。
- 若需要DataSetLogic程式碼內容，可參考「2.理解Logic」單元附上的DataSetLogic 本內容。

DataSetLogic의InitItemInfoTable方法

```
table itemInfoTable
```

```
void InitItemInfoTable()
```

```
{
    -- 這是ItemInfo表格的Row數。
    local rowCount = _DataService:GetRowCount("ItemInfo")

    -- 用來儲存各列資料的變數。
    local rowData

    -- 從1開始依資料集Row數進行巡覽。
    for i = 1 , rowCount , 1 do
        rowData =
        {
            -- 儲存道具名稱。
            Name = tostring(_DataService:GetCell(dataSetName,i,"Name")),
            -- 儲存道具的RUID。
            RUID = tostring(_DataService:GetCell(dataSetName,i,"RUID")),
            -- 儲存道具擁有的效果固有數值。
            Value = tonumber(_DataService:GetCell(dataSetName,i,"Value")),
            -- 儲存道具生成機率。
            SpawnRate =
            tonumber(_DataService:GetCell(dataSetName,i,"SpawnRate")),
            -- 儲存道具模型(Prefab)的EntryID值。
            EntryID =
            tostring(_DataService:GetCell(dataSetName,i,"EntryID"))
        }

        -- 將以rowData儲存的資料存入表格。
        self.itemInfoTable[i] = rowData
    end
}
```

使用道具資料

- 從ItemInfo DataSet儲存資料到DataSetLogic之後， 可再從DataSetLogic中取出資料。
- 在範例世界中，生成道具時需要用到道具資料表。
- 在物件池中，在世界開始時預先生成道具實體。
- 下列程式碼是ObjectPoolLogic在世界開始時生成道具的CreateItem方法。

ObjectPoolLogic的CreateItem

```
table itemDataSetIDTable = {"Coin" = 1, "Health" = 2, "Booster" = 3}
```

```
void CreateItem(string itemName, number createCount)
```

```
{
```

```
local tempTable

local dataSetID = self.itemDataSetIDTable[itemName]

local entryID = _DataSetLogic.itemInfoTable[dataSetID].EntryID
-- 載入資料集中目標道具模型的Name值。
local name = _DataSetLogic.itemInfoTable[dataSetID].Name
-- 載入資料集中目標道具模型的Value值。
local value = _DataSetLogic.itemInfoTable[dataSetID].Value

-- 進行目標表格有效性檢查。

-- /分支1/ 若目標道具池內沒有目標池(表格)
if isValid(self.itemPool[name]) == false or self.itemPool[name] == nil
then
    -- 建立新的臨時表格。
    tempTable = {}
    -- 將道具池中的內部池(表格)設為臨時表格。
    self.itemPool[name] = tempTable

-- /分支2/ 若目標道具池內已存在目標池
else
    -- 將臨時表格(池)設為既有使用中的表格。
    tempTable = self.itemPool[name]
end

-- 執行參數2 createCount次迴圈。
for i = 1, createCount, 1 do
    -- 使用SpawnService建立物件。
    local entity = _SpawnService:SpawnByModelId(entryID,name,Vector3.up *
100, self.ItemPoolParent)

    -- 進行建立物件的設定作業。

    -- 設定建立物件的道具固有數值。
    entity.Item.value = value
    -- 設定建立物件的道具name變數。
    entity.Item.name = name

    -- 將建立的實體插入應存放的表格(池)。
    table.insert(tempTable,entity)

    -- 停用生成的實體物件。
    entity.Enable = false
end
}
```

依道具生成機率 生道具

ID	Name	SpawnRate
1	Coin	1
2	Health	0.03
3	Booster	0.05

Random = 0.98 ⇒ "Coin"

Random = 0.04 ⇒ "Coin"

Random = 0.02 ⇒ "Health"

- 在範例世界中製作的道具共有3種，分別是可取得分數的金幣、可恢復角色體力的恢復型道具，以及可暫時提高捲動 面速度的加速器道具。
- 若所有道具都以相同機率生成，則可能會因恢復型道具太常出現，而 生遊 無法結束的平衡性問題。
- 也就是說，可以透過設定生成機率來調整道具的出現頻率。
- 在生成道具時，會抽出0到1之間的亂數(Random)，再與ItemInfo資料集中的SpawnRate 比較，若亂數小於或等於該 ，便設計成生成該道具。

TileSet的SpawnItem方法

```
-- 進行機率型生成。(產生隨機數字。)
local randomRate = math.random(0,100) * 0.01

-- 宣告要設定為Entity型態的區域變數。
---@type Entity
local tempEntity

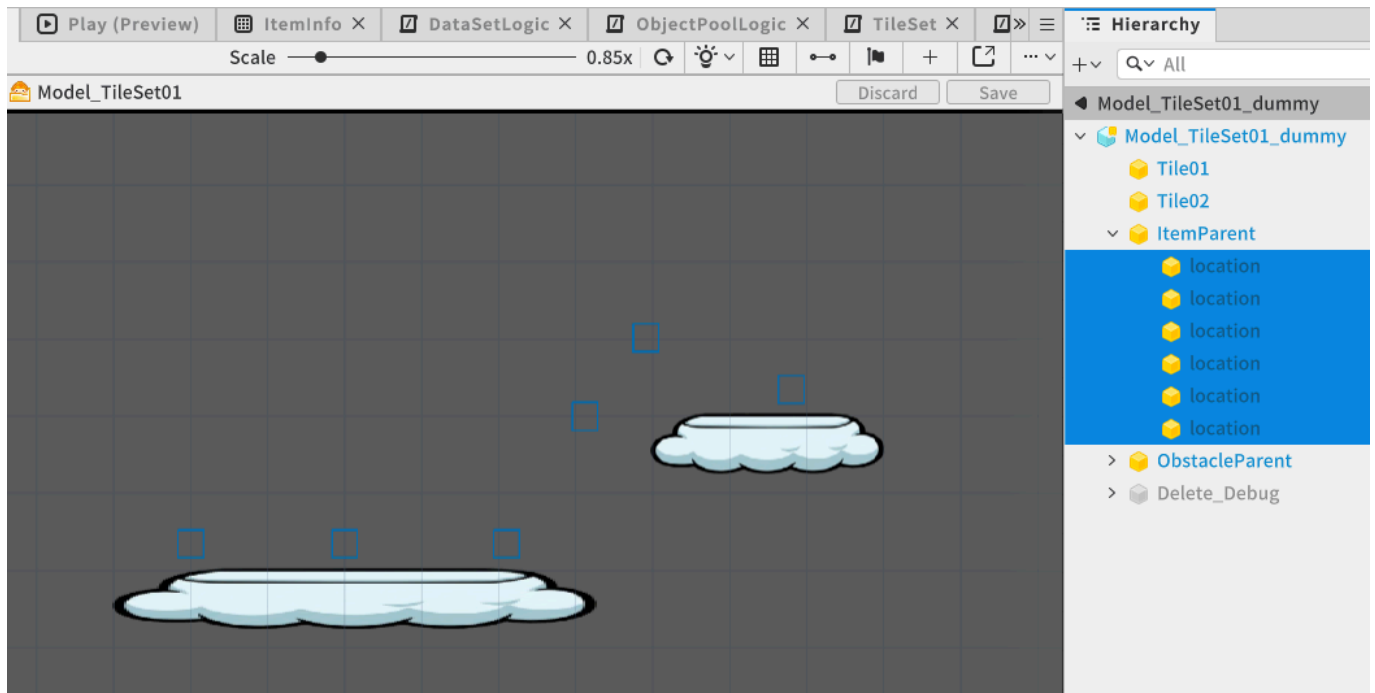
-- /分支1/ 若隨機機率值小於或等於healthSpawnRate值
if _DataSetLogic.itemInfoTable[2].SpawnRate >= randomRate then
-- 從物件池取得Health道具。
tempEntity = _ObjectPoolLogic.GetItem("Health")

-- /分支2/ 若隨機機率值小於或等於boosterSpawnRate
elseif _DataSetLogic.itemInfoTable[3].SpawnRate >= randomRate then
-- 從物件池取得Booster道具。
tempEntity = _ObjectPoolLogic.GetItem("Booster")

-- /分支3/ 若未符合其他已設定道具的生成機率，則生成基本道具(Coin)。
else
tempEntity = _ObjectPoolLogic.GetItem("Coin")

end
```

在指定位置配置道具



- 在地板圖塊集內，已登錄用來設定道具位置的ItemParent實體。
- 從物件池生成的道具，也必須與地板圖塊一樣一起移動。
- 生成的道具會先附加(Attach) 作 基準點實體的子實體(Child)，再將其本地座標初始化 原點(0, 0, 0)，以設定成與父層完全相同的位置。
- 依照ItemParent的子要素數量進行巡覽，重複生成道具。

TileSet的SpawnItem方法

Entity itemParent -- 這是用來設定道具位置之實體的父實體。

```
void SpawnItem
{
    -- 進行巡覽用來設定道具位置之實體的子要素。
    for index, child in pairs(self.itemParent.Children) do

        local tempEntity

        -- 比較隨機機率與SpawnRate值後，設定tempEntity的道具實體過程

        -- 將道具實體的父層設為child。
        tempEntity:AttachTo(child)

        -- 將道具實體的本地座標值設為原點，使其與父層處於相同位置。
        tempEntity.TransformComponent.Position = Vector3(0,0,0)
    }
}
```

最低製作標準

- 當道具生成時，必須附加在地板圖塊集上，隨著地圖捲動 面移動正常往 面左側移動。

- 當角色與道具發生Trigger 撞時，道具實體必須從 面上消失 回收至物件池。

應用與進階應用

- 隨著遊 進入後半段，除了正面效果道具之外，也可以機率性生成像是取得後反而遮蔽視野或 分的陷 道具，以調整遊 難度。
- 也可以企劃 以專用 本擴充新的機制道具，例如在一定時間內無視障 物 撞的護盾道具，或一次吸收周圍Coin的磁鐵道具等。.

[📖進階學習]製作資料儲存系統

影片時間軸

- 可在課程影片中以下時間點學習。
- 製作資料儲存系統：02:13:50

學習目標

- 理解如何使用DataStorageService將使用者的最高分儲存到伺服器， 在重新連線時將其載入。
- 掌握 了在客戶端與伺服器之間同步資料，而設定變數Sync屬性的原因。
- 理解如何使用OnSyncProperty事件，精準掌握從伺服器成功載入資料的時點， 安全地使用這些資料。

觀看該影片後可嘗試執行的內容

- 親自記錄新的最高分數 後，確認重新 動世界時是否能正常載入先前的分數資訊。

儲存伺服器端的最高分數

- 不會在每次分數上升時都與伺服器通訊，而是只在遊 結束等 正必要的時刻才儲存資料，以減少伺服器負擔。
- 由伺服器端管理的BestScore ，只有在GameOver(體力 0)時，目前分數高於原有最高分數的情況下才會更新。

更新伺服器端管理的BestScore值的方法
GameLogic中在遊戲結束時呼叫的EndGame方法

```
[Sync] number bestScore;  
[None] number currentScore;
```

```
[client] -- 僅客戶端方法  
void EndMainGame()  
{  
    -- 遊戲結束時呼叫的功能
```

```

-- 若目前分數高於原有最高分數
if self.currentScore > self.bestScore then
    -- 更新由客戶端 (本地環境) 管理的bestScore分數
    self.bestScore = self.currentScore
    -- 同時請求伺服器端更新分數
    self:UpdateScore(self.bestScore)

end
}

[server] -- 僅伺服器方法
void UpdateScore(number score)
{
    -- 更新由伺服器端管理而非本地端的bestScore值
    self.bestScore = score
}

```

透過資料儲存儲存使用者最高分數

- 使已有由伺服器端管理的資料，若未外儲存就再次連線到世界，仍無法載入使用者先前的資料。
- 若要儲存使用者資料或遊戲資料，需使用DataStorage服務。
- 建議在HandleUserLeaveEvent、HandleUserEnterEvent事件中使用DataStorage功能。

使用資料儲存來儲存使用者與遊戲資料
GameLogic的SaveScore方法

```

HandleUserLeaveEvent(UserLeaveEvent event)
{
    self:SaveScore(self.bestScore)
}

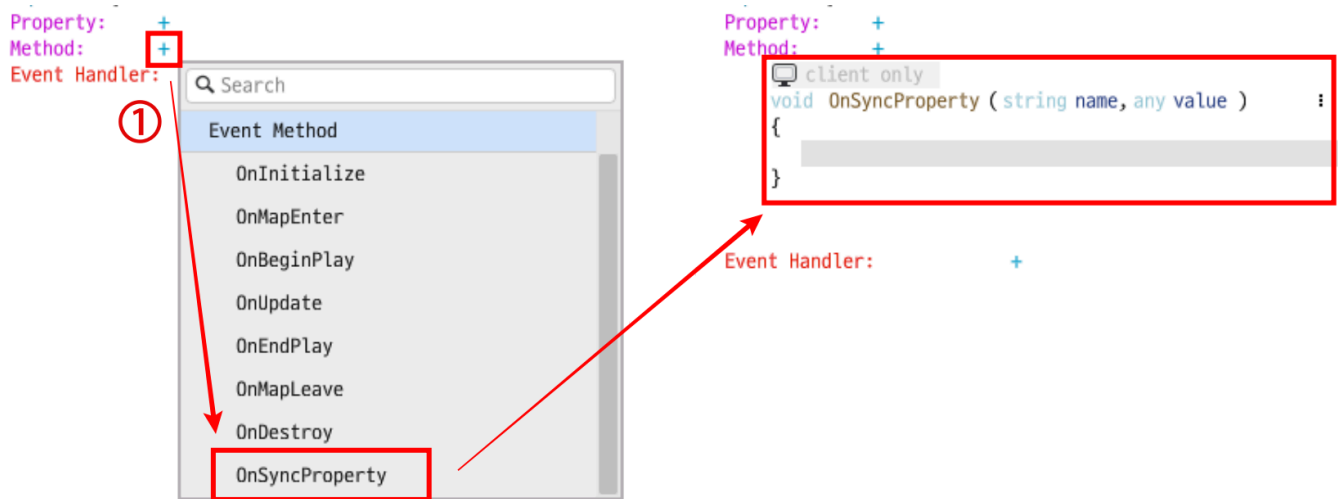
[server] -- 僅伺服器方法
void SaveScore(number score)
{
    local data = _DataStorageService:GetGlobalDataStorage("MyData")
    local errorCode = data:SetAndWait("BestScore", tostring(score))
}

```

載入最高分數

- 與資料儲存相同，在初次進入世界時只請求一次資料，以避免不必要的伺服器通訊。
- 如果玩家連線到世界時已有先前紀錄的最高分數，則載入該分數。

使用資料儲存與OnSyncProperty載入先前分數



1. 可在 本的Method分頁中，對「+」按 按右鍵後，透過OnSyncProperty建立方法。

- 若要載入使用者資料或遊 資料，需使用DataStorage服務。
- 設 Sync的變數，可透過OnSyncProperty方法精準偵測來自伺服器的 已完成同步的時點。

使用DataStorage來載入使用者與遊戲資料
GameLogic的LoadData方法

[Sync] number bestScore

```
HandleUserEnterEvent(UserEnterEvent event)
{
    self:LoadData()
}
```

[server] -- 僅伺服器方法

```
void LoadData()
{
    -- 利用資料儲存服務取得名為「MyData」的資料儲存空間。
    local data = _DataStorageService:GetGlobalDataStorage("MyData")

    -- 取得該資料儲存空間內 'BestScore' 鍵所對應的Value值。
    local errorCode,value = data:GetAndWait("BestScore")

    -- 登錄臨時分數值變數。
    local tempScore

    -- 若沒有已儲存資料
    if value == nil then
        -- 將臨時分數值設為0。
        tempScore = 0

    -- 若有已儲存資料
    else
        -- 將臨時分數值設為原有分數值。
        tempScore = tonumber(value)
    end
}
```

```
-- 更新bestScore值。
-- 因為是以Sync宣告的屬性，當值發生變更時，
-- 客戶端也會同步該值並自動呼叫OnSyncProperty方法。
self.bestScore = tonumber(tempScore)
}

[client only]
void OnSyncProperty(string name, any value)
{
    -- 當偵測到變數(屬性)的變更時

    -- 若變更的變數名稱是「bestScore」
    if name == "bestScore" then
        -- 則請求UIManagerLogic更新主標題畫面的最高分文字。
        _UIManagerLogic:UpdateTitleBestScore(tonumber(value))
    end
}
```

最低製作標準

- 當玩家在遊 中達成新的最高分， 完全結束世界後再次連線時，標題 面上顯示的先前最高分不能被重設 0，而必須正常顯示。
- 在伺服器分數資料同步完成之前，UI文字不可先行更新而顯示空 或錯誤，因此必須讓透過OnSyncProperty進行的時點控制能完美運作。

應用與進階應用

- 除了bestScore之外，也可擴充 將累積貨幣、角色 單、已進行的階段資訊等多種資料 在一起儲存的系統。